FHIR Retriever: Technical Documentation

This document describes the Python implementation line by line. Each source line is shown with its line number and followed by its role in the application. Blank lines are identified because they separate logical sections; test lines describe the behavior being verified.

System Overview

The project retrieves Patient, Condition, Observation, and Encounter resources from a FHIR R4 server. ‘fhir_retriever.py‘ pseudonymizes identifiers and dates, stores normalized fields in SQLite, exports CSV files, and maintains a restartable JSON cache. ‘fhir_api.py‘ exposes the stored data through FastAPI. ‘fhir_analyse.py‘ calculates descriptive statistics for numeric Observations. ‘etl.py‘ assembles retrieval and analysis for Docker. The remaining modules are focused tests, and ‘generate_documentation_pdf.py‘ creates this documentation.

Data and Execution Flow

1. The retriever requests paginated FHIR Bundles.
2. Resources are copied, identifiers are replaced with deterministic HMAC pseudonyms, and patient-linked dates are shifted.
3. ‘FHIRDatabase‘ stores normalized columns in ‘fhir_resources.sqlite3‘; it migrates older ‘resource_json‘ schemas when encountered.
4. CSV exports and the compressed resource cache are updated for restartability.
5. FastAPI reads the configured SQLite file directly, while analysis joins normalized Observation, Patient, and Encounter columns.

Configuration

Set ‘FHIR_PSEUDONYMIZATION_KEY‘ before retrieval. Local output defaults to ‘fhir_output/fhir_resources.sqlite3‘; Docker uses ‘/data/fhir_resources.sqlite3‘. The older ‘data/fhir.sqlite3‘ file is not referenced by the application.

etl.py

Purpose: Docker entry point that retrieves the configured cohort and runs the default analysis.

Line 1: ‘"""Compose entry point: retrieve the configured cohort and produce analysis output."""‘
Explanation: Executes part of the module’s workflow.

Line 2: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 3: ‘import os‘
Explanation: Imports a library or project dependency used by this module.

Line 4: ‘import sys‘
Explanation: Imports a library or project dependency used by this module.

Line 5: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 6: ‘from fhir_analyse import main as analyse_main‘
Explanation: Imports a library or project dependency used by this module.

Line 7: ‘from fhir_retriever import main as retrieve_main‘
Explanation: Imports a library or project dependency used by this module.

Line 8: ‘<blank>‘

Explanation: Blank line used to separate logical sections.

Line 9: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 10: ‘def main() -> int:‘
Explanation: Defines the main callable.

Line 11: ‘   output_dir = os.environ.get("FHIR_OUTPUT_DIR", "/data")‘
Explanation: Performs or configures an HTTP request to the FHIR service.

Line 12: ‘   # FHIR_COHORT_MODE=all loads multiple Patients (capped by FHIR_PATIENT_LIMIT); otherwise one Patient.‘
Explanation: Comment documenting the following code or an operational decision.

Line 13: ‘   if os.environ.get("FHIR_COHORT_MODE", "single").lower() == "all":‘
Explanation: Controls execution flow for the surrounding operation.

Line 14: ‘   sys.argv = [‘
Explanation: Assigns or computes a value used by later code.

Line 15: ‘   "fhir_retriever",‘
Explanation: Executes part of the module’s workflow.

Line 16: ‘   "--all-patients", "--limit", os.environ.get("FHIR_PATIENT_LIMIT", "10"),‘
Explanation: Performs or configures an HTTP request to the FHIR service.

Line 17: ‘   "--output-dir", output_dir,‘
Explanation: Executes part of the module’s workflow.

Line 18: ‘   "--condition", "--observation", "--encounter", "--refresh",‘
Explanation: Executes part of the module’s workflow.

Line 19: ‘   ]‘
Explanation: Executes part of the module’s workflow.

Line 20: ‘   else:‘
Explanation: Controls execution flow for the surrounding operation.

Line 21: ‘   patient_id = os.environ.get("FHIR_COHORT_PATIENT_ID", "sindhu-syn-000004")‘
Explanation: Performs or configures an HTTP request to the FHIR service.

Line 22: ‘   sys.argv = [‘
Explanation: Assigns or computes a value used by later code.

Line 23: ‘   "fhir_retriever",‘
Explanation: Executes part of the module’s workflow.

Line 24: ‘   "--patient-observations-encounters", patient_id,‘
Explanation: Executes part of the module’s workflow.

Line 25: ‘   "--output-dir", output_dir,‘
Explanation: Executes part of the module’s workflow.

Line 26: ‘   "--condition", "--observation", "--encounter", "--refresh",‘
Explanation: Executes part of the module’s workflow.

Line 27: ‘   ]‘
Explanation: Executes part of the module’s workflow.

Line 28: ‘   if retrieve_main():‘
Explanation: Controls execution flow for the surrounding operation.

Line 29: ‘   return 1‘
Explanation: Returns the computed result to the caller.

Line 30: ‘   sys.argv = ["fhir_analyse", "--obs-value", "1742-6", "--group-by", "sex",
"--output-dir", output_dir]‘
Explanation: Assigns or computes a value used by later code.

Line 31: ‘   return analyse_main()‘
Explanation: Returns the computed result to the caller.

Line 32: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 33: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 34: ‘if __name__ == "__main__":‘
Explanation: Controls execution flow for the surrounding operation.

Line 35: ‘   raise SystemExit(main())‘
Explanation: Raises an error so invalid or unavailable work is reported clearly.

fhir_analyse.py

Purpose: Reads normalized Observation data and calculates grouped numeric statistics.

Line 1: ‘"""Calculate Observation summary statistics from the local FHIR SQLite
database."""‘
Explanation: Executes part of the module’s workflow.

Line 2: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 3: ‘from __future__ import annotations‘
Explanation: Imports a library or project dependency used by this module.

Line 4: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 5: ‘import argparse‘
Explanation: Imports a library or project dependency used by this module.

Line 6: ‘import json‘
Explanation: Imports a library or project dependency used by this module.

Line 7: ‘import sqlite3‘
Explanation: Imports a library or project dependency used by this module.

Line 8: ‘import statistics‘
Explanation: Imports a library or project dependency used by this module.

Line 9: ‘from collections import defaultdict‘
Explanation: Imports a library or project dependency used by this module.

Line 10: ‘from datetime import date‘
Explanation: Imports a library or project dependency used by this module.

Line 11: ‘from pathlib import Path‘
Explanation: Imports a library or project dependency used by this module.

Line 12: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 13: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 14: ‘def age_band(birth_date: str | None, effective_date: str | None) -> str:‘
Explanation: Defines the age_band callable.

Line 15: ‘   if not birth_date:‘
Explanation: Controls execution flow for the surrounding operation.

Line 16: ‘   return "unknown"‘
Explanation: Returns the computed result to the caller.

Line 17: ‘   try:‘
Explanation: Controls execution flow for the surrounding operation.

Line 18: ‘   reference_date = date.fromisoformat((effective_date or
date.today().isoformat())[:10])‘
Explanation: Assigns or computes a value used by later code.

Line 19: ‘   born = date.fromisoformat(birth_date)‘
Explanation: Assigns or computes a value used by later code.

Line 20: ‘   except ValueError:‘
Explanation: Controls execution flow for the surrounding operation.

Line 21: ‘   return "unknown"‘
Explanation: Returns the computed result to the caller.

Line 22: ‘   age = reference_date.year - born.year - ((reference_date.month,
reference_date.day) < (born.month, born.day))‘
Explanation: Assigns or computes a value used by later code.

Line 23: ‘   if age < 0:‘
Explanation: Controls execution flow for the surrounding operation.

Line 24: ‘   return "unknown"‘
Explanation: Returns the computed result to the caller.

Line 25: ‘   lower = (age // 10) * 10‘
Explanation: Assigns or computes a value used by later code.

Line 26: ‘   return f"{lower}-{lower + 9}"‘
Explanation: Returns the computed result to the caller.

Line 27: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 28: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 29: ‘def summarize(values: list[float]) -> dict[str, float | int]:‘
Explanation: Defines the summarize callable.

Line 30: ‘   return {‘
Explanation: Returns the computed result to the caller.

Line 31: ‘   "count": len(values),‘
Explanation: Executes part of the module’s workflow.

Line 32: ‘   "mean": statistics.mean(values),‘
Explanation: Executes part of the module’s workflow.

Line 33: ‘   "median": statistics.median(values),‘
Explanation: Executes part of the module’s workflow.

Line 34: ‘   "standard_deviation": statistics.stdev(values) if len(values) > 1 else 0.0,‘
Explanation: Executes part of the module’s workflow.

Line 35: ‘   "minimum": min(values),‘
Explanation: Executes part of the module’s workflow.

Line 36: ‘   "maximum": max(values),‘
Explanation: Executes part of the module’s workflow.

Line 37: ‘   }‘
Explanation: Executes part of the module’s workflow.

Line 38: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 39: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 40: ‘def analyse(‘
Explanation: Defines the analyse callable.

Line 41: ‘   database_path: str | Path,‘
Explanation: Executes part of the module’s workflow.

Line 42: ‘   observation_value: str,‘
Explanation: Executes part of the module’s workflow.

Line 43: ‘   patient_id: str | None = None,‘
Explanation: Assigns or computes a value used by later code.

Line 44: ‘   group_by: str = "age-band",‘
Explanation: Assigns or computes a value used by later code.

Line 45: ‘) -> dict[str, dict[str, float | int]]:‘
Explanation: Executes part of the module’s workflow.

Line 46: ‘   """Return numeric Observation statistics grouped by age band, sex, or encounter type."""‘
Explanation: Executes part of the module’s workflow.

Line 47: ‘   group_columns = {‘
Explanation: Assigns or computes a value used by later code.

Line 48: ‘   "age-band": None,‘
Explanation: Executes part of the module’s workflow.

Line 49: ‘   "sex": "p.gender",‘
Explanation: Executes part of the module’s workflow.

Line 50: ‘   "encounter-type": "e.encounter_type",‘
Explanation: Executes part of the module’s workflow.

Line 51: ‘   }‘
Explanation: Executes part of the module’s workflow.

Line 52: ‘   if group_by not in group_columns:‘
Explanation: Controls execution flow for the surrounding operation.

Line 53: ‘   raise ValueError(f"Unsupported group: {group_by}")‘
Explanation: Raises an error so invalid or unavailable work is reported clearly.

Line 54: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 55: ‘   connection = sqlite3.connect(database_path)‘
Explanation: Opens a SQLite connection to the configured database.

Line 56: ‘   try:‘
Explanation: Controls execution flow for the surrounding operation.

Line 57: ‘   rows = connection.execute(‘
Explanation: Runs a SQL statement or schema script against SQLite.

Line 58: ‘   "SELECT o.value, o.effective_date_time, p.birth_date, p.gender,
e.encounter_type "‘
Explanation: Executes part of the module’s workflow.

Line 59: ‘   "FROM observations o "‘
Explanation: Executes part of the module’s workflow.

Line 60: ‘   "JOIN patients p ON p.patient_id = o.patient_id "‘
Explanation: Assigns or computes a value used by later code.

Line 61: ‘   "LEFT JOIN encounters e ON e.encounter_id = o.encounter_id "‘
Explanation: Assigns or computes a value used by later code.

Line 62: ‘   "WHERE (o.observation_subtype = ? OR o.observation_code = ?) "‘
Explanation: Assigns or computes a value used by later code.

Line 63: ‘   "AND (? IS NULL OR o.patient_id = ?)",‘
Explanation: Assigns or computes a value used by later code.

Line 64: ‘   (observation_value, observation_value, patient_id, patient_id),‘
Explanation: Executes part of the module’s workflow.

Line 65: ‘   ).fetchall()‘
Explanation: Executes part of the module’s workflow.

Line 66: ‘   finally:‘
Explanation: Controls execution flow for the surrounding operation.

Line 67: ‘   connection.close()‘
Explanation: Executes part of the module’s workflow.

Line 68: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 69: ‘   grouped: dict[str, list[float]] = defaultdict(list)‘
Explanation: Assigns or computes a value used by later code.

Line 70: ‘   for value, effective_date, birth_date, gender, encounter_type in rows:‘
Explanation: Controls execution flow for the surrounding operation.

Line 71: ‘   try:‘
Explanation: Controls execution flow for the surrounding operation.

Line 72: ‘   numeric_value = float(value)‘
Explanation: Assigns or computes a value used by later code.

Line 73: ‘   except (TypeError, ValueError):‘
Explanation: Controls execution flow for the surrounding operation.

Line 74: ‘   continue‘
Explanation: Executes part of the module’s workflow.

Line 75: ‘   if group_by == "age-band":‘
Explanation: Controls execution flow for the surrounding operation.

Line 76: ‘   group = age_band(birth_date, effective_date)‘
Explanation: Assigns or computes a value used by later code.

Line 77: ‘   elif group_by == "sex":‘
Explanation: Controls execution flow for the surrounding operation.

Line 78: ‘   group = gender or "unknown"‘
Explanation: Assigns or computes a value used by later code.

Line 79: ‘   else:‘
Explanation: Controls execution flow for the surrounding operation.

Line 80: ‘   group = encounter_type or "unknown"‘
Explanation: Assigns or computes a value used by later code.

Line 81: ‘   grouped[group].append(numeric_value)‘
Explanation: Executes part of the module’s workflow.

Line 82: ‘   return {group: summarize(values) for group, values in
sorted(grouped.items())}‘
Explanation: Returns the computed result to the caller.

Line 83: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 84: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 85: ‘def main() -> int:‘
Explanation: Defines the main callable.

Line 86: ‘   parser = argparse.ArgumentParser(description=__doc__)‘
Explanation: Assigns or computes a value used by later code.

Line 87: ‘   parser.add_argument("--obs-value", required=True, help="Observation
subtype or LOINC code")‘
Explanation: Assigns or computes a value used by later code.

Line 88: ‘   parser.add_argument("--patient-id", help="Pseudonymized patient ID, such
as PAT-...")‘
Explanation: Assigns or computes a value used by later code.

Line 89: ‘   parser.add_argument(‘
Explanation: Executes part of the module’s workflow.

Line 90: ‘   "--group-by",‘
Explanation: Executes part of the module’s workflow.

Line 91: ‘   choices=("age-band", "sex", "encounter-type"),‘
Explanation: Assigns or computes a value used by later code.

Line 92: ‘   default="age-band",‘
Explanation: Assigns or computes a value used by later code.

Line 93: ‘   )‘
Explanation: Executes part of the module’s workflow.

Line 94: ‘   parser.add_argument("--output-dir", default="fhir_output")‘
Explanation: Assigns or computes a value used by later code.

Line 95: ‘   parser.add_argument("--database")‘
Explanation: Executes part of the module’s workflow.

Line 96: ‘   arguments = parser.parse_args()‘
Explanation: Assigns or computes a value used by later code.

Line 97: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 98: ‘   database_path = arguments.database or Path(arguments.output_dir) / "fhir_resources.sqlite3"‘
Explanation: Assigns or computes a value used by later code.

Line 99: ‘   result = analyse(database_path, arguments.obs_value, arguments.patient_id, arguments.group_by)‘
Explanation: Assigns or computes a value used by later code.

Line 100: ‘   output = json.dumps(result, indent=2)‘
Explanation: Assigns or computes a value used by later code.

Line 101: ‘   output_directory = Path(arguments.output_dir)‘
Explanation: Assigns or computes a value used by later code.

Line 102: ‘   output_directory.mkdir(parents=True, exist_ok=True)‘
Explanation: Assigns or computes a value used by later code.

Line 103: ‘   (output_directory / "analysis.txt").write_text(output + "\n", encoding="utf-8")‘
Explanation: Assigns or computes a value used by later code.

Line 104: ‘   print(output)‘
Explanation: Executes part of the module’s workflow.

Line 105: ‘   return 0‘
Explanation: Returns the computed result to the caller.

Line 106: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 107: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 108: ‘if __name__ == "__main__":‘
Explanation: Controls execution flow for the surrounding operation.

Line 109: ‘   raise SystemExit(main())‘
Explanation: Raises an error so invalid or unavailable work is reported clearly.

fhir_api.py

Purpose: FastAPI service exposing health, Patient, Observation, and analysis endpoints.

Line 1: ‘"""HTTP API for the de-identified FHIR SQLite database."""‘
Explanation: Executes part of the module’s workflow.

Line 2: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 3: ‘from __future__ import annotations‘
Explanation: Imports a library or project dependency used by this module.

Line 4: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 5: ‘import os‘
Explanation: Imports a library or project dependency used by this module.

Line 6: ‘import sqlite3‘
Explanation: Imports a library or project dependency used by this module.

Line 7: ‘from pathlib import Path‘
Explanation: Imports a library or project dependency used by this module.

Line 8: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 9: ‘from fastapi import FastAPI, HTTPException, Query‘
Explanation: Imports a library or project dependency used by this module.

Line 10: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 11: ‘from fhir_analyse import analyse‘
Explanation: Imports a library or project dependency used by this module.

Line 12: ‘from fhir_retriever import get_all_patients‘
Explanation: Imports a library or project dependency used by this module.

Line 13: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 14: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 15: ‘DATABASE_PATH = Path(os.environ.get("FHIR_DATABASE_PATH",
"fhir_output/fhir_resources.sqlite3"))‘
Explanation: Performs or configures an HTTP request to the FHIR service.

Line 16: ‘app = FastAPI(title="De-identified FHIR API", version="1.0.0")‘
Explanation: Assigns or computes a value used by later code.

Line 17: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 18: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 19: ‘def connection() -> sqlite3.Connection:‘
Explanation: Defines the connection callable.

Line 20: ‘   DATABASE_PATH.parent.mkdir(parents=True, exist_ok=True)‘
Explanation: Assigns or computes a value used by later code.

Line 21: ‘   database = sqlite3.connect(DATABASE_PATH)‘

Explanation: Opens a SQLite connection to the configured database.

Line 22: ‘   database.row_factory = sqlite3.Row‘
Explanation: Assigns or computes a value used by later code.

Line 23: ‘   return database‘
Explanation: Returns the computed result to the caller.

Line 24: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 25: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 26: ‘def collection(table: str, limit: int, offset: int) -> dict:‘
Explanation: Defines the collection callable.

Line 27: ‘   database = connection()‘
Explanation: Assigns or computes a value used by later code.

Line 28: ‘   try:‘
Explanation: Controls execution flow for the surrounding operation.

Line 29: ‘   total = database.execute(f"SELECT COUNT(*) FROM
{table}").fetchone()[0]‘
Explanation: Runs a SQL statement or schema script against SQLite.

Line 30: ‘   rows = database.execute(f"SELECT * FROM {table} LIMIT ? OFFSET ?",
(limit, offset)).fetchall()‘
Explanation: Runs a SQL statement or schema script against SQLite.

Line 31: ‘   return {"total": total, "limit": limit, "offset": offset, "items":
[dict(row) for row in rows]}‘
Explanation: Returns the computed result to the caller.

Line 32: ‘   except sqlite3.OperationalError as error:‘
Explanation: Controls execution flow for the surrounding operation.

Line 33: ‘   raise HTTPException(status_code=503, detail="Database is not ready")
from error‘
Explanation: Raises an error so invalid or unavailable work is reported clearly.

Line 34: ‘   finally:‘
Explanation: Controls execution flow for the surrounding operation.

Line 35: ‘   database.close()‘
Explanation: Executes part of the module’s workflow.

Line 36: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 37: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 38: ‘def _without_names(row: dict) -> dict:‘
Explanation: Defines the _without_names callable.

Line 39: ‘   """Drop Patient name fields from an API response."""‘
Explanation: Executes part of the module’s workflow.

Line 40: ‘   row.pop("family_name", None)‘
Explanation: Executes part of the module’s workflow.

Line 41: ‘   row.pop("given_name", None)‘
Explanation: Executes part of the module’s workflow.

Line 42: ‘   return row‘
Explanation: Returns the computed result to the caller.

Line 43: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 44: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 45: ‘@app.get("/health")‘
Explanation: Decorator that registers or configures the following definition.

Line 46: ‘def health() -> dict:‘
Explanation: Defines the health callable.

Line 47: ‘   database = connection()‘
Explanation: Assigns or computes a value used by later code.

Line 48: ‘   try:‘
Explanation: Controls execution flow for the surrounding operation.

Line 49: ‘   database.execute("SELECT 1")‘
Explanation: Runs a SQL statement or schema script against SQLite.

Line 50: ‘   return {"status": "ok"}‘
Explanation: Returns the computed result to the caller.

Line 51: ‘   finally:‘
Explanation: Controls execution flow for the surrounding operation.

Line 52: ‘   database.close()‘
Explanation: Executes part of the module’s workflow.

Line 53: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 54: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 55: ‘@app.get("/patients")‘
Explanation: Decorator that registers or configures the following definition.

Line 56: ‘def patients(‘
Explanation: Defines the patients callable.

Line 57: ‘   limit: int = Query(100, ge=1, le=500),‘
Explanation: Assigns or computes a value used by later code.

Line 58: ‘   offset: int = Query(0, ge=0),‘
Explanation: Assigns or computes a value used by later code.

Line 59: ‘   refresh: bool = Query(False, description="Force a fresh fetch from the
FHIR endpoint"),‘
Explanation: Assigns or computes a value used by later code.

Line 60: ‘) -> dict:‘
Explanation: Executes part of the module’s workflow.

Line 61: ‘   database = connection()‘
Explanation: Assigns or computes a value used by later code.

Line 62: ‘   try:‘
Explanation: Controls execution flow for the surrounding operation.

Line 63: ‘   current_total = database.execute("SELECT COUNT(*) FROM
patients").fetchone()[0]‘
Explanation: Runs a SQL statement or schema script against SQLite.

Line 64: ‘   except sqlite3.OperationalError:‘
Explanation: Controls execution flow for the surrounding operation.

Line 65: ‘   current_total = 0‘
Explanation: Assigns or computes a value used by later code.

Line 66: ‘   finally:‘
Explanation: Controls execution flow for the surrounding operation.

Line 67: ‘   database.close()‘
Explanation: Executes part of the module’s workflow.

Line 68: ‘   # Mirror ’fhir_retriever --all-patients --limit N’: fetch live when the
cache is short.‘
Explanation: Comment documenting the following code or an operational decision.

Line 69: ‘   if refresh or current_total < offset + limit:‘
Explanation: Controls execution flow for the surrounding operation.

Line 70: ‘   try:‘
Explanation: Controls execution flow for the surrounding operation.

Line 71: ‘   get_all_patients(‘
Explanation: Executes part of the module’s workflow.

Line 72: ‘   limit=offset + limit,‘
Explanation: Assigns or computes a value used by later code.

Line 73: ‘   output_dir=DATABASE_PATH.parent,‘
Explanation: Assigns or computes a value used by later code.

Line 74: ‘   database_path=DATABASE_PATH,‘
Explanation: Assigns or computes a value used by later code.

Line 75: ‘   refresh=refresh,‘
Explanation: Assigns or computes a value used by later code.

Line 76: ‘   )‘
Explanation: Executes part of the module’s workflow.

Line 77: ‘   except ValueError as error:‘
Explanation: Controls execution flow for the surrounding operation.

Line 78: ‘   raise HTTPException(status_code=500, detail=str(error)) from
error‘
Explanation: Raises an error so invalid or unavailable work is reported clearly.

Line 79: ‘   result = collection("patients", limit, offset)‘
Explanation: Assigns or computes a value used by later code.

Line 80: ‘   result["items"] = [_without_names(item) for item in result["items"]]‘
Explanation: Assigns or computes a value used by later code.

Line 81: ‘   return result‘
Explanation: Returns the computed result to the caller.

Line 82: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 83: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 84: ‘@app.get("/patients/{patient_id}")‘
Explanation: Decorator that registers or configures the following definition.

Line 85: ‘def patient(patient_id: str) -> dict:‘
Explanation: Defines the patient callable.

Line 86: ‘   database = connection()‘
Explanation: Assigns or computes a value used by later code.

Line 87: ‘   try:‘
Explanation: Controls execution flow for the surrounding operation.

Line 88: ‘   row = database.execute("SELECT * FROM patients WHERE patient_id = ?",
(patient_id,)).fetchone()‘
Explanation: Runs a SQL statement or schema script against SQLite.

Line 89: ‘   if row is None:‘
Explanation: Controls execution flow for the surrounding operation.

Line 90: ‘   raise HTTPException(status_code=404, detail="Patient not found")‘
Explanation: Raises an error so invalid or unavailable work is reported clearly.

Line 91: ‘   return _without_names(dict(row))‘
Explanation: Returns the computed result to the caller.

Line 92: ‘   except sqlite3.OperationalError as error:‘
Explanation: Controls execution flow for the surrounding operation.

Line 93: ‘   raise HTTPException(status_code=503, detail="Database is not ready")
from error‘
Explanation: Raises an error so invalid or unavailable work is reported clearly.

Line 94: ‘   finally:‘
Explanation: Controls execution flow for the surrounding operation.

Line 95: ‘   database.close()‘
Explanation: Executes part of the module’s workflow.

Line 96: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 97: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 98: ‘@app.get("/observations")‘
Explanation: Decorator that registers or configures the following definition.

Line 99: ‘def observations(‘
Explanation: Defines the observations callable.

Line 100: ‘   patient_id: str | None = None,‘
Explanation: Assigns or computes a value used by later code.

Line 101: ‘   observation_code: str | None = None,‘
Explanation: Assigns or computes a value used by later code.

Line 102: ‘   encounter_id: str | None = None,‘
Explanation: Assigns or computes a value used by later code.

Line 103: ‘   limit: int = Query(100, ge=1, le=500),‘
Explanation: Assigns or computes a value used by later code.

Line 104: ‘   offset: int = Query(0, ge=0),‘
Explanation: Assigns or computes a value used by later code.

Line 105: ‘) -> dict:‘
Explanation: Executes part of the module’s workflow.

Line 106: ‘   filters, parameters = [], []‘
Explanation: Assigns or computes a value used by later code.

Line 107: ‘   for column, value in (("patient_id", patient_id), ("observation_code", observation_code), ("encounter_id", encounter_id)):‘
Explanation: Controls execution flow for the surrounding operation.

Line 108: ‘   if value:‘
Explanation: Controls execution flow for the surrounding operation.

Line 109: ‘   filters.append(f"{column} = ?")‘
Explanation: Assigns or computes a value used by later code.

Line 110: ‘   parameters.append(value)‘
Explanation: Executes part of the module’s workflow.

Line 111: ‘   where = f" WHERE {’ AND ’.join(filters)}" if filters else ""‘
Explanation: Assigns or computes a value used by later code.

Line 112: ‘   database = connection()‘
Explanation: Assigns or computes a value used by later code.

Line 113: ‘   try:‘
Explanation: Controls execution flow for the surrounding operation.

Line 114: ‘   total = database.execute(f"SELECT COUNT(*) FROM observations{where}", parameters).fetchone()[0]‘
Explanation: Runs a SQL statement or schema script against SQLite.

Line 115: ‘   rows = database.execute(‘
Explanation: Runs a SQL statement or schema script against SQLite.

Line 116: ‘   f"SELECT * FROM observations{where} LIMIT ? OFFSET ?", [*parameters, limit, offset]‘
Explanation: Executes part of the module’s workflow.

Line 117: ‘   ).fetchall()‘
Explanation: Executes part of the module’s workflow.

Line 118: ‘   return {"total": total, "limit": limit, "offset": offset, "items": [dict(row) for row in rows]}‘
Explanation: Returns the computed result to the caller.

Line 119: ‘   except sqlite3.OperationalError as error:‘
Explanation: Controls execution flow for the surrounding operation.

Line 120: ‘   raise HTTPException(status_code=503, detail="Database is not ready") from error‘
Explanation: Raises an error so invalid or unavailable work is reported clearly.

Line 121: ‘   finally:‘
Explanation: Controls execution flow for the surrounding operation.

Line 122: ‘   database.close()‘
Explanation: Executes part of the module’s workflow.

Line 123: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 124: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 125: ‘@app.get("/analysis/observations")‘
Explanation: Decorator that registers or configures the following definition.

Line 126: ‘def observation_analysis(‘
Explanation: Defines the observation_analysis callable.

Line 127: ‘   observation: str = Query(min_length=1),‘
Explanation: Assigns or computes a value used by later code.

Line 128: ‘   group_by: str = Query("age-band", pattern="^(age-band|sex|encounter-type)$"),‘
Explanation: Assigns or computes a value used by later code.

Line 129: ‘   patient_id: str | None = None,‘
Explanation: Assigns or computes a value used by later code.

Line 130: ‘) -> dict:‘
Explanation: Executes part of the module’s workflow.

Line 131: ‘   try:‘
Explanation: Controls execution flow for the surrounding operation.

Line 132: ‘   return analyse(DATABASE_PATH, observation, patient_id, group_by)‘
Explanation: Returns the computed result to the caller.

Line 133: ‘   except ValueError as error:‘
Explanation: Controls execution flow for the surrounding operation.

Line 134: ‘   raise HTTPException(status_code=400, detail=str(error)) from error‘
Explanation: Raises an error so invalid or unavailable work is reported clearly.

fhir_retriever.py

Purpose: FHIR client, pseudonymization layer, SQLite projection, cache, CSV export, and CLI.

Line 1: ‘"""Retrieve Patients and their Conditions and Observations from a FHIR R4 server."""‘
Explanation: Executes part of the module’s workflow.

Line 2: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 3: ‘from __future__ import annotations‘
Explanation: Imports a library or project dependency used by this module.

Line 4: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 5: ‘import argparse‘
Explanation: Imports a library or project dependency used by this module.

Line 6: ‘import csv‘
Explanation: Imports a library or project dependency used by this module.

Line 7: ‘import copy‘
Explanation: Imports a library or project dependency used by this module.

Line 8: ‘import hashlib‘
Explanation: Imports a library or project dependency used by this module.

Line 9: ‘import hmac‘
Explanation: Imports a library or project dependency used by this module.

Line 10: ‘import gzip‘
Explanation: Imports a library or project dependency used by this module.

Line 11: ‘import json‘
Explanation: Imports a library or project dependency used by this module.

Line 12: ‘import logging‘
Explanation: Imports a library or project dependency used by this module.

Line 13: ‘import os‘
Explanation: Imports a library or project dependency used by this module.

Line 14: ‘import sqlite3‘
Explanation: Imports a library or project dependency used by this module.

Line 15: ‘from dataclasses import asdict, dataclass, field‘
Explanation: Imports a library or project dependency used by this module.

Line 16: ‘from datetime import date, datetime, timedelta‘
Explanation: Imports a library or project dependency used by this module.

Line 17: ‘from pathlib import Path‘
Explanation: Imports a library or project dependency used by this module.

Line 18: ‘from typing import Any, Iterable‘
Explanation: Imports a library or project dependency used by this module.

Line 19: ‘from urllib.parse import urljoin‘
Explanation: Imports a library or project dependency used by this module.

Line 20: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 21: ‘import requests‘
Explanation: Imports a library or project dependency used by this module.

Line 22: ‘from requests.adapters import HTTPAdapter‘
Explanation: Imports a library or project dependency used by this module.

Line 23: ‘from urllib3.util.retry import Retry‘
Explanation: Imports a library or project dependency used by this module.

Line 24: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 25: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 26: ‘LOGGER = logging.getLogger(__name__)‘
Explanation: Assigns or computes a value used by later code.

Line 27: ‘DEFAULT_ENDPOINT = "http://hapi.fhir.org/baseR4"‘
Explanation: Assigns or computes a value used by later code.

Line 28: ‘PATIENT_QUERY = "Patient"‘
Explanation: Assigns or computes a value used by later code.

Line 29: ‘PSEUDONYMIZATION_KEY_ENV = "FHIR_PSEUDONYMIZATION_KEY"‘
Explanation: Assigns or computes a value used by later code.

Line 30: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 31: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 32: ‘class Pseudonymizer:‘
Explanation: Declares a class that groups related state and behavior.

Line 33: ‘   """Create stable, non-reversible HMAC identifiers and date offsets."""‘
Explanation: Executes part of the module’s workflow.

Line 34: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 35: ‘   def __init__(self, key: str) -> None:‘
Explanation: Defines the __init__ callable.

Line 36: ‘   if not key:‘
Explanation: Controls execution flow for the surrounding operation.

Line 37: ‘   raise ValueError(f"Set {PSEUDONYMIZATION_KEY_ENV} before running
the retriever")‘
Explanation: Raises an error so invalid or unavailable work is reported clearly.

Line 38: ‘   self.key = key.encode("utf-8")‘
Explanation: Assigns or computes a value used by later code.

Line 39: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 40: ‘   def identifier(self, resource_type: str, raw_id: str) -> str:‘
Explanation: Defines the identifier callable.

Line 41: ‘   digest = hmac.new(self.key, f"{resource_type}:{raw_id}".encode(),
hashlib.sha256).hexdigest()‘
Explanation: Assigns or computes a value used by later code.

Line 42: ‘   return f"{resource_type[:3].upper()}-{digest[:20]}"‘
Explanation: Returns the computed result to the caller.

Line 43: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 44: ‘   def patient_offset(self, patient_id: str) -> int:‘
Explanation: Defines the patient_offset callable.

Line 45: ‘   digest = hmac.new(self.key, f"date:{patient_id}".encode(), hashlib.sha256).digest()‘
Explanation: Assigns or computes a value used by later code.

Line 46: ‘   return int.from_bytes(digest[:2], "big") % 731 - 365‘
Explanation: Returns the computed result to the caller.

Line 47: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 48: ‘   def shift_date(self, value: str, offset_days: int) -> str:‘
Explanation: Defines the shift_date callable.

Line 49: ‘   try:‘
Explanation: Controls execution flow for the surrounding operation.

Line 50: ‘   if "T" in value:‘
Explanation: Controls execution flow for the surrounding operation.

Line 51: ‘   parsed = datetime.fromisoformat(value.replace("Z", "+00:00"))‘
Explanation: Assigns or computes a value used by later code.

Line 52: ‘   return (parsed + timedelta(days=offset_days)).isoformat().replace("+00:00", "Z")‘
Explanation: Returns the computed result to the caller.

Line 53: ‘   return (date.fromisoformat(value) + timedelta(days=offset_days)).isoformat()‘
Explanation: Returns the computed result to the caller.

Line 54: ‘   except (ValueError, OverflowError):‘
Explanation: Controls execution flow for the surrounding operation.

Line 55: ‘   # Some real-world dates (e.g. near year 1 or 9999 on public test servers)‘
Explanation: Comment documenting the following code or an operational decision.

Line 56: ‘   # cannot be shifted without leaving the valid date range; keep as-is.‘
Explanation: Comment documenting the following code or an operational decision.

Line 57: ‘   return value‘
Explanation: Returns the computed result to the caller.

Line 58: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 59: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 60: ‘def _reference_id(resource: dict[str, Any], field_name: str, resource_type: str) -> str | None:‘
Explanation: Defines the _reference_id callable.

Line 61: ‘   reference = resource.get(field_name, {}).get("reference")‘
Explanation: Performs or configures an HTTP request to the FHIR service.

Line 62: ‘   if not isinstance(reference, str):‘
Explanation: Controls execution flow for the surrounding operation.

Line 63: ‘   return None‘

Explanation: Returns the computed result to the caller.

Line 64: ‘   parts = reference.rstrip("/").split("/")‘
Explanation: Assigns or computes a value used by later code.

Line 65: ‘   try:‘
Explanation: Controls execution flow for the surrounding operation.

Line 66: ‘   return parts[parts.index(resource_type) + 1]‘
Explanation: Returns the computed result to the caller.

Line 67: ‘   except (ValueError, IndexError):‘
Explanation: Controls execution flow for the surrounding operation.

Line 68: ‘   return None‘
Explanation: Returns the computed result to the caller.

Line 69: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 70: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 71: ‘@dataclass‘
Explanation: Decorator that registers or configures the following definition.

Line 72: ‘class RetrievalFailure:‘
Explanation: Declares a class that groups related state and behavior.

Line 73: ‘   query: str‘
Explanation: Executes part of the module’s workflow.

Line 74: ‘   error: str‘
Explanation: Executes part of the module’s workflow.

Line 75: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 76: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 77: ‘@dataclass‘
Explanation: Decorator that registers or configures the following definition.

Line 78: ‘class RetrievalReport:‘
Explanation: Declares a class that groups related state and behavior.

Line 79: ‘   resources: dict[str, int] = field(‘
Explanation: Assigns or computes a value used by later code.

Line 80: ‘   default_factory=lambda: {"Patient": 0, "Condition": 0, "Observation": 0, "Encounter": 0}‘
Explanation: Assigns or computes a value used by later code.

Line 81: ‘   )‘
Explanation: Executes part of the module’s workflow.

Line 82: ‘   failures: list[RetrievalFailure] = field(default_factory=list)‘
Explanation: Assigns or computes a value used by later code.

Line 83: ‘   # Maps each explicitly requested raw Patient ID to its stored pseudonym, e.g. sindhu-syn-000004 -> PAT-....‘

Explanation: Comment documenting the following code or an operational decision.

Line 84: ‘   patient_pseudonyms: dict[str, str] = field(default_factory=dict)‘
Explanation: Assigns or computes a value used by later code.

Line 85: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 86: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 87: ‘class FHIRDatabase:‘
Explanation: Declares a class that groups related state and behavior.

Line 88: ‘   """SQLite projection of the cached patient-related FHIR resources."""‘
Explanation: Executes part of the module’s workflow.

Line 89: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 90: ‘   def __init__(self, path: str | Path) -> None:‘
Explanation: Defines the __init__ callable.

Line 91: ‘   self.path = Path(path)‘
Explanation: Assigns or computes a value used by later code.

Line 92: ‘   self.path.parent.mkdir(parents=True, exist_ok=True)‘
Explanation: Assigns or computes a value used by later code.

Line 93: ‘   self.connection = sqlite3.connect(self.path)‘
Explanation: Opens a SQLite connection to the configured database.

Line 94: ‘   self._encounter_id_map: dict[str, str] = {}‘
Explanation: Assigns or computes a value used by later code.

Line 95: ‘   self.connection.execute("PRAGMA foreign_keys = ON")‘
Explanation: Runs a SQL statement or schema script against SQLite.

Line 96: ‘   self.connection.executescript(‘
Explanation: Runs a SQL statement or schema script against SQLite.

Line 97: ‘   """‘
Explanation: Executes part of the module’s workflow.

Line 98: ‘   CREATE TABLE IF NOT EXISTS patients (‘
Explanation: Executes part of the module’s workflow.

Line 99: ‘   patient_id TEXT PRIMARY KEY,‘
Explanation: Executes part of the module’s workflow.

Line 100: ‘   family_name TEXT,‘
Explanation: Executes part of the module’s workflow.

Line 101: ‘   given_name TEXT,‘
Explanation: Executes part of the module’s workflow.

Line 102: ‘   gender TEXT,‘
Explanation: Executes part of the module’s workflow.

Line 103: ‘   birth_date TEXT,‘
Explanation: Executes part of the module’s workflow.

Line 104: ‘   date_shift_days INTEGER NOT NULL‘
Explanation: Executes part of the module’s workflow.

Line 105: ‘   );‘
Explanation: Executes part of the module’s workflow.

Line 106: ‘   CREATE TABLE IF NOT EXISTS conditions (‘
Explanation: Executes part of the module’s workflow.

Line 107: ‘   condition_id TEXT PRIMARY KEY,‘
Explanation: Executes part of the module’s workflow.

Line 108: ‘   clinicalStatus TEXT,‘
Explanation: Executes part of the module’s workflow.

Line 109: ‘   verificationStatus TEXT,‘
Explanation: Executes part of the module’s workflow.

Line 110: ‘   category TEXT,‘
Explanation: Executes part of the module’s workflow.

Line 111: ‘   condition TEXT,‘
Explanation: Executes part of the module’s workflow.

Line 112: ‘   condition_code TEXT,‘
Explanation: Executes part of the module’s workflow.

Line 113: ‘   patient_id TEXT REFERENCES patients(patient_id),‘
Explanation: Executes part of the module’s workflow.

Line 114: ‘   encounter_id TEXT,‘
Explanation: Executes part of the module’s workflow.

Line 115: ‘   onsetDateTime TEXT,‘
Explanation: Executes part of the module’s workflow.

Line 116: ‘   recorded_Date TEXT‘
Explanation: Executes part of the module’s workflow.

Line 117: ‘   );‘
Explanation: Executes part of the module’s workflow.

Line 118: ‘   CREATE TABLE IF NOT EXISTS observations (‘
Explanation: Executes part of the module’s workflow.

Line 119: ‘   observation_id TEXT PRIMARY KEY,‘
Explanation: Executes part of the module’s workflow.

Line 120: ‘   patient_id TEXT REFERENCES patients(patient_id),‘
Explanation: Executes part of the module’s workflow.

Line 121: ‘   encounter_id TEXT,‘
Explanation: Executes part of the module’s workflow.

Line 122: ‘   observation_type TEXT,‘
Explanation: Executes part of the module’s workflow.

Line 123: ‘   observation_code TEXT,‘
Explanation: Executes part of the module’s workflow.

Line 124: ‘   observation_subtype TEXT,‘
Explanation: Executes part of the module’s workflow.

Line 125: ‘   effective_date_time TEXT,‘
Explanation: Executes part of the module’s workflow.

Line 126: ‘   issued TEXT,‘
Explanation: Executes part of the module’s workflow.

Line 127: ‘   value TEXT,‘
Explanation: Executes part of the module’s workflow.

Line 128: ‘   unit TEXT,‘
Explanation: Executes part of the module’s workflow.

Line 129: ‘   value_code TEXT‘
Explanation: Executes part of the module’s workflow.

Line 130: ‘   );‘
Explanation: Executes part of the module’s workflow.

Line 131: ‘   CREATE TABLE IF NOT EXISTS encounters (‘
Explanation: Executes part of the module’s workflow.

Line 132: ‘   encounter_type TEXT,‘
Explanation: Executes part of the module’s workflow.

Line 133: ‘   encounter_id TEXT PRIMARY KEY,‘
Explanation: Executes part of the module’s workflow.

Line 134: ‘   start TEXT,‘
Explanation: Executes part of the module’s workflow.

Line 135: ‘   end TEXT,‘
Explanation: Executes part of the module’s workflow.

Line 136: ‘   patient_id TEXT REFERENCES patients(patient_id)‘
Explanation: Executes part of the module’s workflow.

Line 137: ‘   );‘
Explanation: Executes part of the module’s workflow.

Line 138: ‘   CREATE INDEX IF NOT EXISTS conditions_patient_encounter_idx‘
Explanation: Executes part of the module’s workflow.

Line 139: ‘   ON conditions(patient_id, encounter_id);‘
Explanation: Executes part of the module’s workflow.

Line 140: ‘   CREATE INDEX IF NOT EXISTS observations_patient_encounter_idx‘
Explanation: Executes part of the module’s workflow.

Line 141: ‘   ON observations(patient_id, encounter_id);‘
Explanation: Executes part of the module’s workflow.

Line 142: ‘   CREATE INDEX IF NOT EXISTS encounters_patient_idx‘
Explanation: Executes part of the module’s workflow.

Line 143: ‘   ON encounters(patient_id);‘
Explanation: Executes part of the module’s workflow.

Line 144: ‘   """‘
Explanation: Executes part of the module’s workflow.

Line 145: ‘   )‘

Explanation: Executes part of the module’s workflow.

Line 146: ‘   self._migrate_patients_table()‘
Explanation: Executes part of the module’s workflow.

Line 147: ‘   self._migrate_patient_columns()‘
Explanation: Executes part of the module’s workflow.

Line 148: ‘   self._migrate_conditions_table()‘
Explanation: Executes part of the module’s workflow.

Line 149: ‘   self._migrate_encounters_table()‘
Explanation: Executes part of the module’s workflow.

Line 150: ‘   self._migrate_observations_table()‘
Explanation: Executes part of the module’s workflow.

Line 151: ‘   self._translate_existing_observation_encounter_ids()‘
Explanation: Executes part of the module’s workflow.

Line 152: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 153: ‘   @staticmethod‘
Explanation: Decorator that registers or configures the following definition.

Line 154: ‘   def _patient_columns(resource: dict[str, Any]) -> tuple[str, str | None, str | None, str | None, str | None, int]:‘
Explanation: Defines the _patient_columns callable.

Line 155: ‘   return (‘
Explanation: Returns the computed result to the caller.

Line 156: ‘   resource["id"],‘
Explanation: Executes part of the module’s workflow.

Line 157: ‘   resource.get("familyName"),‘
Explanation: Performs or configures an HTTP request to the FHIR service.

Line 158: ‘   resource.get("givenName"),‘
Explanation: Performs or configures an HTTP request to the FHIR service.

Line 159: ‘   resource.get("gender"),‘
Explanation: Performs or configures an HTTP request to the FHIR service.

Line 160: ‘   resource.get("birthDate"),‘
Explanation: Performs or configures an HTTP request to the FHIR service.

Line 161: ‘   resource["dateShiftDays"],‘
Explanation: Executes part of the module’s workflow.

Line 162: ‘   )‘
Explanation: Executes part of the module’s workflow.

Line 163: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 164: ‘   def _migrate_patients_table(self) -> None:‘
Explanation: Defines the _migrate_patients_table callable.

Line 165: ‘   columns = {column[1] for column in self.connection.execute("PRAGMA table_info(patients)")}‘

Explanation: Runs a SQL statement or schema script against SQLite.

Line 166: ‘   if "resource_json" not in columns:‘
Explanation: Controls execution flow for the surrounding operation.

Line 167: ‘   return‘
Explanation: Executes part of the module’s workflow.

Line 168: ‘   legacy_patients = self.connection.execute(‘
Explanation: Runs a SQL statement or schema script against SQLite.

Line 169: ‘   "SELECT patient_id, resource_json FROM patients"‘
Explanation: Executes part of the module’s workflow.

Line 170: ‘   ).fetchall()‘
Explanation: Executes part of the module’s workflow.

Line 171: ‘   self.connection.execute("PRAGMA foreign_keys = OFF")‘
Explanation: Runs a SQL statement or schema script against SQLite.

Line 172: ‘   try:‘
Explanation: Controls execution flow for the surrounding operation.

Line 173: ‘   with self.connection:‘
Explanation: Controls execution flow for the surrounding operation.

Line 174: ‘   self.connection.execute("DROP TABLE patients")‘
Explanation: Runs a SQL statement or schema script against SQLite.

Line 175: ‘   self.connection.execute(‘
Explanation: Runs a SQL statement or schema script against SQLite.

Line 176: ‘   "CREATE TABLE patients ("‘
Explanation: Executes part of the module’s workflow.

Line 177: ‘   "patient_id TEXT PRIMARY KEY, family_name TEXT,
given_name TEXT, "‘
Explanation: Executes part of the module’s workflow.

Line 178: ‘   "gender TEXT, birth_date TEXT, date_shift_days INTEGER
NOT NULL)"‘
Explanation: Executes part of the module’s workflow.

Line 179: ‘   )‘
Explanation: Executes part of the module’s workflow.

Line 180: ‘   for patient_id, resource_json in legacy_patients:‘
Explanation: Controls execution flow for the surrounding operation.

Line 181: ‘   try:‘
Explanation: Controls execution flow for the surrounding operation.

Line 182: ‘   patient = json.loads(resource_json)‘
Explanation: Assigns or computes a value used by later code.

Line 183: ‘   except json.JSONDecodeError:‘
Explanation: Controls execution flow for the surrounding operation.

Line 184: ‘   LOGGER.warning("Skipping malformed legacy Patient
JSON for %s", patient_id)‘
Explanation: Executes part of the module’s workflow.

Line 185: ‘   continue‘
Explanation: Executes part of the module’s workflow.

Line 186: ‘   names = patient.get("name", [])‘
Explanation: Performs or configures an HTTP request to the FHIR service.

Line 187: ‘   name = names[0] if isinstance(names, list) and names and isinstance(names[0], dict) else {}‘
Explanation: Assigns or computes a value used by later code.

Line 188: ‘   given_names = name.get("given", [])‘
Explanation: Performs or configures an HTTP request to the FHIR service.

Line 189: ‘   family_name = name.get("family")‘
Explanation: Performs or configures an HTTP request to the FHIR service.

Line 190: ‘   given_name = " ".join(given_names) if isinstance(given_names, list) else None‘
Explanation: Assigns or computes a value used by later code.

Line 191: ‘   self.connection.execute(‘
Explanation: Runs a SQL statement or schema script against SQLite.

Line 192: ‘   "INSERT INTO patients(patient_id, family_name, given_name, gender, birth_date, date_shift_days) "‘
Explanation: Executes part of the module’s workflow.

Line 193: ‘   "VALUES (?, ?, ?, ?, ?, ?)",‘
Explanation: Executes part of the module’s workflow.

Line 194: ‘   (patient_id, family_name, given_name, patient.get("gender"), None, 0),‘
Explanation: Performs or configures an HTTP request to the FHIR service.

Line 195: ‘   )‘
Explanation: Executes part of the module’s workflow.

Line 196: ‘   finally:‘
Explanation: Controls execution flow for the surrounding operation.

Line 197: ‘   self.connection.execute("PRAGMA foreign_keys = ON")‘
Explanation: Runs a SQL statement or schema script against SQLite.

Line 198: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 199: ‘   def _migrate_patient_columns(self) -> None:‘
Explanation: Defines the _migrate_patient_columns callable.

Line 200: ‘   columns = {column[1] for column in self.connection.execute("PRAGMA table_info(patients)")}‘
Explanation: Runs a SQL statement or schema script against SQLite.

Line 201: ‘   if "date_shift_days" not in columns:‘
Explanation: Controls execution flow for the surrounding operation.

Line 202: ‘   with self.connection:‘
Explanation: Controls execution flow for the surrounding operation.

Line 203: ‘   self.connection.execute(‘
Explanation: Runs a SQL statement or schema script against SQLite.

Line 204: ‘   "ALTER TABLE patients ADD COLUMN date_shift_days INTEGER NOT NULL DEFAULT 0"‘
Explanation: Executes part of the module’s workflow.

Line 205: ‘   )‘
Explanation: Executes part of the module’s workflow.

Line 206: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 207: ‘   @classmethod‘
Explanation: Decorator that registers or configures the following definition.

Line 208: ‘   def _condition_columns(cls, resource: dict[str, Any]) -> tuple[Any, ...]:‘
Explanation: Defines the _condition_columns callable.

Line 209: ‘   clinical_status = cls._first_coding(resource.get("clinicalStatus", {})).get("code")‘
Explanation: Performs or configures an HTTP request to the FHIR service.

Line 210: ‘   verification_status = cls._first_coding(resource.get("verificationStatus", {})).get("code")‘
Explanation: Performs or configures an HTTP request to the FHIR service.

Line 211: ‘   categories = resource.get("category", [])‘
Explanation: Performs or configures an HTTP request to the FHIR service.

Line 212: ‘   category = cls._first_coding(categories[0] if isinstance(categories, list) and categories else {}).get("code")‘
Explanation: Performs or configures an HTTP request to the FHIR service.

Line 213: ‘   code = resource.get("code", {})‘
Explanation: Performs or configures an HTTP request to the FHIR service.

Line 214: ‘   coding = cls._first_coding(code)‘
Explanation: Assigns or computes a value used by later code.

Line 215: ‘   return (‘
Explanation: Returns the computed result to the caller.

Line 216: ‘   resource["id"],‘
Explanation: Executes part of the module’s workflow.

Line 217: ‘   clinical_status,‘
Explanation: Executes part of the module’s workflow.

Line 218: ‘   verification_status,‘
Explanation: Executes part of the module’s workflow.

Line 219: ‘   category,‘
Explanation: Executes part of the module’s workflow.

Line 220: ‘   coding.get("display") or code.get("text") if isinstance(code, dict) else None,‘
Explanation: Performs or configures an HTTP request to the FHIR service.

Line 221: ‘   coding.get("code"),‘
Explanation: Performs or configures an HTTP request to the FHIR service.

Line 222: ‘   _reference_id(resource, "subject", "Patient"),‘
Explanation: Executes part of the module’s workflow.

Line 223: ‘   _reference_id(resource, "encounter", "Encounter"),‘
Explanation: Executes part of the module’s workflow.

Line 224: ‘   resource.get("onsetDateTime"),‘
Explanation: Performs or configures an HTTP request to the FHIR service.

Line 225: ‘   resource.get("recordedDate"),‘
Explanation: Performs or configures an HTTP request to the FHIR service.

Line 226: ‘   )‘
Explanation: Executes part of the module’s workflow.

Line 227: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 228: ‘   def _migrate_conditions_table(self) -> None:‘
Explanation: Defines the _migrate_conditions_table callable.

Line 229: ‘   columns = {column[1] for column in self.connection.execute("PRAGMA table_info(conditions)")}‘
Explanation: Runs a SQL statement or schema script against SQLite.

Line 230: ‘   if "resource_json" not in columns:‘
Explanation: Controls execution flow for the surrounding operation.

Line 231: ‘   return‘
Explanation: Executes part of the module’s workflow.

Line 232: ‘   legacy_conditions = self.connection.execute("SELECT resource_json FROM conditions").fetchall()‘
Explanation: Runs a SQL statement or schema script against SQLite.

Line 233: ‘   self.connection.execute("PRAGMA foreign_keys = OFF")‘
Explanation: Runs a SQL statement or schema script against SQLite.

Line 234: ‘   try:‘
Explanation: Controls execution flow for the surrounding operation.

Line 235: ‘   with self.connection:‘
Explanation: Controls execution flow for the surrounding operation.

Line 236: ‘   self.connection.execute("DROP TABLE conditions")‘
Explanation: Runs a SQL statement or schema script against SQLite.

Line 237: ‘   self.connection.execute(‘
Explanation: Runs a SQL statement or schema script against SQLite.

Line 238: ‘   "CREATE TABLE conditions (condition_id TEXT PRIMARY KEY, clinicalStatus TEXT, verificationStatus TEXT, category TEXT, "‘
Explanation: Executes part of the module’s workflow.

Line 239: ‘   "condition TEXT, condition_code TEXT, patient_id TEXT REFERENCES patients(patient_id), "‘
Explanation: Executes part of the module’s workflow.

Line 240: ‘   "encounter_id TEXT, onsetDateTime TEXT, recorded_Date TEXT)"‘
Explanation: Executes part of the module’s workflow.

Line 241: ‘   )‘
Explanation: Executes part of the module’s workflow.

Line 242: ‘   for (resource_json,) in legacy_conditions:‘
Explanation: Controls execution flow for the surrounding operation.

Line 243: ‘   try:‘
Explanation: Controls execution flow for the surrounding operation.

Line 244: ‘   resource = json.loads(resource_json)‘
Explanation: Assigns or computes a value used by later code.

Line 245: ‘   except json.JSONDecodeError:‘
Explanation: Controls execution flow for the surrounding operation.

Line 246: ‘   continue‘
Explanation: Executes part of the module’s workflow.

Line 247: ‘   self.connection.execute("INSERT INTO conditions VALUES
(?, ?, ?, ?, ?, ?, ?, ?, ?, ?)", self._condition_columns(resource))‘
Explanation: Runs a SQL statement or schema script against SQLite.

Line 248: ‘   finally:‘
Explanation: Controls execution flow for the surrounding operation.

Line 249: ‘   self.connection.execute("PRAGMA foreign_keys = ON")‘
Explanation: Runs a SQL statement or schema script against SQLite.

Line 250: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 251: ‘   @staticmethod‘
Explanation: Decorator that registers or configures the following definition.

Line 252: ‘   def _first_coding(value: Any) -> dict[str, Any]:‘
Explanation: Defines the _first_coding callable.

Line 253: ‘   codings = value.get("coding", []) if isinstance(value, dict) else []‘
Explanation: Performs or configures an HTTP request to the FHIR service.

Line 254: ‘   return codings[0] if isinstance(codings, list) and codings and
isinstance(codings[0], dict) else {}‘
Explanation: Returns the computed result to the caller.

Line 255: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 256: ‘   @classmethod‘
Explanation: Decorator that registers or configures the following definition.

Line 257: ‘   def _observation_columns(cls, resource: dict[str, Any]) -> tuple[Any,
...]:‘
Explanation: Defines the _observation_columns callable.

Line 258: ‘   categories = resource.get("category", [])‘
Explanation: Performs or configures an HTTP request to the FHIR service.

Line 259: ‘   category = categories[0] if isinstance(categories, list) and
categories else {}‘
Explanation: Assigns or computes a value used by later code.

Line 260: ‘   category_coding = cls._first_coding(category)‘
Explanation: Assigns or computes a value used by later code.

Line 261: ‘   code = resource.get("code", {})‘
Explanation: Performs or configures an HTTP request to the FHIR service.

Line 262: ‘   code_coding = cls._first_coding(code)‘
Explanation: Assigns or computes a value used by later code.

Line 263: ‘   quantity = resource.get("valueQuantity", {})‘
Explanation: Performs or configures an HTTP request to the FHIR service.

Line 264: ‘   if isinstance(quantity, dict):‘
Explanation: Controls execution flow for the surrounding operation.

Line 265: ‘   value = quantity.get("value")‘
Explanation: Performs or configures an HTTP request to the FHIR service.

Line 266: ‘   unit = quantity.get("unit")‘
Explanation: Performs or configures an HTTP request to the FHIR service.

Line 267: ‘   value_code = quantity.get("code")‘
Explanation: Performs or configures an HTTP request to the FHIR service.

Line 268: ‘   else:‘
Explanation: Controls execution flow for the surrounding operation.

Line 269: ‘   value = next((resource[key] for key in resource if
key.startswith("value") and key != "valueQuantity"), None)‘
Explanation: Assigns or computes a value used by later code.

Line 270: ‘   unit = None‘
Explanation: Assigns or computes a value used by later code.

Line 271: ‘   value_code = None‘
Explanation: Assigns or computes a value used by later code.

Line 272: ‘   return (‘
Explanation: Returns the computed result to the caller.

Line 273: ‘   resource["id"],‘
Explanation: Executes part of the module’s workflow.

Line 274: ‘   _reference_id(resource, "subject", "Patient"),‘
Explanation: Executes part of the module’s workflow.

Line 275: ‘   _reference_id(resource, "encounter", "Encounter"),‘
Explanation: Executes part of the module’s workflow.

Line 276: ‘   category_coding.get("code"),‘
Explanation: Performs or configures an HTTP request to the FHIR service.

Line 277: ‘   code_coding.get("code"),‘
Explanation: Performs or configures an HTTP request to the FHIR service.

Line 278: ‘   code_coding.get("display") or code.get("text") if
isinstance(code, dict) else None,‘
Explanation: Performs or configures an HTTP request to the FHIR service.

Line 279: ‘   resource.get("effectiveDateTime"),‘
Explanation: Performs or configures an HTTP request to the FHIR service.

Line 280: ‘   resource.get("issued"),‘
Explanation: Performs or configures an HTTP request to the FHIR service.

Line 281: ‘   str(value) if value is not None else None,‘
Explanation: Executes part of the module’s workflow.

Line 282: ‘   unit,‘
Explanation: Executes part of the module’s workflow.

Line 283: ‘   value_code,‘
Explanation: Executes part of the module’s workflow.

Line 284: ‘   )‘
Explanation: Executes part of the module’s workflow.

Line 285: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 286: ‘   def _migrate_observations_table(self) -> None:‘
Explanation: Defines the _migrate_observations_table callable.

Line 287: ‘   columns = {column[1] for column in self.connection.execute("PRAGMA table_info(observations)")}‘
Explanation: Runs a SQL statement or schema script against SQLite.

Line 288: ‘   if "resource_json" not in columns:‘
Explanation: Controls execution flow for the surrounding operation.

Line 289: ‘   return‘
Explanation: Executes part of the module’s workflow.

Line 290: ‘   legacy_observations = self.connection.execute(‘
Explanation: Runs a SQL statement or schema script against SQLite.

Line 291: ‘   "SELECT observation_id, resource_json FROM observations"‘
Explanation: Executes part of the module’s workflow.

Line 292: ‘   ).fetchall()‘
Explanation: Executes part of the module’s workflow.

Line 293: ‘   self.connection.execute("PRAGMA foreign_keys = OFF")‘
Explanation: Runs a SQL statement or schema script against SQLite.

Line 294: ‘   try:‘
Explanation: Controls execution flow for the surrounding operation.

Line 295: ‘   with self.connection:‘
Explanation: Controls execution flow for the surrounding operation.

Line 296: ‘   self.connection.execute("DROP TABLE observations")‘
Explanation: Runs a SQL statement or schema script against SQLite.

Line 297: ‘   self.connection.execute(‘
Explanation: Runs a SQL statement or schema script against SQLite.

Line 298: ‘   "CREATE TABLE observations (observation_id TEXT PRIMARY KEY, "‘
Explanation: Executes part of the module’s workflow.

Line 299: ‘   "patient_id TEXT REFERENCES patients(patient_id), encounter_id TEXT, "‘
Explanation: Executes part of the module’s workflow.

Line 300: ‘   "observation_type TEXT, observation_code TEXT, observation_subtype TEXT, "‘

Explanation: Executes part of the module’s workflow.

Line 301: ‘   "effective_date_time TEXT, issued TEXT, value TEXT, unit TEXT, value_code TEXT)"‘
Explanation: Executes part of the module’s workflow.

Line 302: ‘   )‘
Explanation: Executes part of the module’s workflow.

Line 303: ‘   self.connection.execute(‘
Explanation: Runs a SQL statement or schema script against SQLite.

Line 304: ‘   "CREATE INDEX IF NOT EXISTS observations_patient_encounter_idx "‘
Explanation: Executes part of the module’s workflow.

Line 305: ‘   "ON observations(patient_id, encounter_id)"‘
Explanation: Executes part of the module’s workflow.

Line 306: ‘   )‘
Explanation: Executes part of the module’s workflow.

Line 307: ‘   for observation_id, resource_json in legacy_observations:‘
Explanation: Controls execution flow for the surrounding operation.

Line 308: ‘   try:‘
Explanation: Controls execution flow for the surrounding operation.

Line 309: ‘   observation = json.loads(resource_json)‘
Explanation: Assigns or computes a value used by later code.

Line 310: ‘   except json.JSONDecodeError:‘
Explanation: Controls execution flow for the surrounding operation.

Line 311: ‘   LOGGER.warning("Skipping malformed legacy Observation JSON for %s", observation_id)‘
Explanation: Executes part of the module’s workflow.

Line 312: ‘   continue‘
Explanation: Executes part of the module’s workflow.

Line 313: ‘   observation_columns = list(self._observation_columns(observation))‘
Explanation: Assigns or computes a value used by later code.

Line 314: ‘   observation_columns[2] = self._encounter_id_map.get(‘
Explanation: Performs or configures an HTTP request to the FHIR service.

Line 315: ‘   observation_columns[2], observation_columns[2]‘
Explanation: Executes part of the module’s workflow.

Line 316: ‘   )‘
Explanation: Executes part of the module’s workflow.

Line 317: ‘   self.connection.execute(‘
Explanation: Runs a SQL statement or schema script against SQLite.

Line 318: ‘   "INSERT INTO observations VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)",‘
Explanation: Executes part of the module’s workflow.

Line 319: ‘   observation_columns,‘

Explanation: Executes part of the module’s workflow.

Line 320: ‘   )‘
Explanation: Executes part of the module’s workflow.

Line 321: ‘   finally:‘
Explanation: Controls execution flow for the surrounding operation.

Line 322: ‘   self.connection.execute("PRAGMA foreign_keys = ON")‘
Explanation: Runs a SQL statement or schema script against SQLite.

Line 323: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 324: ‘   @staticmethod‘
Explanation: Decorator that registers or configures the following definition.

Line 325: ‘   def _encounter_columns(resource: dict[str, Any]) -> tuple[str | None, str, str | None, str | None, str | None]:‘
Explanation: Defines the _encounter_columns callable.

Line 326: ‘   identifiers = resource.get("identifier", [])‘
Explanation: Performs or configures an HTTP request to the FHIR service.

Line 327: ‘   identifier = identifiers[0] if isinstance(identifiers, list) and identifiers else {}‘
Explanation: Assigns or computes a value used by later code.

Line 328: ‘   encounter_id = identifier.get("value") if isinstance(identifier, dict) else None‘
Explanation: Performs or configures an HTTP request to the FHIR service.

Line 329: ‘   if not encounter_id:‘
Explanation: Controls execution flow for the surrounding operation.

Line 330: ‘   encounter_id = resource["id"]‘
Explanation: Assigns or computes a value used by later code.

Line 331: ‘   encounter_class = resource.get("class", {})‘
Explanation: Performs or configures an HTTP request to the FHIR service.

Line 332: ‘   period = resource.get("period", {})‘
Explanation: Performs or configures an HTTP request to the FHIR service.

Line 333: ‘   return (‘
Explanation: Returns the computed result to the caller.

Line 334: ‘   encounter_class.get("display") or encounter_class.get("code")‘
Explanation: Performs or configures an HTTP request to the FHIR service.

Line 335: ‘   if isinstance(encounter_class, dict)‘
Explanation: Controls execution flow for the surrounding operation.

Line 336: ‘   else None,‘
Explanation: Executes part of the module’s workflow.

Line 337: ‘   encounter_id,‘
Explanation: Executes part of the module’s workflow.

Line 338: ‘   period.get("start") if isinstance(period, dict) else None,‘
Explanation: Performs or configures an HTTP request to the FHIR service.

Line 339: ‘   period.get("end") if isinstance(period, dict) else None,‘
Explanation: Performs or configures an HTTP request to the FHIR service.

Line 340: ‘   _reference_id(resource, "subject", "Patient"),‘
Explanation: Executes part of the module’s workflow.

Line 341: ‘   )‘
Explanation: Executes part of the module’s workflow.

Line 342: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 343: ‘   def _migrate_encounters_table(self) -> None:‘
Explanation: Defines the _migrate_encounters_table callable.

Line 344: ‘   columns = {column[1] for column in self.connection.execute("PRAGMA table_info(encounters)")}‘
Explanation: Runs a SQL statement or schema script against SQLite.

Line 345: ‘   if "resource_json" not in columns:‘
Explanation: Controls execution flow for the surrounding operation.

Line 346: ‘   return‘
Explanation: Executes part of the module’s workflow.

Line 347: ‘   legacy_encounters = self.connection.execute(‘
Explanation: Runs a SQL statement or schema script against SQLite.

Line 348: ‘   "SELECT encounter_id, resource_json FROM encounters"‘
Explanation: Executes part of the module’s workflow.

Line 349: ‘   ).fetchall()‘
Explanation: Executes part of the module’s workflow.

Line 350: ‘   self.connection.execute("PRAGMA foreign_keys = OFF")‘
Explanation: Runs a SQL statement or schema script against SQLite.

Line 351: ‘   try:‘
Explanation: Controls execution flow for the surrounding operation.

Line 352: ‘   with self.connection:‘
Explanation: Controls execution flow for the surrounding operation.

Line 353: ‘   self.connection.execute("DROP TABLE encounters")‘
Explanation: Runs a SQL statement or schema script against SQLite.

Line 354: ‘   self.connection.execute(‘
Explanation: Runs a SQL statement or schema script against SQLite.

Line 355: ‘   "CREATE TABLE encounters (encounter_type TEXT, encounter_id TEXT PRIMARY KEY, "‘
Explanation: Executes part of the module’s workflow.

Line 356: ‘   "start TEXT, end TEXT, patient_id TEXT REFERENCES patients(patient_id))"‘
Explanation: Executes part of the module’s workflow.

Line 357: ‘   )‘
Explanation: Executes part of the module’s workflow.

Line 358: ‘   self.connection.execute(‘
Explanation: Runs a SQL statement or schema script against SQLite.

Line 359: ‘   "CREATE INDEX IF NOT EXISTS encounters_patient_idx ON encounters(patient_id)"‘
Explanation: Executes part of the module’s workflow.

Line 360: ‘   )‘
Explanation: Executes part of the module’s workflow.

Line 361: ‘   for encounter_id, resource_json in legacy_encounters:‘
Explanation: Controls execution flow for the surrounding operation.

Line 362: ‘   try:‘
Explanation: Controls execution flow for the surrounding operation.

Line 363: ‘   encounter = json.loads(resource_json)‘
Explanation: Assigns or computes a value used by later code.

Line 364: ‘   except json.JSONDecodeError:‘
Explanation: Controls execution flow for the surrounding operation.

Line 365: ‘   LOGGER.warning("Skipping malformed legacy Encounter JSON for %s", encounter_id)‘
Explanation: Executes part of the module’s workflow.

Line 366: ‘   continue‘
Explanation: Executes part of the module’s workflow.

Line 367: ‘   encounter_columns = self._encounter_columns(encounter)‘
Explanation: Assigns or computes a value used by later code.

Line 368: ‘   self._encounter_id_map[encounter["id"]] = encounter_columns[1]‘
Explanation: Assigns or computes a value used by later code.

Line 369: ‘   self.connection.execute(‘
Explanation: Runs a SQL statement or schema script against SQLite.

Line 370: ‘   "INSERT INTO encounters(encounter_type, encounter_id, start, end, patient_id) "‘
Explanation: Executes part of the module’s workflow.

Line 371: ‘   "VALUES (?, ?, ?, ?, ?)",‘
Explanation: Executes part of the module’s workflow.

Line 372: ‘   encounter_columns,‘
Explanation: Executes part of the module’s workflow.

Line 373: ‘   )‘
Explanation: Executes part of the module’s workflow.

Line 374: ‘   finally:‘
Explanation: Controls execution flow for the surrounding operation.

Line 375: ‘   self.connection.execute("PRAGMA foreign_keys = ON")‘
Explanation: Runs a SQL statement or schema script against SQLite.

Line 376: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 377: ‘   def _translate_existing_observation_encounter_ids(self) -> None:‘
Explanation: Defines the _translate_existing_observation_encounter_ids callable.

Line 378: ‘   if not self._encounter_id_map:‘
Explanation: Controls execution flow for the surrounding operation.

Line 379: ‘   return‘
Explanation: Executes part of the module’s workflow.

Line 380: ‘   with self.connection:‘
Explanation: Controls execution flow for the surrounding operation.

Line 381: ‘   for fhir_encounter_id, encounter_id in
self._encounter_id_map.items():‘
Explanation: Controls execution flow for the surrounding operation.

Line 382: ‘   self.connection.execute(‘
Explanation: Runs a SQL statement or schema script against SQLite.

Line 383: ‘   "UPDATE observations SET encounter_id = ? WHERE
encounter_id = ?",‘
Explanation: Assigns or computes a value used by later code.

Line 384: ‘   (encounter_id, fhir_encounter_id),‘
Explanation: Executes part of the module’s workflow.

Line 385: ‘   )‘
Explanation: Executes part of the module’s workflow.

Line 386: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 387: ‘   def sync(self, resources: Iterable[dict[str, Any]]) -> None:‘
Explanation: Defines the sync callable.

Line 388: ‘   grouped = {"Patient": [], "Condition": [], "Observation": [],
"Encounter": []}‘
Explanation: Assigns or computes a value used by later code.

Line 389: ‘   for resource in resources:‘
Explanation: Controls execution flow for the surrounding operation.

Line 390: ‘   resource_type = resource.get("resourceType")‘
Explanation: Performs or configures an HTTP request to the FHIR service.

Line 391: ‘   if resource_type in grouped and resource.get("id"):‘
Explanation: Controls execution flow for the surrounding operation.

Line 392: ‘   grouped[resource_type].append(resource)‘
Explanation: Executes part of the module’s workflow.

Line 393: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 394: ‘   with self.connection:‘
Explanation: Controls execution flow for the surrounding operation.

Line 395: ‘   for resource in grouped["Patient"]:‘
Explanation: Controls execution flow for the surrounding operation.

Line 396: ‘   self.connection.execute(‘
Explanation: Runs a SQL statement or schema script against SQLite.

Line 397: ‘   "INSERT INTO patients(patient_id, family_name,
given_name, gender, birth_date, date_shift_days) "‘

Explanation: Executes part of the module’s workflow.

Line 398: ‘   "VALUES (?, ?, ?, ?, ?, ?) ON CONFLICT(patient_id) DO UPDATE SET "‘
Explanation: Executes part of the module’s workflow.

Line 399: ‘   "family_name = excluded.family_name, given_name = excluded.given_name, "‘
Explanation: Assigns or computes a value used by later code.

Line 400: ‘   "gender = excluded.gender, birth_date = excluded.birth_date, "‘
Explanation: Assigns or computes a value used by later code.

Line 401: ‘   "date_shift_days = excluded.date_shift_days",‘
Explanation: Assigns or computes a value used by later code.

Line 402: ‘   self._patient_columns(resource),‘
Explanation: Executes part of the module’s workflow.

Line 403: ‘   )‘
Explanation: Executes part of the module’s workflow.

Line 404: ‘   for resource_type, table_name, id_column in (‘
Explanation: Controls execution flow for the surrounding operation.

Line 405: ‘   ("Encounter", "encounters", "encounter_id"),‘
Explanation: Executes part of the module’s workflow.

Line 406: ‘   ("Condition", "conditions", "condition_id"),‘
Explanation: Executes part of the module’s workflow.

Line 407: ‘   ("Observation", "observations", "observation_id"),‘
Explanation: Executes part of the module’s workflow.

Line 408: ‘   ):‘
Explanation: Executes part of the module’s workflow.

Line 409: ‘   for resource in grouped[resource_type]:‘
Explanation: Controls execution flow for the surrounding operation.

Line 410: ‘   patient_id = _reference_id(resource, "subject", "Patient")‘
Explanation: Assigns or computes a value used by later code.

Line 411: ‘   resource_json = json.dumps(resource, separators=(",", ":"), sort_keys=True)‘
Explanation: Assigns or computes a value used by later code.

Line 412: ‘   if resource_type == "Encounter":‘
Explanation: Controls execution flow for the surrounding operation.

Line 413: ‘   self.connection.execute(‘
Explanation: Runs a SQL statement or schema script against SQLite.

Line 414: ‘   "INSERT INTO encounters(encounter_type, encounter_id, start, end, patient_id) "‘
Explanation: Executes part of the module’s workflow.

Line 415: ‘   "VALUES (?, ?, ?, ?, ?) ON CONFLICT(encounter_id) DO UPDATE SET "‘
Explanation: Executes part of the module’s workflow.

Line 416: ‘   "encounter_type = excluded.encounter_type, start = excluded.start, "‘
Explanation: Assigns or computes a value used by later code.

Line 417: ‘   "end = excluded.end, patient_id = excluded.patient_id",‘
Explanation: Assigns or computes a value used by later code.

Line 418: ‘   self._encounter_columns(resource),‘
Explanation: Executes part of the module’s workflow.

Line 419: ‘   )‘
Explanation: Executes part of the module’s workflow.

Line 420: ‘   self.connection.execute(‘
Explanation: Runs a SQL statement or schema script against SQLite.

Line 421: ‘   "UPDATE observations SET encounter_id = ? WHERE encounter_id = ?",‘
Explanation: Assigns or computes a value used by later code.

Line 422: ‘   (self._encounter_columns(resource)[1], resource["id"]),‘
Explanation: Executes part of the module’s workflow.

Line 423: ‘   )‘
Explanation: Executes part of the module’s workflow.

Line 424: ‘   else:‘
Explanation: Controls execution flow for the surrounding operation.

Line 425: ‘   if resource_type == "Observation":‘
Explanation: Controls execution flow for the surrounding operation.

Line 426: ‘   self.connection.execute(‘
Explanation: Runs a SQL statement or schema script against SQLite.

Line 427: ‘   "INSERT INTO observations VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?) "‘
Explanation: Executes part of the module’s workflow.

Line 428: ‘   "ON CONFLICT(observation_id) DO UPDATE SET "‘
Explanation: Executes part of the module’s workflow.

Line 429: ‘   "patient_id = excluded.patient_id, encounter_id = excluded.encounter_id, "‘
Explanation: Assigns or computes a value used by later code.

Line 430: ‘   "observation_type = excluded.observation_type, "‘
Explanation: Assigns or computes a value used by later code.

Line 431: ‘   "observation_code = excluded.observation_code, "‘
Explanation: Assigns or computes a value used by later code.

Line 432: ‘   "observation_subtype = excluded.observation_subtype, "‘
Explanation: Assigns or computes a value used by later code.

Line 433: ‘   "effective_date_time =

```
excluded.effective_date_time, issued = excluded.issued, "‘
Explanation: Assigns or computes a value used by later code.

Line 434: ‘                                "value = excluded.value, unit =
excluded.unit, value_code = excluded.value_code",‘
Explanation: Assigns or computes a value used by later code.

Line 435: ‘                                self._observation_columns(resource),‘
Explanation: Executes part of the module’s workflow.

Line 436: ‘                            )‘
Explanation: Executes part of the module’s workflow.

Line 437: ‘                            continue‘
Explanation: Executes part of the module’s workflow.

Line 438: ‘                        if resource_type == "Condition":‘
Explanation: Controls execution flow for the surrounding operation.

Line 439: ‘                            self.connection.execute(‘
Explanation: Runs a SQL statement or schema script against SQLite.

Line 440: ‘                                "INSERT INTO conditions VALUES (?, ?, ?, ?,
?, ?, ?, ?, ?, ?) "‘
Explanation: Executes part of the module’s workflow.

Line 441: ‘                                "ON CONFLICT(condition_id) DO UPDATE SET "‘
Explanation: Executes part of the module’s workflow.

Line 442: ‘                                "clinicalStatus = excluded.clinicalStatus, "‘
Explanation: Assigns or computes a value used by later code.

Line 443: ‘                                "verificationStatus =
excluded.verificationStatus, category = excluded.category, "‘
Explanation: Assigns or computes a value used by later code.

Line 444: ‘                                "condition = excluded.condition,
condition_code = excluded.condition_code, "‘
Explanation: Assigns or computes a value used by later code.

Line 445: ‘                                "patient_id = excluded.patient_id,
encounter_id = excluded.encounter_id, "‘
Explanation: Assigns or computes a value used by later code.

Line 446: ‘                                "onsetDateTime = excluded.onsetDateTime,
recorded_Date = excluded.recorded_Date",‘
Explanation: Assigns or computes a value used by later code.

Line 447: ‘                                self._condition_columns(resource),‘
Explanation: Executes part of the module’s workflow.

Line 448: ‘                            )‘
Explanation: Executes part of the module’s workflow.

Line 449: ‘                            continue‘
Explanation: Executes part of the module’s workflow.

Line 450: ‘                        self.connection.execute(‘
Explanation: Runs a SQL statement or schema script against SQLite.

Line 451: ‘                            f"INSERT INTO {table_name}({id_column},
patient_id, encounter_id, resource_json) "‘
```

Explanation: Executes part of the module’s workflow.

Line 452: ‘   f"VALUES (?, ?, ?, ?) ON CONFLICT({id_column}) DO UPDATE SET "‘
Explanation: Executes part of the module’s workflow.

Line 453: ‘   "patient_id = excluded.patient_id, encounter_id = excluded.encounter_id, "‘
Explanation: Assigns or computes a value used by later code.

Line 454: ‘   "resource_json = excluded.resource_json",‘
Explanation: Assigns or computes a value used by later code.

Line 455: ‘   (‘
Explanation: Executes part of the module’s workflow.

Line 456: ‘   resource["id"],‘
Explanation: Executes part of the module’s workflow.

Line 457: ‘   patient_id,‘
Explanation: Executes part of the module’s workflow.

Line 458: ‘   _reference_id(resource, "encounter", "Encounter"),‘
Explanation: Executes part of the module’s workflow.

Line 459: ‘   resource_json,‘
Explanation: Executes part of the module’s workflow.

Line 460: ‘   ),‘
Explanation: Executes part of the module’s workflow.

Line 461: ‘   )‘
Explanation: Executes part of the module’s workflow.

Line 462: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 463: ‘   def close(self) -> None:‘
Explanation: Defines the close callable.

Line 464: ‘   self.connection.close()‘
Explanation: Executes part of the module’s workflow.

Line 465: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 466: ‘   def export_csv(self, output_dir: str | Path) -> None:‘
Explanation: Defines the export_csv callable.

Line 467: ‘   """Overwrite CSV exports from the current SQLite tables (atomic, always fully rewritten)."""‘
Explanation: Executes part of the module’s workflow.

Line 468: ‘   destination = Path(output_dir)‘
Explanation: Assigns or computes a value used by later code.

Line 469: ‘   destination.mkdir(parents=True, exist_ok=True)‘
Explanation: Assigns or computes a value used by later code.

Line 470: ‘   for table_name in ("patients", "conditions", "observations", "encounters"):‘

Explanation: Controls execution flow for the surrounding operation.

Line 471: ‘   columns = [column[1] for column in
self.connection.execute(f"PRAGMA table_info({table_name})")]‘
Explanation: Runs a SQL statement or schema script against SQLite.

Line 472: ‘   quoted_columns = ", ".join(f’"{column}"’ for column in columns)‘
Explanation: Assigns or computes a value used by later code.

Line 473: ‘   rows = self.connection.execute(f"SELECT {quoted_columns} FROM
{table_name}")‘
Explanation: Runs a SQL statement or schema script against SQLite.

Line 474: ‘   target_path = destination / f"{table_name}.csv"‘
Explanation: Assigns or computes a value used by later code.

Line 475: ‘   temporary_path = target_path.with_suffix(".csv.tmp")‘
Explanation: Assigns or computes a value used by later code.

Line 476: ‘   with temporary_path.open("w", newline="", encoding="utf-8") as
file:‘
Explanation: Controls execution flow for the surrounding operation.

Line 477: ‘   writer = csv.writer(file)‘
Explanation: Assigns or computes a value used by later code.

Line 478: ‘   writer.writerow(columns)‘
Explanation: Executes part of the module’s workflow.

Line 479: ‘   writer.writerows(rows)‘
Explanation: Executes part of the module’s workflow.

Line 480: ‘   temporary_path.replace(target_path)‘
Explanation: Executes part of the module’s workflow.

Line 481: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 482: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 483: ‘class FHIRRetriever:‘
Explanation: Declares a class that groups related state and behavior.

Line 484: ‘   """Download patient data while retaining completed work across process
runs.‘
Explanation: Executes part of the module’s workflow.

Line 485: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 486: ‘   Resources are deduplicated in the compact ’’resources.json.gz’’ cache.
Every‘
Explanation: Executes part of the module’s workflow.

Line 487: ‘   completed search is checkpointed, so a later run uses local data and
retries‘
Explanation: Executes part of the module’s workflow.

Line 488: ‘   only incomplete searches. Pass ’’refresh=True’’ to intentionally re-
query.‘
Explanation: Assigns or computes a value used by later code.

Line 489: ‘   """‘
Explanation: Executes part of the module’s workflow.

Line 490: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 491: ‘   def __init__(‘
Explanation: Defines the __init__ callable.

Line 492: ‘   self,‘
Explanation: Executes part of the module’s workflow.

Line 493: ‘   endpoint: str = DEFAULT_ENDPOINT,‘
Explanation: Assigns or computes a value used by later code.

Line 494: ‘   output_dir: str | Path = "fhir_output",‘
Explanation: Assigns or computes a value used by later code.

Line 495: ‘   timeout: tuple[float, float] = (5.0, 30.0),‘
Explanation: Assigns or computes a value used by later code.

Line 496: ‘   retries: int = 2,‘
Explanation: Assigns or computes a value used by later code.

Line 497: ‘   page_size: int = 50,‘
Explanation: Assigns or computes a value used by later code.

Line 498: ‘   patient_limit: int | None = None,‘
Explanation: Assigns or computes a value used by later code.

Line 499: ‘   refresh: bool = False,‘
Explanation: Assigns or computes a value used by later code.

Line 500: ‘   database_path: str | Path | None = None,‘
Explanation: Assigns or computes a value used by later code.

Line 501: ‘   patient_id: str | None = None,‘
Explanation: Assigns or computes a value used by later code.

Line 502: ‘   pseudonymization_key: str | None = None,‘
Explanation: Assigns or computes a value used by later code.

Line 503: ‘   session: requests.Session | None = None,‘
Explanation: Performs or configures an HTTP request to the FHIR service.

Line 504: ‘   ) -> None:‘
Explanation: Executes part of the module’s workflow.

Line 505: ‘   if retries < 0:‘
Explanation: Controls execution flow for the surrounding operation.

Line 506: ‘   raise ValueError("retries must be non-negative")‘
Explanation: Raises an error so invalid or unavailable work is reported clearly.

Line 507: ‘   if not 1 <= page_size <= 100:‘
Explanation: Controls execution flow for the surrounding operation.

Line 508: ‘   raise ValueError("page_size must be between 1 and 100")‘
Explanation: Raises an error so invalid or unavailable work is reported clearly.

Line 509: ‘   if patient_limit is not None and patient_limit < 1:‘

Explanation: Controls execution flow for the surrounding operation.

Line 510: ‘   raise ValueError("patient_limit must be positive")‘
Explanation: Raises an error so invalid or unavailable work is reported clearly.

Line 511: ‘   self.endpoint = endpoint.rstrip("/") + "/"‘
Explanation: Assigns or computes a value used by later code.

Line 512: ‘   self.output_dir = Path(output_dir)‘
Explanation: Assigns or computes a value used by later code.

Line 513: ‘   self.timeout = timeout‘
Explanation: Assigns or computes a value used by later code.

Line 514: ‘   self.page_size = page_size‘
Explanation: Assigns or computes a value used by later code.

Line 515: ‘   self.patient_limit = patient_limit‘
Explanation: Assigns or computes a value used by later code.

Line 516: ‘   self.refresh = refresh‘
Explanation: Assigns or computes a value used by later code.

Line 517: ‘   self.patient_id = patient_id‘
Explanation: Assigns or computes a value used by later code.

Line 518: ‘   self.pseudonymizer = Pseudonymizer(‘
Explanation: Assigns or computes a value used by later code.

Line 519: ‘   pseudonymization_key or os.environ.get(PSEUDONYMIZATION_KEY_ENV,
"")‘
Explanation: Performs or configures an HTTP request to the FHIR service.

Line 520: ‘   )‘
Explanation: Executes part of the module’s workflow.

Line 521: ‘   self._raw_patient_ids: dict[str, str] = {}‘
Explanation: Assigns or computes a value used by later code.

Line 522: ‘   self.session = session or self._make_session(retries)‘
Explanation: Assigns or computes a value used by later code.

Line 523: ‘   self.resources_path = self.output_dir / "resources.json.gz"‘
Explanation: Assigns or computes a value used by later code.

Line 524: ‘   self.checkpoint_path = self.output_dir / "checkpoint.json"‘
Explanation: Assigns or computes a value used by later code.

Line 525: ‘   self.database_path = Path(database_path) if database_path else
self.output_dir / "fhir_resources.sqlite3"‘
Explanation: Assigns or computes a value used by later code.

Line 526: ‘   loaded_resources = self._load_json(self.resources_path, {})‘
Explanation: Assigns or computes a value used by later code.

Line 527: ‘   self._resources = {‘
Explanation: Assigns or computes a value used by later code.

Line 528: ‘   key: value if value.get("_deidentified") else
self._pseudonymize_resource(value)‘
Explanation: Performs or configures an HTTP request to the FHIR service.

Line 529: ‘   for key, value in loaded_resources.items()‘
Explanation: Controls execution flow for the surrounding operation.

Line 530: ‘   }‘
Explanation: Executes part of the module’s workflow.

Line 531: ‘   checkpoint = self._load_json(self.checkpoint_path, {"endpoint":
self.endpoint, "completed": []})‘
Explanation: Assigns or computes a value used by later code.

Line 532: ‘   if checkpoint["endpoint"] != self.endpoint:‘
Explanation: Controls execution flow for the surrounding operation.

Line 533: ‘   raise ValueError(f"Cache belongs to {checkpoint[’endpoint’]};
choose another output directory")‘
Explanation: Raises an error so invalid or unavailable work is reported clearly.

Line 534: ‘   completed = set() if refresh else set(checkpoint["completed"])‘
Explanation: Assigns or computes a value used by later code.

Line 535: ‘   self._completed = {‘
Explanation: Assigns or computes a value used by later code.

Line 536: ‘   query‘
Explanation: Executes part of the module’s workflow.

Line 537: ‘   for query in completed‘
Explanation: Controls execution flow for the surrounding operation.

Line 538: ‘   if query == PATIENT_QUERY or "?patient=PAT-" in query or
"?_id=PAT-" in query‘
Explanation: Controls execution flow for the surrounding operation.

Line 539: ‘   }‘
Explanation: Executes part of the module’s workflow.

Line 540: ‘   self.database = FHIRDatabase(self.database_path)‘
Explanation: Assigns or computes a value used by later code.

Line 541: ‘   self.database.sync(self._resources.values())‘
Explanation: Executes part of the module’s workflow.

Line 542: ‘   self._repair_encounter_links(loaded_resources)‘
Explanation: Executes part of the module’s workflow.

Line 543: ‘   if loaded_resources and loaded_resources != self._resources:‘
Explanation: Controls execution flow for the surrounding operation.

Line 544: ‘   self._save_resources()‘
Explanation: Executes part of the module’s workflow.

Line 545: ‘   if self._completed != completed:‘
Explanation: Controls execution flow for the surrounding operation.

Line 546: ‘   self._save_checkpoint()‘
Explanation: Executes part of the module’s workflow.

Line 547: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 548: ‘   def _patient_pseudonym(self, raw_patient_id: str) -> str:‘
Explanation: Defines the _patient_pseudonym callable.

Line 549: ‘   return self.pseudonymizer.identifier("Patient", raw_patient_id)‘
Explanation: Returns the computed result to the caller.

Line 550: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 551: ‘   def _sync_single_patient_scope(self, patient_id: str) -> None:‘
Explanation: Defines the _sync_single_patient_scope callable.

Line 552: ‘   """Restrict the SQLite/CSV output to only this Patient’s resources."""‘
Explanation: Executes part of the module’s workflow.

Line 553: ‘   target = self._patient_pseudonym(patient_id)‘
Explanation: Assigns or computes a value used by later code.

Line 554: ‘   with self.database.connection:‘
Explanation: Controls execution flow for the surrounding operation.

Line 555: ‘   # Delete children before the parent Patient row to satisfy foreign keys.‘
Explanation: Comment documenting the following code or an operational decision.

Line 556: ‘   for table in ("conditions", "observations", "encounters"):‘
Explanation: Controls execution flow for the surrounding operation.

Line 557: ‘   self.database.connection.execute(f"DELETE FROM {table} WHERE patient_id != ?", (target,))‘
Explanation: Runs a SQL statement or schema script against SQLite.

Line 558: ‘   self.database.connection.execute("DELETE FROM patients WHERE patient_id != ?", (target,))‘
Explanation: Runs a SQL statement or schema script against SQLite.

Line 559: ‘   scoped_resources = [‘
Explanation: Assigns or computes a value used by later code.

Line 560: ‘   resource‘
Explanation: Executes part of the module’s workflow.

Line 561: ‘   for resource in self._resources.values()‘
Explanation: Controls execution flow for the surrounding operation.

Line 562: ‘   if resource.get("id") == target‘
Explanation: Controls execution flow for the surrounding operation.

Line 563: ‘   or (resource.get("subject") or {}).get("reference") == f"Patient/{target}"‘
Explanation: Performs or configures an HTTP request to the FHIR service.

Line 564: ‘   ]‘
Explanation: Executes part of the module’s workflow.

Line 565: ‘   self.database.sync(scoped_resources)‘
Explanation: Executes part of the module’s workflow.

Line 566: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 567: ‘   def _repair_encounter_links(self, resources: dict[str, dict[str, Any]]) -> None:‘

Explanation: Defines the _repair_encounter_links callable.

Line 568: ‘   """Migrate old Observation links using Encounter IDs in the local cache."""‘
Explanation: Executes part of the module’s workflow.

Line 569: ‘   for encounter in resources.values():‘
Explanation: Controls execution flow for the surrounding operation.

Line 570: ‘   if encounter.get("resourceType") != "Encounter" or not encounter.get("id"):‘
Explanation: Controls execution flow for the surrounding operation.

Line 571: ‘   continue‘
Explanation: Executes part of the module’s workflow.

Line 572: ‘   identifiers = encounter.get("identifier", [])‘
Explanation: Performs or configures an HTTP request to the FHIR service.

Line 573: ‘   identifier = identifiers[0] if isinstance(identifiers, list) and identifiers else {}‘
Explanation: Assigns or computes a value used by later code.

Line 574: ‘   stored_encounter_id = identifier.get("value") if isinstance(identifier, dict) else None‘
Explanation: Performs or configures an HTTP request to the FHIR service.

Line 575: ‘   if stored_encounter_id:‘
Explanation: Controls execution flow for the surrounding operation.

Line 576: ‘   for legacy_encounter_id in (encounter["id"], f"Encounter/{encounter[’id’]}"):‘
Explanation: Controls execution flow for the surrounding operation.

Line 577: ‘   self.database.connection.execute(‘
Explanation: Runs a SQL statement or schema script against SQLite.

Line 578: ‘   "UPDATE observations SET encounter_id = ? WHERE encounter_id = ?",‘
Explanation: Assigns or computes a value used by later code.

Line 579: ‘   (stored_encounter_id, legacy_encounter_id),‘
Explanation: Executes part of the module’s workflow.

Line 580: ‘   )‘
Explanation: Executes part of the module’s workflow.

Line 581: ‘   self.database.connection.commit()‘
Explanation: Executes part of the module’s workflow.

Line 582: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 583: ‘   def _pseudonymize_resource(self, resource: dict[str, Any]) -> dict[str, Any]:‘
Explanation: Defines the _pseudonymize_resource callable.

Line 584: ‘   if resource.get("_deidentified"):‘
Explanation: Controls execution flow for the surrounding operation.

Line 585: ‘   return resource‘
Explanation: Returns the computed result to the caller.

Line 586: ‘   result = copy.deepcopy(resource)‘
Explanation: Assigns or computes a value used by later code.

Line 587: ‘   resource_type = result["resourceType"]‘
Explanation: Assigns or computes a value used by later code.

Line 588: ‘   raw_id = result["id"]‘
Explanation: Assigns or computes a value used by later code.

Line 589: ‘   patient_raw_id = raw_id if resource_type == "Patient" else _reference_id(result, "subject", "Patient")‘
Explanation: Assigns or computes a value used by later code.

Line 590: ‘   patient_offset = self.pseudonymizer.patient_offset(patient_raw_id) if patient_raw_id else 0‘
Explanation: Assigns or computes a value used by later code.

Line 591: ‘   result["id"] = self.pseudonymizer.identifier(resource_type, raw_id)‘
Explanation: Assigns or computes a value used by later code.

Line 592: ‘   if resource_type == "Patient":‘
Explanation: Controls execution flow for the surrounding operation.

Line 593: ‘   self._raw_patient_ids[result["id"]] = raw_id‘
Explanation: Assigns or computes a value used by later code.

Line 594: ‘   names = result.get("name", [])‘
Explanation: Performs or configures an HTTP request to the FHIR service.

Line 595: ‘   name = names[0] if isinstance(names, list) and names and isinstance(names[0], dict) else {}‘
Explanation: Assigns or computes a value used by later code.

Line 596: ‘   given_names = name.get("given", [])‘
Explanation: Performs or configures an HTTP request to the FHIR service.

Line 597: ‘   result["familyName"] = name.get("family")‘
Explanation: Performs or configures an HTTP request to the FHIR service.

Line 598: ‘   result["givenName"] = " ".join(given_names) if isinstance(given_names, list) else None‘
Explanation: Assigns or computes a value used by later code.

Line 599: ‘   result.pop("name", None)‘
Explanation: Executes part of the module’s workflow.

Line 600: ‘   result.pop("identifier", None)‘
Explanation: Executes part of the module’s workflow.

Line 601: ‘   result.pop("telecom", None)‘
Explanation: Executes part of the module’s workflow.

Line 602: ‘   result.pop("address", None)‘
Explanation: Executes part of the module’s workflow.

Line 603: ‘   result.pop("contact", None)‘
Explanation: Executes part of the module’s workflow.

Line 604: ‘   result.pop("communication", None)‘
Explanation: Executes part of the module’s workflow.

Line 605: ‘   result["dateShiftDays"] = patient_offset‘
Explanation: Assigns or computes a value used by later code.

Line 606: ‘   if isinstance(result.get("birthDate"), str):‘
Explanation: Controls execution flow for the surrounding operation.

Line 607: ‘   result["birthDate"] =
self.pseudonymizer.shift_date(result["birthDate"], patient_offset)‘
Explanation: Assigns or computes a value used by later code.

Line 608: ‘   elif patient_raw_id:‘
Explanation: Controls execution flow for the surrounding operation.

Line 609: ‘   result["subject"] = {"reference":
f"Patient/{self._patient_pseudonym(patient_raw_id)}"}‘
Explanation: Assigns or computes a value used by later code.

Line 610: ‘   if "encounter" in result:‘
Explanation: Controls execution flow for the surrounding operation.

Line 611: ‘   raw_encounter_id = _reference_id(result, "encounter",
"Encounter")‘
Explanation: Assigns or computes a value used by later code.

Line 612: ‘   if raw_encounter_id:‘
Explanation: Controls execution flow for the surrounding operation.

Line 613: ‘   result["encounter"] = {‘
Explanation: Assigns or computes a value used by later code.

Line 614: ‘   "reference":
f"Encounter/{self.pseudonymizer.identifier(’Encounter’, raw_encounter_id)}"‘
Explanation: Executes part of the module’s workflow.

Line 615: ‘   }‘
Explanation: Executes part of the module’s workflow.

Line 616: ‘   for field_name in ("effectiveDateTime", "issued"):‘
Explanation: Controls execution flow for the surrounding operation.

Line 617: ‘   if isinstance(result.get(field_name), str):‘
Explanation: Controls execution flow for the surrounding operation.

Line 618: ‘   result[field_name] =
self.pseudonymizer.shift_date(result[field_name], patient_offset)‘
Explanation: Assigns or computes a value used by later code.

Line 619: ‘   if resource_type == "Encounter":‘
Explanation: Controls execution flow for the surrounding operation.

Line 620: ‘   result.pop("identifier", None)‘
Explanation: Executes part of the module’s workflow.

Line 621: ‘   if isinstance(result.get("period"), dict):‘
Explanation: Controls execution flow for the surrounding operation.

Line 622: ‘   for field_name in ("start", "end"):‘
Explanation: Controls execution flow for the surrounding operation.

Line 623: ‘   if isinstance(result["period"].get(field_name), str):‘
Explanation: Controls execution flow for the surrounding operation.

Line 624: ‘   result["period"][field_name] =
self.pseudonymizer.shift_date(‘
Explanation: Assigns or computes a value used by later code.

Line 625: ‘   result["period"][field_name], patient_offset‘
Explanation: Executes part of the module’s workflow.

Line 626: ‘   )‘
Explanation: Executes part of the module’s workflow.

Line 627: ‘   if resource_type == "Condition":‘
Explanation: Controls execution flow for the surrounding operation.

Line 628: ‘   result.pop("note", None)‘
Explanation: Executes part of the module’s workflow.

Line 629: ‘   for field_name in ("onsetDateTime", "recordedDate"):‘
Explanation: Controls execution flow for the surrounding operation.

Line 630: ‘   if isinstance(result.get(field_name), str):‘
Explanation: Controls execution flow for the surrounding operation.

Line 631: ‘   result[field_name] =
self.pseudonymizer.shift_date(result[field_name], patient_offset)‘
Explanation: Assigns or computes a value used by later code.

Line 632: ‘   result["_deidentified"] = True‘
Explanation: Assigns or computes a value used by later code.

Line 633: ‘   return result‘
Explanation: Returns the computed result to the caller.

Line 634: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 635: ‘   @staticmethod‘
Explanation: Decorator that registers or configures the following definition.

Line 636: ‘   def _make_session(retries: int) -> requests.Session:‘
Explanation: Defines the _make_session callable.

Line 637: ‘   retry_policy = Retry(‘
Explanation: Assigns or computes a value used by later code.

Line 638: ‘   total=retries,‘
Explanation: Assigns or computes a value used by later code.

Line 639: ‘   connect=retries,‘
Explanation: Assigns or computes a value used by later code.

Line 640: ‘   read=retries,‘
Explanation: Assigns or computes a value used by later code.

Line 641: ‘   status=retries,‘
Explanation: Assigns or computes a value used by later code.

Line 642: ‘   backoff_factor=1.0,‘
Explanation: Assigns or computes a value used by later code.

Line 643: ‘   backoff_max=60,‘
Explanation: Assigns or computes a value used by later code.

Line 644: ‘   status_forcelist=(429, 500, 502, 503, 504),‘
Explanation: Assigns or computes a value used by later code.

Line 645: ‘   allowed_methods=frozenset({"GET"}),‘
Explanation: Assigns or computes a value used by later code.

Line 646: ‘   respect_retry_after_header=True,‘
Explanation: Assigns or computes a value used by later code.

Line 647: ‘   )‘
Explanation: Executes part of the module’s workflow.

Line 648: ‘   session = requests.Session()‘
Explanation: Performs or configures an HTTP request to the FHIR service.

Line 649: ‘   session.headers["User-Agent"] = "fhir-resource-retriever/1.0 (polite cache-first client)"‘
Explanation: Assigns or computes a value used by later code.

Line 650: ‘   adapter = HTTPAdapter(max_retries=retry_policy)‘
Explanation: Assigns or computes a value used by later code.

Line 651: ‘   session.mount("http://", adapter)‘
Explanation: Executes part of the module’s workflow.

Line 652: ‘   session.mount("https://", adapter)‘
Explanation: Executes part of the module’s workflow.

Line 653: ‘   return session‘
Explanation: Returns the computed result to the caller.

Line 654: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 655: ‘   @staticmethod‘
Explanation: Decorator that registers or configures the following definition.

Line 656: ‘   def _load_json(path: Path, default: Any) -> Any:‘
Explanation: Defines the _load_json callable.

Line 657: ‘   if not path.exists():‘
Explanation: Controls execution flow for the surrounding operation.

Line 658: ‘   return default‘
Explanation: Returns the computed result to the caller.

Line 659: ‘   if path.suffix == ".gz":‘
Explanation: Controls execution flow for the surrounding operation.

Line 660: ‘   file_context = gzip.open(path, "rt", encoding="utf-8")‘
Explanation: Assigns or computes a value used by later code.

Line 661: ‘   else:‘
Explanation: Controls execution flow for the surrounding operation.

Line 662: ‘   file_context = path.open(encoding="utf-8")‘
Explanation: Assigns or computes a value used by later code.

Line 663: ‘   with file_context as file:‘
Explanation: Controls execution flow for the surrounding operation.

Line 664: ‘   return json.load(file)‘

Explanation: Returns the computed result to the caller.

Line 665: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 666: ‘   @staticmethod‘
Explanation: Decorator that registers or configures the following definition.

Line 667: ‘   def _write_json(path: Path, value: Any) -> None:‘
Explanation: Defines the _write_json callable.

Line 668: ‘   path.parent.mkdir(parents=True, exist_ok=True)‘
Explanation: Assigns or computes a value used by later code.

Line 669: ‘   temporary_path = path.with_suffix(path.suffix + ".tmp")‘
Explanation: Assigns or computes a value used by later code.

Line 670: ‘   if path.suffix == ".gz":‘
Explanation: Controls execution flow for the surrounding operation.

Line 671: ‘   file_context = gzip.open(temporary_path, "wt", encoding="utf-8")‘
Explanation: Assigns or computes a value used by later code.

Line 672: ‘   else:‘
Explanation: Controls execution flow for the surrounding operation.

Line 673: ‘   file_context = temporary_path.open("w", encoding="utf-8")‘
Explanation: Assigns or computes a value used by later code.

Line 674: ‘   with file_context as file:‘
Explanation: Controls execution flow for the surrounding operation.

Line 675: ‘   json.dump(value, file, separators=(",", ":"), sort_keys=True)‘
Explanation: Assigns or computes a value used by later code.

Line 676: ‘   file.write("\n")‘
Explanation: Executes part of the module’s workflow.

Line 677: ‘   temporary_path.replace(path)‘
Explanation: Executes part of the module’s workflow.

Line 678: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 679: ‘   def _save_resources(self) -> None:‘
Explanation: Defines the _save_resources callable.

Line 680: ‘   self._write_json(self.resources_path, self._resources)‘
Explanation: Executes part of the module’s workflow.

Line 681: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 682: ‘   def _save_checkpoint(self) -> None:‘
Explanation: Defines the _save_checkpoint callable.

Line 683: ‘   self._write_json(‘
Explanation: Executes part of the module’s workflow.

Line 684: ‘   self.checkpoint_path,‘
Explanation: Executes part of the module’s workflow.

Line 685: ‘   {"endpoint": self.endpoint, "completed": sorted(self._completed)},‘
Explanation: Executes part of the module’s workflow.

Line 686: ‘   )‘
Explanation: Executes part of the module’s workflow.

Line 687: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 688: ‘   def _remember(self, resources: Iterable[dict[str, Any]], report: RetrievalReport) -> None:‘
Explanation: Defines the _remember callable.

Line 689: ‘   changed = False‘
Explanation: Assigns or computes a value used by later code.

Line 690: ‘   valid_resources = []‘
Explanation: Assigns or computes a value used by later code.

Line 691: ‘   for resource in resources:‘
Explanation: Controls execution flow for the surrounding operation.

Line 692: ‘   resource = self._pseudonymize_resource(resource)‘
Explanation: Assigns or computes a value used by later code.

Line 693: ‘   resource_type = resource.get("resourceType")‘
Explanation: Performs or configures an HTTP request to the FHIR service.

Line 694: ‘   resource_id = resource.get("id")‘
Explanation: Performs or configures an HTTP request to the FHIR service.

Line 695: ‘   if not resource_type or not resource_id:‘
Explanation: Controls execution flow for the surrounding operation.

Line 696: ‘   LOGGER.warning("Skipping resource without resourceType and id: %r", resource)‘
Explanation: Executes part of the module’s workflow.

Line 697: ‘   continue‘
Explanation: Executes part of the module’s workflow.

Line 698: ‘   valid_resources.append(resource)‘
Explanation: Executes part of the module’s workflow.

Line 699: ‘   key = f"{resource_type}/{resource_id}"‘
Explanation: Assigns or computes a value used by later code.

Line 700: ‘   if key not in self._resources:‘
Explanation: Controls execution flow for the surrounding operation.

Line 701: ‘   report.resources.setdefault(resource_type, 0)‘
Explanation: Executes part of the module’s workflow.

Line 702: ‘   report.resources[resource_type] += 1‘
Explanation: Assigns or computes a value used by later code.

Line 703: ‘   changed = True‘
Explanation: Assigns or computes a value used by later code.

Line 704: ‘   elif self._resources[key] != resource:‘
Explanation: Controls execution flow for the surrounding operation.

Line 705: ‘   changed = True‘
Explanation: Assigns or computes a value used by later code.

Line 706: ‘   self._resources[key] = resource‘
Explanation: Assigns or computes a value used by later code.

Line 707: ‘   self.database.sync(valid_resources)‘
Explanation: Executes part of the module’s workflow.

Line 708: ‘   if changed:‘
Explanation: Controls execution flow for the surrounding operation.

Line 709: ‘   self._save_resources()‘
Explanation: Executes part of the module’s workflow.

Line 710: ‘   if valid_resources:‘
Explanation: Controls execution flow for the surrounding operation.

Line 711: ‘   self.database.export_csv(self.output_dir)‘
Explanation: Executes part of the module’s workflow.

Line 712: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 713: ‘   def _fetch_pages(‘
Explanation: Defines the _fetch_pages callable.

Line 714: ‘   self,‘
Explanation: Executes part of the module’s workflow.

Line 715: ‘   path_or_url: str,‘
Explanation: Executes part of the module’s workflow.

Line 716: ‘   params: dict[str, str] | None,‘
Explanation: Executes part of the module’s workflow.

Line 717: ‘   report: RetrievalReport,‘
Explanation: Executes part of the module’s workflow.

Line 718: ‘   query_name: str | None = None,‘
Explanation: Assigns or computes a value used by later code.

Line 719: ‘   max_resources: int | None = None,‘
Explanation: Assigns or computes a value used by later code.

Line 720: ‘   ) -> bool:‘
Explanation: Executes part of the module’s workflow.

Line 721: ‘   url = urljoin(self.endpoint, path_or_url)‘
Explanation: Assigns or computes a value used by later code.

Line 722: ‘   request_params = params‘
Explanation: Assigns or computes a value used by later code.

Line 723: ‘   received_resources = 0‘
Explanation: Assigns or computes a value used by later code.

Line 724: ‘   while url:‘
Explanation: Controls execution flow for the surrounding operation.

Line 725: ‘   try:‘

Explanation: Controls execution flow for the surrounding operation.

Line 726: ‘   response = self.session.get(url, params=request_params, timeout=self.timeout)‘
Explanation: Performs or configures an HTTP request to the FHIR service.

Line 727: ‘   response.raise_for_status()‘
Explanation: Executes part of the module’s workflow.

Line 728: ‘   bundle = response.json()‘
Explanation: Assigns or computes a value used by later code.

Line 729: ‘   if bundle.get("resourceType") != "Bundle":‘
Explanation: Controls execution flow for the surrounding operation.

Line 730: ‘   raise ValueError("FHIR search response was not a Bundle")‘
Explanation: Raises an error so invalid or unavailable work is reported clearly.

Line 731: ‘   entries = bundle.get("entry", [])‘
Explanation: Performs or configures an HTTP request to the FHIR service.

Line 732: ‘   if max_resources is not None:‘
Explanation: Controls execution flow for the surrounding operation.

Line 733: ‘   remaining = max_resources - received_resources‘
Explanation: Assigns or computes a value used by later code.

Line 734: ‘   entries = entries[:remaining]‘
Explanation: Assigns or computes a value used by later code.

Line 735: ‘   self._remember(‘
Explanation: Executes part of the module’s workflow.

Line 736: ‘   (entry["resource"] for entry in entries if "resource" in entry), report‘
Explanation: Executes part of the module’s workflow.

Line 737: ‘   )‘
Explanation: Executes part of the module’s workflow.

Line 738: ‘   received_resources += len(entries)‘
Explanation: Assigns or computes a value used by later code.

Line 739: ‘   if max_resources is not None and received_resources >= max_resources:‘
Explanation: Controls execution flow for the surrounding operation.

Line 740: ‘   return True‘
Explanation: Returns the computed result to the caller.

Line 741: ‘   url = next(‘
Explanation: Assigns or computes a value used by later code.

Line 742: ‘   (link["url"] for link in bundle.get("link", []) if link.get("relation") == "next"),‘
Explanation: Performs or configures an HTTP request to the FHIR service.

Line 743: ‘   None,‘
Explanation: Executes part of the module’s workflow.

Line 744: ‘   )‘

Explanation: Executes part of the module’s workflow.

Line 745: ‘   request_params = None‘
Explanation: Assigns or computes a value used by later code.

Line 746: ‘   except (requests.RequestException, ValueError, KeyError,
TypeError, json.JSONDecodeError) as error:‘
Explanation: Controls execution flow for the surrounding operation.

Line 747: ‘   failed_query = query_name or path_or_url‘
Explanation: Assigns or computes a value used by later code.

Line 748: ‘   report.failures.append(RetrievalFailure(query=failed_query,
error=str(error)))‘
Explanation: Assigns or computes a value used by later code.

Line 749: ‘   LOGGER.warning("FHIR request failed for %s: %s",
failed_query, error)‘
Explanation: Executes part of the module’s workflow.

Line 750: ‘   return False‘
Explanation: Returns the computed result to the caller.

Line 751: ‘   return True‘
Explanation: Returns the computed result to the caller.

Line 752: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 753: ‘   def _load_patients(‘
Explanation: Defines the _load_patients callable.

Line 754: ‘   self, report: RetrievalReport, patient_id: str | None, limit: int |
None = None‘
Explanation: Assigns or computes a value used by later code.

Line 755: ‘   ) -> bool:‘
Explanation: Executes part of the module’s workflow.

Line 756: ‘   query = (‘
Explanation: Assigns or computes a value used by later code.

Line 757: ‘   f"Patient?limit={limit}"‘
Explanation: Assigns or computes a value used by later code.

Line 758: ‘   if patient_id is None and limit is not None‘
Explanation: Controls execution flow for the surrounding operation.

Line 759: ‘   else PATIENT_QUERY if patient_id is None else
f"Patient?_id={self._patient_pseudonym(patient_id)}"‘
Explanation: Assigns or computes a value used by later code.

Line 760: ‘   )‘
Explanation: Executes part of the module’s workflow.

Line 761: ‘   count = min(self.page_size, limit) if limit is not None else
self.page_size‘
Explanation: Assigns or computes a value used by later code.

Line 762: ‘   params = {"_count": str(count)}‘
Explanation: Assigns or computes a value used by later code.

Line 763: ‘   if patient_id is not None:‘
Explanation: Controls execution flow for the surrounding operation.

Line 764: ‘   params["_id"] = patient_id‘
Explanation: Assigns or computes a value used by later code.

Line 765: ‘   if query in self._completed:‘
Explanation: Controls execution flow for the surrounding operation.

Line 766: ‘   if patient_id is None or
f"Patient/{self._patient_pseudonym(patient_id)}" in self._resources:‘
Explanation: Controls execution flow for the surrounding operation.

Line 767: ‘   if patient_id is not None:‘
Explanation: Controls execution flow for the surrounding operation.

Line 768: ‘   report.patient_pseudonyms[patient_id] =
self._patient_pseudonym(patient_id)‘
Explanation: Assigns or computes a value used by later code.

Line 769: ‘   self._sync_single_patient_scope(patient_id)‘
Explanation: Executes part of the module’s workflow.

Line 770: ‘   return True‘
Explanation: Returns the computed result to the caller.

Line 771: ‘   # A checkpoint without the requested cached Patient cannot
satisfy a rerun.‘
Explanation: Comment documenting the following code or an operational decision.

Line 772: ‘   self._completed.remove(query)‘
Explanation: Executes part of the module’s workflow.

Line 773: ‘   self._save_checkpoint()‘
Explanation: Executes part of the module’s workflow.

Line 774: ‘   if not self._fetch_pages(PATIENT_QUERY, params, report, query,
limit):‘
Explanation: Controls execution flow for the surrounding operation.

Line 775: ‘   return False‘
Explanation: Returns the computed result to the caller.

Line 776: ‘   self._completed.add(query)‘
Explanation: Executes part of the module’s workflow.

Line 777: ‘   self._save_checkpoint()‘
Explanation: Executes part of the module’s workflow.

Line 778: ‘   if patient_id is not None:‘
Explanation: Controls execution flow for the surrounding operation.

Line 779: ‘   report.patient_pseudonyms[patient_id] =
self._patient_pseudonym(patient_id)‘
Explanation: Assigns or computes a value used by later code.

Line 780: ‘   self._sync_single_patient_scope(patient_id)‘
Explanation: Executes part of the module’s workflow.

Line 781: ‘   return True‘
Explanation: Returns the computed result to the caller.

Line 782: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 783: ‘   def _patient_ids(self, patient_id: str | None) -> list[str]:‘
Explanation: Defines the _patient_ids callable.

Line 784: ‘   if patient_id is not None:‘
Explanation: Controls execution flow for the surrounding operation.

Line 785: ‘   return [patient_id] if
f"Patient/{self._patient_pseudonym(patient_id)}" in self._resources else []‘
Explanation: Returns the computed result to the caller.

Line 786: ‘   return sorted(self._raw_patient_ids.values())‘
Explanation: Returns the computed result to the caller.

Line 787: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 788: ‘   def _load_related(‘
Explanation: Defines the _load_related callable.

Line 789: ‘   self, patient_ids: Iterable[str], resource_types: Iterable[str],
report: RetrievalReport‘
Explanation: Executes part of the module’s workflow.

Line 790: ‘   ) -> None:‘
Explanation: Executes part of the module’s workflow.

Line 791: ‘   for current_patient_id in patient_ids:‘
Explanation: Controls execution flow for the surrounding operation.

Line 792: ‘   for resource_type in resource_types:‘
Explanation: Controls execution flow for the surrounding operation.

Line 793: ‘   query =
f"{resource_type}?patient={self._patient_pseudonym(current_patient_id)}"‘
Explanation: Assigns or computes a value used by later code.

Line 794: ‘   if query in self._completed:‘
Explanation: Controls execution flow for the surrounding operation.

Line 795: ‘   continue‘
Explanation: Executes part of the module’s workflow.

Line 796: ‘   if self._fetch_pages(‘
Explanation: Controls execution flow for the surrounding operation.

Line 797: ‘   resource_type,‘
Explanation: Executes part of the module’s workflow.

Line 798: ‘   {"patient": current_patient_id, "_count":
str(self.page_size)},‘
Explanation: Executes part of the module’s workflow.

Line 799: ‘   report,‘
Explanation: Executes part of the module’s workflow.

Line 800: ‘   query,‘
Explanation: Executes part of the module’s workflow.

Line 801: ‘   ):‘

Explanation: Executes part of the module’s workflow.

Line 802: ‘   self._completed.add(query)‘
Explanation: Executes part of the module’s workflow.

Line 803: ‘   self._save_checkpoint()‘
Explanation: Executes part of the module’s workflow.

Line 804: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 805: ‘   def get_all_patients(self, limit: int | None = None) -> RetrievalReport:‘
Explanation: Defines the get_all_patients callable.

Line 806: ‘   """Retrieve all Patient resources and store them in the local cache and database."""‘
Explanation: Executes part of the module’s workflow.

Line 807: ‘   report = RetrievalReport()‘
Explanation: Assigns or computes a value used by later code.

Line 808: ‘   self._load_patients(report, None, limit or self.patient_limit)‘
Explanation: Executes part of the module’s workflow.

Line 809: ‘   return report‘
Explanation: Returns the computed result to the caller.

Line 810: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 811: ‘   def get_patient(self, patient_id: str) -> RetrievalReport:‘
Explanation: Defines the get_patient callable.

Line 812: ‘   """Retrieve one Patient by ID, even when it is not yet in the local database."""‘
Explanation: Executes part of the module’s workflow.

Line 813: ‘   report = RetrievalReport()‘
Explanation: Assigns or computes a value used by later code.

Line 814: ‘   self._load_patients(report, patient_id)‘
Explanation: Executes part of the module’s workflow.

Line 815: ‘   return report‘
Explanation: Returns the computed result to the caller.

Line 816: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 817: ‘   def get_all_observations_and_encounters(self) -> RetrievalReport:‘
Explanation: Defines the get_all_observations_and_encounters callable.

Line 818: ‘   """Retrieve all Patients, then all of their Observation and Encounter resources."""‘
Explanation: Executes part of the module’s workflow.

Line 819: ‘   report = RetrievalReport()‘
Explanation: Assigns or computes a value used by later code.

Line 820: ‘   if self._load_patients(report, None):‘
Explanation: Controls execution flow for the surrounding operation.

Line 821: ‘   self._load_related(self._patient_ids(None), ("Observation", "Encounter"), report)‘
Explanation: Executes part of the module’s workflow.

Line 822: ‘   return report‘
Explanation: Returns the computed result to the caller.

Line 823: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 824: ‘   def get_all_observations(self) -> RetrievalReport:‘
Explanation: Defines the get_all_observations callable.

Line 825: ‘   """Retrieve all Patients, then all of their Observation resources only."""‘
Explanation: Executes part of the module’s workflow.

Line 826: ‘   report = RetrievalReport()‘
Explanation: Assigns or computes a value used by later code.

Line 827: ‘   if self._load_patients(report, None):‘
Explanation: Controls execution flow for the surrounding operation.

Line 828: ‘   self._load_related(self._patient_ids(None), ("Observation",), report)‘
Explanation: Executes part of the module’s workflow.

Line 829: ‘   return report‘
Explanation: Returns the computed result to the caller.

Line 830: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 831: ‘   def get_all_encounters(self) -> RetrievalReport:‘
Explanation: Defines the get_all_encounters callable.

Line 832: ‘   """Retrieve all Patients, then all of their Encounter resources only."""‘
Explanation: Executes part of the module’s workflow.

Line 833: ‘   report = RetrievalReport()‘
Explanation: Assigns or computes a value used by later code.

Line 834: ‘   if self._load_patients(report, None):‘
Explanation: Controls execution flow for the surrounding operation.

Line 835: ‘   self._load_related(self._patient_ids(None), ("Encounter",), report)‘
Explanation: Executes part of the module’s workflow.

Line 836: ‘   return report‘
Explanation: Returns the computed result to the caller.

Line 837: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 838: ‘   def get_all_conditions(self) -> RetrievalReport:‘
Explanation: Defines the get_all_conditions callable.

Line 839: ‘   """Retrieve all Patients, then all of their Condition resources only."""‘
Explanation: Executes part of the module’s workflow.

Line 840: ‘   report = RetrievalReport()‘
Explanation: Assigns or computes a value used by later code.

Line 841: ‘   if self._load_patients(report, None):‘
Explanation: Controls execution flow for the surrounding operation.

Line 842: ‘   self._load_related(self._patient_ids(None), ("Condition",), report)‘
Explanation: Executes part of the module’s workflow.

Line 843: ‘   return report‘
Explanation: Returns the computed result to the caller.

Line 844: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 845: ‘   def get_related_for_all_patients(self, resource_types: Iterable[str]) -> RetrievalReport:‘
Explanation: Defines the get_related_for_all_patients callable.

Line 846: ‘   """Retrieve all Patients and the selected related resource types."""‘
Explanation: Executes part of the module’s workflow.

Line 847: ‘   report = RetrievalReport()‘
Explanation: Assigns or computes a value used by later code.

Line 848: ‘   if self._load_patients(report, None):‘
Explanation: Controls execution flow for the surrounding operation.

Line 849: ‘   self._load_related(self._patient_ids(None), resource_types, report)‘
Explanation: Executes part of the module’s workflow.

Line 850: ‘   return report‘
Explanation: Returns the computed result to the caller.

Line 851: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 852: ‘   def get_observations_and_encounters_for_patient(self, patient_id: str) -> RetrievalReport:‘
Explanation: Defines the get_observations_and_encounters_for_patient callable.

Line 853: ‘   """Retrieve one Patient and only that Patient’s Observation and Encounter resources."""‘
Explanation: Executes part of the module’s workflow.

Line 854: ‘   report = RetrievalReport()‘
Explanation: Assigns or computes a value used by later code.

Line 855: ‘   if self._load_patients(report, patient_id):‘
Explanation: Controls execution flow for the surrounding operation.

Line 856: ‘   self._load_related(self._patient_ids(patient_id), ("Observation", "Encounter"), report)‘
Explanation: Executes part of the module’s workflow.

Line 857: ‘   return report‘
Explanation: Returns the computed result to the caller.

Line 858: ‘<blank>‘

Explanation: Blank line used to separate logical sections.

Line 859: ‘   def get_observations_for_patient(self, patient_id: str) ->
RetrievalReport:‘
Explanation: Defines the get_observations_for_patient callable.

Line 860: ‘   """Retrieve one Patient and only that Patient’s Observation
resources."""‘
Explanation: Executes part of the module’s workflow.

Line 861: ‘   report = RetrievalReport()‘
Explanation: Assigns or computes a value used by later code.

Line 862: ‘   if self._load_patients(report, patient_id):‘
Explanation: Controls execution flow for the surrounding operation.

Line 863: ‘   self._load_related(self._patient_ids(patient_id),
("Observation",), report)‘
Explanation: Executes part of the module’s workflow.

Line 864: ‘   return report‘
Explanation: Returns the computed result to the caller.

Line 865: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 866: ‘   def get_encounters_for_patient(self, patient_id: str) ->
RetrievalReport:‘
Explanation: Defines the get_encounters_for_patient callable.

Line 867: ‘   """Retrieve one Patient and only that Patient’s Encounter
resources."""‘
Explanation: Executes part of the module’s workflow.

Line 868: ‘   report = RetrievalReport()‘
Explanation: Assigns or computes a value used by later code.

Line 869: ‘   if self._load_patients(report, patient_id):‘
Explanation: Controls execution flow for the surrounding operation.

Line 870: ‘   self._load_related(self._patient_ids(patient_id), ("Encounter",),
report)‘
Explanation: Executes part of the module’s workflow.

Line 871: ‘   return report‘
Explanation: Returns the computed result to the caller.

Line 872: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 873: ‘   def get_conditions_for_patient(self, patient_id: str) ->
RetrievalReport:‘
Explanation: Defines the get_conditions_for_patient callable.

Line 874: ‘   """Retrieve one Patient and only that Patient’s Condition
resources."""‘
Explanation: Executes part of the module’s workflow.

Line 875: ‘   report = RetrievalReport()‘
Explanation: Assigns or computes a value used by later code.

Line 876: ‘   if self._load_patients(report, patient_id):‘

Explanation: Controls execution flow for the surrounding operation.

Line 877: ‘   self._load_related(self._patient_ids(patient_id), ("Condition",), report)‘
Explanation: Executes part of the module’s workflow.

Line 878: ‘   return report‘
Explanation: Returns the computed result to the caller.

Line 879: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 880: ‘   def get_related_for_patient(‘
Explanation: Defines the get_related_for_patient callable.

Line 881: ‘   self, patient_id: str, resource_types: Iterable[str]‘
Explanation: Executes part of the module’s workflow.

Line 882: ‘   ) -> RetrievalReport:‘
Explanation: Executes part of the module’s workflow.

Line 883: ‘   """Retrieve one Patient and the selected related resource types."""‘
Explanation: Executes part of the module’s workflow.

Line 884: ‘   report = RetrievalReport()‘
Explanation: Assigns or computes a value used by later code.

Line 885: ‘   if self._load_patients(report, patient_id):‘
Explanation: Controls execution flow for the surrounding operation.

Line 886: ‘   self._load_related(self._patient_ids(patient_id), resource_types, report)‘
Explanation: Executes part of the module’s workflow.

Line 887: ‘   return report‘
Explanation: Returns the computed result to the caller.

Line 888: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 889: ‘   def retrieve(self) -> RetrievalReport:‘
Explanation: Defines the retrieve callable.

Line 890: ‘   """Fetch Patients, then their missing Condition and Observation searches."""‘
Explanation: Executes part of the module’s workflow.

Line 891: ‘   report = RetrievalReport()‘
Explanation: Assigns or computes a value used by later code.

Line 892: ‘   try:‘
Explanation: Controls execution flow for the surrounding operation.

Line 893: ‘   if self._load_patients(report, self.patient_id):‘
Explanation: Controls execution flow for the surrounding operation.

Line 894: ‘   self._load_related(self._patient_ids(self.patient_id), ("Condition", "Observation"), report)‘
Explanation: Executes part of the module’s workflow.

Line 895: ‘   return report‘
Explanation: Returns the computed result to the caller.

Line 896: ‘   finally:‘
Explanation: Controls execution flow for the surrounding operation.

Line 897: ‘   self.database.export_csv(self.output_dir)‘
Explanation: Executes part of the module’s workflow.

Line 898: ‘   self.database.close()‘
Explanation: Executes part of the module’s workflow.

Line 899: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 900: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 901: ‘def _run_operation(operation: str, *operation_args: str, **retriever_options:
Any) -> RetrievalReport:‘
Explanation: Defines the _run_operation callable.

Line 902: ‘   retriever = FHIRRetriever(**retriever_options)‘
Explanation: Assigns or computes a value used by later code.

Line 903: ‘   try:‘
Explanation: Controls execution flow for the surrounding operation.

Line 904: ‘   return getattr(retriever, operation)(*operation_args)‘
Explanation: Returns the computed result to the caller.

Line 905: ‘   finally:‘
Explanation: Controls execution flow for the surrounding operation.

Line 906: ‘   retriever.database.export_csv(retriever.output_dir)‘
Explanation: Executes part of the module’s workflow.

Line 907: ‘   retriever.database.close()‘
Explanation: Executes part of the module’s workflow.

Line 908: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 909: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 910: ‘def get_all_patients(limit: int | None = None, **retriever_options: Any) ->
RetrievalReport:‘
Explanation: Defines the get_all_patients callable.

Line 911: ‘   """Retrieve every Patient resource and store it in the local SQLite
database."""‘
Explanation: Executes part of the module’s workflow.

Line 912: ‘   return _run_operation("get_all_patients", limit, **retriever_options)‘
Explanation: Returns the computed result to the caller.

Line 913: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 914: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 915: ‘def get_patient(patient_id: str, **retriever_options: Any) ->

RetrievalReport:‘
Explanation: Defines the get_patient callable.

Line 916: ‘   """Retrieve one Patient by ID and store it even if it is not already cached."""‘
Explanation: Executes part of the module’s workflow.

Line 917: ‘   return _run_operation("get_patient", patient_id, **retriever_options)‘
Explanation: Returns the computed result to the caller.

Line 918: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 919: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 920: ‘def get_all_observations_and_encounters(**retriever_options: Any) -> RetrievalReport:‘
Explanation: Defines the get_all_observations_and_encounters callable.

Line 921: ‘   """Retrieve all Patients and each Patient’s Observation and Encounter resources."""‘
Explanation: Executes part of the module’s workflow.

Line 922: ‘   return _run_operation("get_all_observations_and_encounters", **retriever_options)‘
Explanation: Returns the computed result to the caller.

Line 923: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 924: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 925: ‘def get_all_observations(**retriever_options: Any) -> RetrievalReport:‘
Explanation: Defines the get_all_observations callable.

Line 926: ‘   """Retrieve all Patients and each Patient’s Observation resources only."""‘
Explanation: Executes part of the module’s workflow.

Line 927: ‘   return _run_operation("get_all_observations", **retriever_options)‘
Explanation: Returns the computed result to the caller.

Line 928: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 929: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 930: ‘def get_all_encounters(**retriever_options: Any) -> RetrievalReport:‘
Explanation: Defines the get_all_encounters callable.

Line 931: ‘   """Retrieve all Patients and each Patient’s Encounter resources only."""‘
Explanation: Executes part of the module’s workflow.

Line 932: ‘   return _run_operation("get_all_encounters", **retriever_options)‘
Explanation: Returns the computed result to the caller.

Line 933: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 934: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 935: ‘def get_all_conditions(**retriever_options: Any) -> RetrievalReport:‘
Explanation: Defines the get_all_conditions callable.

Line 936: ‘   """Retrieve all Patients and each Patient’s Condition resources only."""‘
Explanation: Executes part of the module’s workflow.

Line 937: ‘   return _run_operation("get_all_conditions", **retriever_options)‘
Explanation: Returns the computed result to the caller.

Line 938: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 939: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 940: ‘def get_related_for_all_patients(‘
Explanation: Defines the get_related_for_all_patients callable.

Line 941: ‘   resource_types: Iterable[str], **retriever_options: Any‘
Explanation: Executes part of the module’s workflow.

Line 942: ‘) -> RetrievalReport:‘
Explanation: Executes part of the module’s workflow.

Line 943: ‘   """Retrieve all Patients and selected Condition, Observation, and
Encounter resources."""‘
Explanation: Executes part of the module’s workflow.

Line 944: ‘   return _run_operation("get_related_for_all_patients", resource_types,
**retriever_options)‘
Explanation: Returns the computed result to the caller.

Line 945: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 946: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 947: ‘def get_observations_and_encounters_for_patient(‘
Explanation: Defines the get_observations_and_encounters_for_patient callable.

Line 948: ‘   patient_id: str, **retriever_options: Any‘
Explanation: Executes part of the module’s workflow.

Line 949: ‘) -> RetrievalReport:‘
Explanation: Executes part of the module’s workflow.

Line 950: ‘   """Retrieve one Patient’s Observation and Encounter resources."""‘
Explanation: Executes part of the module’s workflow.

Line 951: ‘   return _run_operation("get_observations_and_encounters_for_patient",
patient_id, **retriever_options)‘
Explanation: Returns the computed result to the caller.

Line 952: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 953: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 954: ‘def get_observations_for_patient(patient_id: str, **retriever_options: Any) -> RetrievalReport:‘
Explanation: Defines the get_observations_for_patient callable.

Line 955: ‘   """Retrieve one Patient’s Observation resources only."""‘
Explanation: Executes part of the module’s workflow.

Line 956: ‘   return _run_operation("get_observations_for_patient", patient_id, **retriever_options)‘
Explanation: Returns the computed result to the caller.

Line 957: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 958: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 959: ‘def get_encounters_for_patient(patient_id: str, **retriever_options: Any) -> RetrievalReport:‘
Explanation: Defines the get_encounters_for_patient callable.

Line 960: ‘   """Retrieve one Patient’s Encounter resources only."""‘
Explanation: Executes part of the module’s workflow.

Line 961: ‘   return _run_operation("get_encounters_for_patient", patient_id, **retriever_options)‘
Explanation: Returns the computed result to the caller.

Line 962: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 963: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 964: ‘def get_conditions_for_patient(patient_id: str, **retriever_options: Any) -> RetrievalReport:‘
Explanation: Defines the get_conditions_for_patient callable.

Line 965: ‘   """Retrieve one Patient’s Condition resources only."""‘
Explanation: Executes part of the module’s workflow.

Line 966: ‘   return _run_operation("get_conditions_for_patient", patient_id, **retriever_options)‘
Explanation: Returns the computed result to the caller.

Line 967: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 968: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 969: ‘def get_related_for_patient(‘
Explanation: Defines the get_related_for_patient callable.

Line 970: ‘   patient_id: str, resource_types: Iterable[str], **retriever_options: Any‘
Explanation: Executes part of the module’s workflow.

Line 971: ‘) -> RetrievalReport:‘
Explanation: Executes part of the module’s workflow.

Line 972: ‘   """Retrieve one Patient and selected Condition, Observation, and

Encounter resources."""‘
Explanation: Executes part of the module’s workflow.

Line 973: ‘   return _run_operation("get_related_for_patient", patient_id,
resource_types, **retriever_options)‘
Explanation: Returns the computed result to the caller.

Line 974: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 975: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 976: ‘def main() -> int:‘
Explanation: Defines the main callable.

Line 977: ‘   parser = argparse.ArgumentParser(description=__doc__)‘
Explanation: Assigns or computes a value used by later code.

Line 978: ‘   parser.add_argument("--endpoint", default=DEFAULT_ENDPOINT)‘
Explanation: Assigns or computes a value used by later code.

Line 979: ‘   parser.add_argument("--output-dir", default="fhir_output")‘
Explanation: Assigns or computes a value used by later code.

Line 980: ‘   parser.add_argument("--connect-timeout", type=float, default=5.0)‘
Explanation: Assigns or computes a value used by later code.

Line 981: ‘   parser.add_argument("--read-timeout", type=float, default=30.0)‘
Explanation: Assigns or computes a value used by later code.

Line 982: ‘   parser.add_argument("--retries", type=int, default=2)‘
Explanation: Assigns or computes a value used by later code.

Line 983: ‘   parser.add_argument("--page-size", type=int, default=50)‘
Explanation: Assigns or computes a value used by later code.

Line 984: ‘   parser.add_argument("--limit", type=int, help="Maximum number of Patients
to retrieve with --all-patients")‘
Explanation: Assigns or computes a value used by later code.

Line 985: ‘   parser.add_argument("--refresh", action="store_true", help="Deliberately
refresh the local cache")‘
Explanation: Assigns or computes a value used by later code.

Line 986: ‘   parser.add_argument("--database", help="SQLite database path (default:
OUTPUT_DIR/fhir_resources.sqlite3)")‘
Explanation: Assigns or computes a value used by later code.

Line 987: ‘   parser.add_argument(‘
Explanation: Executes part of the module’s workflow.

Line 988: ‘   "--observation",‘
Explanation: Executes part of the module’s workflow.

Line 989: ‘   action="store_true",‘
Explanation: Assigns or computes a value used by later code.

Line 990: ‘   help="Retrieve Observations only, without Encounter resources",‘
Explanation: Assigns or computes a value used by later code.

Line 991: ‘   )‘

Explanation: Executes part of the module’s workflow.

Line 992: ‘   parser.add_argument(‘
Explanation: Executes part of the module’s workflow.

Line 993: ‘   "--encounter",‘
Explanation: Executes part of the module’s workflow.

Line 994: ‘   action="store_true",‘
Explanation: Assigns or computes a value used by later code.

Line 995: ‘   help="Retrieve Encounters only, without Observation resources",‘
Explanation: Assigns or computes a value used by later code.

Line 996: ‘   )‘
Explanation: Executes part of the module’s workflow.

Line 997: ‘   parser.add_argument("--condition", action="store_true", help="Retrieve Conditions only")‘
Explanation: Assigns or computes a value used by later code.

Line 998: ‘   actions = parser.add_mutually_exclusive_group()‘
Explanation: Assigns or computes a value used by later code.

Line 999: ‘   actions.add_argument("--all-patients", action="store_true", help="Retrieve and store all Patients")‘
Explanation: Assigns or computes a value used by later code.

Line 1000: ‘   actions.add_argument("--patient-id", help="Retrieve and store one Patient by FHIR ID")‘
Explanation: Assigns or computes a value used by later code.

Line 1001: ‘   actions.add_argument(‘
Explanation: Executes part of the module’s workflow.

Line 1002: ‘   "--all-observations-encounters",‘
Explanation: Executes part of the module’s workflow.

Line 1003: ‘   action="store_true",‘
Explanation: Assigns or computes a value used by later code.

Line 1004: ‘   help="Retrieve all Patients plus their Observations and Encounters",‘
Explanation: Assigns or computes a value used by later code.

Line 1005: ‘   )‘
Explanation: Executes part of the module’s workflow.

Line 1006: ‘   actions.add_argument(‘
Explanation: Executes part of the module’s workflow.

Line 1007: ‘   "--patient-observations-encounters",‘
Explanation: Executes part of the module’s workflow.

Line 1008: ‘   metavar="PATIENT_ID",‘
Explanation: Assigns or computes a value used by later code.

Line 1009: ‘   help="Retrieve one Patient plus its Observations and Encounters",‘
Explanation: Assigns or computes a value used by later code.

Line 1010: ‘   )‘
Explanation: Executes part of the module’s workflow.

Line 1011: ‘   arguments = parser.parse_args()‘
Explanation: Assigns or computes a value used by later code.

Line 1012: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 1013: ‘   options = {‘
Explanation: Assigns or computes a value used by later code.

Line 1014: ‘   "endpoint": arguments.endpoint,‘
Explanation: Executes part of the module’s workflow.

Line 1015: ‘   "output_dir": arguments.output_dir,‘
Explanation: Executes part of the module’s workflow.

Line 1016: ‘   "timeout": (arguments.connect_timeout, arguments.read_timeout),‘
Explanation: Executes part of the module’s workflow.

Line 1017: ‘   "retries": arguments.retries,‘
Explanation: Executes part of the module’s workflow.

Line 1018: ‘   "page_size": arguments.page_size,‘
Explanation: Executes part of the module’s workflow.

Line 1019: ‘   "patient_limit": arguments.limit,‘
Explanation: Executes part of the module’s workflow.

Line 1020: ‘   "refresh": arguments.refresh,‘
Explanation: Executes part of the module’s workflow.

Line 1021: ‘   "database_path": arguments.database,‘
Explanation: Executes part of the module’s workflow.

Line 1022: ‘   }‘
Explanation: Executes part of the module’s workflow.

Line 1023: ‘   resource_types = tuple(‘
Explanation: Assigns or computes a value used by later code.

Line 1024: ‘   resource_type‘
Explanation: Executes part of the module’s workflow.

Line 1025: ‘   for enabled, resource_type in (‘
Explanation: Controls execution flow for the surrounding operation.

Line 1026: ‘   (arguments.condition, "Condition"),‘
Explanation: Executes part of the module’s workflow.

Line 1027: ‘   (arguments.observation, "Observation"),‘
Explanation: Executes part of the module’s workflow.

Line 1028: ‘   (arguments.encounter, "Encounter"),‘
Explanation: Executes part of the module’s workflow.

Line 1029: ‘   )‘
Explanation: Executes part of the module’s workflow.

Line 1030: ‘   if enabled‘
Explanation: Controls execution flow for the surrounding operation.

Line 1031: ‘   )‘

Explanation: Executes part of the module’s workflow.

Line 1032: ‘   if arguments.all_patients:‘
Explanation: Controls execution flow for the surrounding operation.

Line 1033: ‘   if resource_types:‘
Explanation: Controls execution flow for the surrounding operation.

Line 1034: ‘   report = get_related_for_all_patients(resource_types,
**options)‘
Explanation: Assigns or computes a value used by later code.

Line 1035: ‘   else:‘
Explanation: Controls execution flow for the surrounding operation.

Line 1036: ‘   report = get_all_patients(arguments.limit, **options)‘
Explanation: Assigns or computes a value used by later code.

Line 1037: ‘   elif arguments.patient_id:‘
Explanation: Controls execution flow for the surrounding operation.

Line 1038: ‘   if resource_types:‘
Explanation: Controls execution flow for the surrounding operation.

Line 1039: ‘   report = get_related_for_patient(arguments.patient_id,
resource_types, **options)‘
Explanation: Assigns or computes a value used by later code.

Line 1040: ‘   else:‘
Explanation: Controls execution flow for the surrounding operation.

Line 1041: ‘   report = get_patient(arguments.patient_id, **options)‘
Explanation: Assigns or computes a value used by later code.

Line 1042: ‘   elif arguments.all_observations_encounters:‘
Explanation: Controls execution flow for the surrounding operation.

Line 1043: ‘   if arguments.observation or arguments.encounter:‘
Explanation: Controls execution flow for the surrounding operation.

Line 1044: ‘   parser.error("resource modifiers cannot be combined with --all-
observations-encounters")‘
Explanation: Executes part of the module’s workflow.

Line 1045: ‘   report = get_all_observations_and_encounters(**options)‘
Explanation: Assigns or computes a value used by later code.

Line 1046: ‘   elif arguments.patient_observations_encounters:‘
Explanation: Controls execution flow for the surrounding operation.

Line 1047: ‘   if resource_types:‘
Explanation: Controls execution flow for the surrounding operation.

Line 1048: ‘   report = get_related_for_patient(‘
Explanation: Assigns or computes a value used by later code.

Line 1049: ‘   arguments.patient_observations_encounters, resource_types,
**options‘
Explanation: Executes part of the module’s workflow.

Line 1050: ‘   )‘
Explanation: Executes part of the module’s workflow.

Line 1051: ‘   else:‘
Explanation: Controls execution flow for the surrounding operation.

Line 1052: ‘   report = get_observations_and_encounters_for_patient(‘
Explanation: Assigns or computes a value used by later code.

Line 1053: ‘   arguments.patient_observations_encounters, **options‘
Explanation: Executes part of the module’s workflow.

Line 1054: ‘   )‘
Explanation: Executes part of the module’s workflow.

Line 1055: ‘   else:‘
Explanation: Controls execution flow for the surrounding operation.

Line 1056: ‘   report = FHIRRetriever(**options).retrieve()‘
Explanation: Assigns or computes a value used by later code.

Line 1057: ‘   print(json.dumps(asdict(report), indent=2))‘
Explanation: Assigns or computes a value used by later code.

Line 1058: ‘   return 1 if report.failures else 0‘
Explanation: Returns the computed result to the caller.

Line 1059: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 1060: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 1061: ‘if __name__ == "__main__":‘
Explanation: Controls execution flow for the surrounding operation.

Line 1062: ‘   raise SystemExit(main())‘
Explanation: Raises an error so invalid or unavailable work is reported clearly.

generate_documentation_pdf.py

Purpose: Builds the Markdown companion and renders this line-oriented PDF.

Line 1: ‘"""Generate project documentation and a simple PDF using only the standard
library."""‘
Explanation: Executes part of the module’s workflow.

Line 2: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 3: ‘from __future__ import annotations‘
Explanation: Imports a library or project dependency used by this module.

Line 4: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 5: ‘import re‘
Explanation: Imports a library or project dependency used by this module.

Line 6: ‘import textwrap‘
Explanation: Imports a library or project dependency used by this module.

Line 7: ‘from pathlib import Path‘
Explanation: Imports a library or project dependency used by this module.

Line 8: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 9: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 10: ‘ROOT = Path(__file__).parent‘
Explanation: Assigns or computes a value used by later code.

Line 11: ‘SOURCE = ROOT / "docs/TECHNICAL_USER_DOCUMENTATION.md"‘
Explanation: Assigns or computes a value used by later code.

Line 12: ‘OUTPUT = Path("docs/FHIR_Retriever_Documentation.pdf")‘
Explanation: Assigns or computes a value used by later code.

Line 13: ‘PYTHON_FILES = sorted(ROOT.glob("*.py"))‘
Explanation: Assigns or computes a value used by later code.

Line 14: ‘PAGE_WIDTH, PAGE_HEIGHT = 595, 842‘
Explanation: Assigns or computes a value used by later code.

Line 15: ‘LEFT, TOP, LINE_HEIGHT = 42, 800, 12‘
Explanation: Assigns or computes a value used by later code.

Line 16: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 17: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 18: ‘def escape_pdf(text: str) -> str:‘
Explanation: Defines the escape_pdf callable.

Line 19: ‘   return text.replace("\\", "\\\\").replace("(", "\\(").replace(")", "\\)")‘
Explanation: Returns the computed result to the caller.

Line 20: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 21: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 22: ‘def page_stream(lines: list[str]) -> bytes:‘
Explanation: Defines the page_stream callable.

Line 23: ‘   commands = ["BT", "/F1 9 Tf", f"{LEFT} {TOP} Td"]‘
Explanation: Assigns or computes a value used by later code.

Line 24: ‘   for index, line in enumerate(lines):‘
Explanation: Controls execution flow for the surrounding operation.

Line 25: ‘   if index:‘
Explanation: Controls execution flow for the surrounding operation.

Line 26: ‘   commands.append(f"0 -{LINE_HEIGHT} Td")‘
Explanation: Executes part of the module’s workflow.

Line 27: ‘   commands.append(f"({escape_pdf(line)}) Tj")‘
Explanation: Executes part of the module’s workflow.

Line 28: ‘   commands.append("ET")‘

Explanation: Executes part of the module’s workflow.

Line 29: ‘   return "\n".join(commands).encode("latin-1", "replace")‘
Explanation: Returns the computed result to the caller.

Line 30: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 31: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 32: ‘def build_pdf(pages: list[list[str]]) -> bytes:‘
Explanation: Defines the build_pdf callable.

Line 33: ‘   objects: list[bytes] = [b"<< /Type /Catalog /Pages 2 0 R >>", b""]‘
Explanation: Assigns or computes a value used by later code.

Line 34: ‘   page_object_numbers = []‘
Explanation: Assigns or computes a value used by later code.

Line 35: ‘   font_number = 3 + len(pages) * 2‘
Explanation: Assigns or computes a value used by later code.

Line 36: ‘   for page in pages:‘
Explanation: Controls execution flow for the surrounding operation.

Line 37: ‘   stream = page_stream(page)‘
Explanation: Assigns or computes a value used by later code.

Line 38: ‘   content_number = len(objects) + 1‘
Explanation: Assigns or computes a value used by later code.

Line 39: ‘   page_number = content_number + 1‘
Explanation: Assigns or computes a value used by later code.

Line 40: ‘   objects.append(b"<< /Length %d >>\nstream\n%s\nendstream" %
(len(stream), stream))‘
Explanation: Executes part of the module’s workflow.

Line 41: ‘   objects.append(b"<< /Type /Page /Parent 2 0 R /MediaBox [0 0 595 842]
/Resources << /Font << /F1 %d 0 R >> >> /Contents %d 0 R >>" % (font_number,
content_number))‘
Explanation: Executes part of the module’s workflow.

Line 42: ‘   page_object_numbers.append(page_number)‘
Explanation: Executes part of the module’s workflow.

Line 43: ‘   objects.append(b"<< /Type /Font /Subtype /Type1 /BaseFont /Courier >>")‘
Explanation: Executes part of the module’s workflow.

Line 44: ‘   objects[1] = ("<< /Type /Pages /Kids [%s] /Count %d >>" % ("
".join(f"{number} 0 R" for number in page_object_numbers),
len(page_object_numbers))).encode()‘
Explanation: Assigns or computes a value used by later code.

Line 45: ‘   payload = bytearray(b"%PDF-1.4\n")‘
Explanation: Assigns or computes a value used by later code.

Line 46: ‘   offsets = [0]‘
Explanation: Assigns or computes a value used by later code.

Line 47: ‘   for number, object_data in enumerate(objects, start=1):‘

Explanation: Controls execution flow for the surrounding operation.

Line 48: ‘   offsets.append(len(payload))‘
Explanation: Executes part of the module’s workflow.

Line 49: ‘   payload.extend(f"{number} 0 obj\n".encode())‘
Explanation: Executes part of the module’s workflow.

Line 50: ‘   payload.extend(object_data)‘
Explanation: Executes part of the module’s workflow.

Line 51: ‘   payload.extend(b"\nendobj\n")‘
Explanation: Executes part of the module’s workflow.

Line 52: ‘   cross_reference = len(payload)‘
Explanation: Assigns or computes a value used by later code.

Line 53: ‘   payload.extend(f"xref\n0 {len(objects) + 1}\n0000000000 65535 f
\n".encode())‘
Explanation: Executes part of the module’s workflow.

Line 54: ‘   for offset in offsets[1:]:‘
Explanation: Controls execution flow for the surrounding operation.

Line 55: ‘   payload.extend(f"{offset:010d} 00000 n \n".encode())‘
Explanation: Executes part of the module’s workflow.

Line 56: ‘   payload.extend(f"trailer\n<< /Size {len(objects) + 1} /Root 1 0 R
>>\nstartxref\n{cross_reference}\n%%EOF\n".encode())‘
Explanation: Executes part of the module’s workflow.

Line 57: ‘   return bytes(payload)‘
Explanation: Returns the computed result to the caller.

Line 58: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 59: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 60: ‘def explain_line(line: str) -> str:‘
Explanation: Defines the explain_line callable.

Line 61: ‘   """Return a concise role description for one source line."""‘
Explanation: Executes part of the module’s workflow.

Line 62: ‘   stripped = line.strip()‘
Explanation: Assigns or computes a value used by later code.

Line 63: ‘   if not stripped:‘
Explanation: Controls execution flow for the surrounding operation.

Line 64: ‘   return "Blank line used to separate logical sections."‘
Explanation: Returns the computed result to the caller.

Line 65: ‘   if stripped.startswith("#"):‘
Explanation: Controls execution flow for the surrounding operation.

Line 66: ‘   return "Comment documenting the following code or an operational
decision."‘
Explanation: Returns the computed result to the caller.

Line 67: ‘   if stripped.startswith(("import ", "from ")):‘
Explanation: Controls execution flow for the surrounding operation.

Line 68: ‘   return "Imports a library or project dependency used by this module."‘
Explanation: Returns the computed result to the caller.

Line 69: ‘   if stripped.startswith("class "):‘
Explanation: Controls execution flow for the surrounding operation.

Line 70: ‘   return "Declares a class that groups related state and behavior."‘
Explanation: Returns the computed result to the caller.

Line 71: ‘   if stripped.startswith("def ") or stripped.startswith("async def "):‘
Explanation: Controls execution flow for the surrounding operation.

Line 72: ‘   name = re.match(r"(?:async)?def (\w+)", stripped)‘
Explanation: Assigns or computes a value used by later code.

Line 73: ‘   return f"Defines the {name.group(1) if name else ’function’} callable."‘
Explanation: Returns the computed result to the caller.

Line 74: ‘   if stripped.startswith("@"):‘
Explanation: Controls execution flow for the surrounding operation.

Line 75: ‘   return "Decorator that registers or configures the following definition."‘
Explanation: Returns the computed result to the caller.

Line 76: ‘   if stripped.startswith("return "):‘
Explanation: Controls execution flow for the surrounding operation.

Line 77: ‘   return "Returns the computed result to the caller."‘
Explanation: Returns the computed result to the caller.

Line 78: ‘   if stripped.startswith(("if ", "elif ", "else:", "try:", "except", "finally:", "for ", "while ", "with ")):‘
Explanation: Controls execution flow for the surrounding operation.

Line 79: ‘   return "Controls execution flow for the surrounding operation."‘
Explanation: Returns the computed result to the caller.

Line 80: ‘   if "sqlite3.connect" in stripped:‘
Explanation: Controls execution flow for the surrounding operation.

Line 81: ‘   return "Opens a SQLite connection to the configured database."‘
Explanation: Returns the computed result to the caller.

Line 82: ‘   if "execute(" in stripped or "executescript(" in stripped:‘
Explanation: Controls execution flow for the surrounding operation.

Line 83: ‘   return "Runs a SQL statement or schema script against SQLite."‘
Explanation: Returns the computed result to the caller.

Line 84: ‘   if "requests" in stripped or ".get(" in stripped:‘
Explanation: Controls execution flow for the surrounding operation.

Line 85: ‘   return "Performs or configures an HTTP request to the FHIR service."‘
Explanation: Returns the computed result to the caller.

Line 86: ‘   if "raise " in stripped:‘
Explanation: Controls execution flow for the surrounding operation.

Line 87: ‘   return "Raises an error so invalid or unavailable work is reported clearly."‘
Explanation: Returns the computed result to the caller.

Line 88: ‘   if "assert" in stripped:‘
Explanation: Controls execution flow for the surrounding operation.

Line 89: ‘   return "Test assertion verifying the expected behavior."‘
Explanation: Returns the computed result to the caller.

Line 90: ‘   if "=" in stripped and not stripped.startswith(("==", ">=")):‘
Explanation: Controls execution flow for the surrounding operation.

Line 91: ‘   return "Assigns or computes a value used by later code."‘
Explanation: Returns the computed result to the caller.

Line 92: ‘   return "Executes part of the module’s workflow."‘
Explanation: Returns the computed result to the caller.

Line 93: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 94: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 95: ‘def module_purpose(name: str) -> str:‘
Explanation: Defines the module_purpose callable.

Line 96: ‘   purposes = {‘
Explanation: Assigns or computes a value used by later code.

Line 97: ‘   "etl.py": "Docker entry point that retrieves the configured cohort and runs the default analysis.",‘
Explanation: Executes part of the module’s workflow.

Line 98: ‘   "fhir_analyse.py": "Reads normalized Observation data and calculates grouped numeric statistics.",‘
Explanation: Executes part of the module’s workflow.

Line 99: ‘   "fhir_api.py": "FastAPI service exposing health, Patient, Observation, and analysis endpoints.",‘
Explanation: Executes part of the module’s workflow.

Line 100: ‘   "fhir_retriever.py": "FHIR client, pseudonymization layer, SQLite projection, cache, CSV export, and CLI.",‘
Explanation: Executes part of the module’s workflow.

Line 101: ‘   "generate_documentation_pdf.py": "Builds the Markdown companion and renders this line-oriented PDF.",‘
Explanation: Executes part of the module’s workflow.

Line 102: ‘   "test_encounter_link_repair.py": "Checks repair of Observation links after Encounter identifiers are normalized.",‘
Explanation: Executes part of the module’s workflow.

Line 103: ‘   "test_fhir_analyse.py": "Checks statistics, grouping, and analysis CLI output.",‘
Explanation: Executes part of the module’s workflow.

Line 104: ‘   "test_fhir_api.py": "Checks API endpoints, validation, filtering, and name removal.",‘

Explanation: Executes part of the module’s workflow.

Line 105: ‘   "test_fhir_retriever.py": "Checks retrieval, pagination, caching, pseudonymization, database projection, and links.",‘
Explanation: Executes part of the module’s workflow.

Line 106: ‘   }‘
Explanation: Executes part of the module’s workflow.

Line 107: ‘   return purposes.get(name, "Python module in the FHIR retrieval project.")‘
Explanation: Returns the computed result to the caller.

Line 108: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 109: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 110: ‘def documentation() -> str:‘
Explanation: Defines the documentation callable.

Line 111: ‘   sections = [‘
Explanation: Assigns or computes a value used by later code.

Line 112: ‘   "# FHIR Retriever: Technical Documentation",‘
Explanation: Executes part of the module’s workflow.

Line 113: ‘   "",‘
Explanation: Executes part of the module’s workflow.

Line 114: ‘   "This document describes the Python implementation line by line. Each source line is shown with its line number and followed by its role in the application. Blank lines are identified because they separate logical sections; test lines describe the behavior being verified.",‘
Explanation: Executes part of the module’s workflow.

Line 115: ‘   "",‘
Explanation: Executes part of the module’s workflow.

Line 116: ‘   "## System Overview",‘
Explanation: Executes part of the module’s workflow.

Line 117: ‘   "",‘
Explanation: Executes part of the module’s workflow.

Line 118: ‘   "The project retrieves Patient, Condition, Observation, and Encounter resources from a FHIR R4 server. ’fhir_retriever.py’ pseudonymizes identifiers and dates, stores normalized fields in SQLite, exports CSV files, and maintains a restartable JSON cache. ’fhir_api.py’ exposes the stored data through FastAPI. ’fhir_analyse.py’ calculates descriptive statistics for numeric Observations. ’etl.py’ assembles retrieval and analysis for Docker. The remaining modules are focused tests, and ’generate_documentation_pdf.py’ creates this documentation.",‘
Explanation: Executes part of the module’s workflow.

Line 119: ‘   "",‘
Explanation: Executes part of the module’s workflow.

Line 120: ‘   "## Data and Execution Flow",‘
Explanation: Executes part of the module’s workflow.

Line 121: ‘   "",‘

Explanation: Executes part of the module’s workflow.

Line 122: ‘   "1. The retriever requests paginated FHIR Bundles.",‘
Explanation: Performs or configures an HTTP request to the FHIR service.

Line 123: ‘   "2. Resources are copied, identifiers are replaced with deterministic HMAC pseudonyms, and patient-linked dates are shifted.",‘
Explanation: Executes part of the module’s workflow.

Line 124: ‘   "3. ’FHIRDatabase’ stores normalized columns in ’fhir_resources.sqlite3’; it migrates older ’resource_json’ schemas when encountered.",‘
Explanation: Executes part of the module’s workflow.

Line 125: ‘   "4. CSV exports and the compressed resource cache are updated for restartability.",‘
Explanation: Executes part of the module’s workflow.

Line 126: ‘   "5. FastAPI reads the configured SQLite file directly, while analysis joins normalized Observation, Patient, and Encounter columns.",‘
Explanation: Executes part of the module’s workflow.

Line 127: ‘   "",‘
Explanation: Executes part of the module’s workflow.

Line 128: ‘   "## Configuration",‘
Explanation: Executes part of the module’s workflow.

Line 129: ‘   "",‘
Explanation: Executes part of the module’s workflow.

Line 130: ‘   "Set ’FHIR_PSEUDONYMIZATION_KEY’ before retrieval. Local output defaults to ’fhir_output/fhir_resources.sqlite3’; Docker uses ’/data/fhir_resources.sqlite3’. The older ’data/fhir.sqlite3’ file is not referenced by the application.",‘
Explanation: Executes part of the module’s workflow.

Line 131: ‘   "",‘
Explanation: Executes part of the module’s workflow.

Line 132: ‘   ]‘
Explanation: Executes part of the module’s workflow.

Line 133: ‘   for source in PYTHON_FILES:‘
Explanation: Controls execution flow for the surrounding operation.

Line 134: ‘   sections.extend([f"## {source.name}", "", f"Purpose: {module_purpose(source.name)}", ""])‘
Explanation: Executes part of the module’s workflow.

Line 135: ‘   for number, line in enumerate(source.read_text(encoding="utf-8").splitlines(), start=1):‘
Explanation: Controls execution flow for the surrounding operation.

Line 136: ‘   code = line.replace("’", "’") or "<blank>"‘
Explanation: Assigns or computes a value used by later code.

Line 137: ‘   sections.append(f"Line {number}: ’{code}’")‘
Explanation: Executes part of the module’s workflow.

Line 138: ‘   sections.append(f"Explanation: {explain_line(line)}")‘
Explanation: Executes part of the module’s workflow.

Line 139: ‘   sections.append("")‘
Explanation: Executes part of the module’s workflow.

Line 140: ‘   return "\n".join(sections)‘
Explanation: Returns the computed result to the caller.

Line 141: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 142: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 143: ‘def main() -> None:‘
Explanation: Defines the main callable.

Line 144: ‘   SOURCE.parent.mkdir(parents=True, exist_ok=True)‘
Explanation: Assigns or computes a value used by later code.

Line 145: ‘   SOURCE.write_text(documentation(), encoding="utf-8")‘
Explanation: Assigns or computes a value used by later code.

Line 146: ‘   lines = []‘
Explanation: Assigns or computes a value used by later code.

Line 147: ‘   for source_line in SOURCE.read_text(encoding="utf-8").splitlines():‘
Explanation: Controls execution flow for the surrounding operation.

Line 148: ‘   for line in textwrap.wrap(source_line or " ", width=88,
replace_whitespace=False) or [" "]:‘
Explanation: Controls execution flow for the surrounding operation.

Line 149: ‘   lines.append(line)‘
Explanation: Executes part of the module’s workflow.

Line 150: ‘   page_length = 62‘
Explanation: Assigns or computes a value used by later code.

Line 151: ‘   pages = [lines[index : index + page_length] for index in range(0,
len(lines), page_length)]‘
Explanation: Assigns or computes a value used by later code.

Line 152: ‘   output = ROOT / OUTPUT‘
Explanation: Assigns or computes a value used by later code.

Line 153: ‘   output.parent.mkdir(parents=True, exist_ok=True)‘
Explanation: Assigns or computes a value used by later code.

Line 154: ‘   output.write_bytes(build_pdf(pages))‘
Explanation: Executes part of the module’s workflow.

Line 155: ‘   print(output)‘
Explanation: Executes part of the module’s workflow.

Line 156: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 157: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 158: ‘if __name__ == "__main__":‘
Explanation: Controls execution flow for the surrounding operation.

Line 159: ‘   main()‘
Explanation: Executes part of the module’s workflow.

test_encounter_link_repair.py

Purpose: Checks repair of Observation links after Encounter identifiers are normalized.

Line 1: ‘import os‘
Explanation: Imports a library or project dependency used by this module.

Line 2: ‘import sqlite3‘
Explanation: Imports a library or project dependency used by this module.

Line 3: ‘import tempfile‘
Explanation: Imports a library or project dependency used by this module.

Line 4: ‘import unittest‘
Explanation: Imports a library or project dependency used by this module.

Line 5: ‘from pathlib import Path‘
Explanation: Imports a library or project dependency used by this module.

Line 6: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 7: ‘from fhir_retriever import FHIRRetriever‘
Explanation: Imports a library or project dependency used by this module.

Line 8: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 9: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 10: ‘class EncounterLinkRepairTests(unittest.TestCase):‘
Explanation: Declares a class that groups related state and behavior.

Line 11: ‘   def
test_repairs_observation_links_using_cached_encounter_identifier(self):‘
Explanation: Defines the
test_repairs_observation_links_using_cached_encounter_identifier callable.

Line 12: ‘   os.environ.setdefault("FHIR_PSEUDONYMIZATION_KEY", "test-only-key-not-
for-production")‘
Explanation: Executes part of the module’s workflow.

Line 13: ‘   with tempfile.TemporaryDirectory() as temporary_directory:‘
Explanation: Controls execution flow for the surrounding operation.

Line 14: ‘   output_directory = Path(temporary_directory)‘
Explanation: Assigns or computes a value used by later code.

Line 15: ‘   database = sqlite3.connect(output_directory /
"fhir_resources.sqlite3")‘
Explanation: Opens a SQLite connection to the configured database.

Line 16: ‘   database.executescript(‘
Explanation: Runs a SQL statement or schema script against SQLite.

Line 17: ‘   """‘
Explanation: Executes part of the module’s workflow.

Line 18: ‘   CREATE TABLE patients (patient_id TEXT PRIMARY KEY,
family_name TEXT, given_name TEXT,‘
Explanation: Executes part of the module’s workflow.

Line 19: ‘   gender TEXT, birth_date TEXT, date_shift_days INTEGER NOT
NULL DEFAULT 0);‘
Explanation: Executes part of the module’s workflow.

Line 20: ‘   CREATE TABLE observations (observation_id TEXT PRIMARY KEY,
patient_id TEXT, encounter_id TEXT,‘
Explanation: Executes part of the module’s workflow.

Line 21: ‘   observation_type TEXT, observation_code TEXT,
observation_subtype TEXT,‘
Explanation: Executes part of the module’s workflow.

Line 22: ‘   effective_date_time TEXT, issued TEXT, value TEXT, unit
TEXT, value_code TEXT);‘
Explanation: Executes part of the module’s workflow.

Line 23: ‘   CREATE TABLE encounters (encounter_type TEXT, encounter_id
TEXT PRIMARY KEY, start TEXT,‘
Explanation: Executes part of the module’s workflow.

Line 24: ‘   end TEXT, patient_id TEXT);‘
Explanation: Executes part of the module’s workflow.

Line 25: ‘   INSERT INTO observations VALUES (’o1’, ’p1’, ’Encounter/e1’,
NULL, NULL, NULL, NULL, NULL,‘
Explanation: Executes part of the module’s workflow.

Line 26: ‘   NULL, NULL, NULL);‘
Explanation: Executes part of the module’s workflow.

Line 27: ‘   INSERT INTO encounters VALUES (’ambulatory’, ’visit-1’, NULL,
NULL, ’p1’);‘
Explanation: Executes part of the module’s workflow.

Line 28: ‘   """‘
Explanation: Executes part of the module’s workflow.

Line 29: ‘   )‘
Explanation: Executes part of the module’s workflow.

Line 30: ‘   database.close()‘
Explanation: Executes part of the module’s workflow.

Line 31: ‘   FHIRRetriever._write_json(‘
Explanation: Executes part of the module’s workflow.

Line 32: ‘   output_directory / "resources.json.gz",‘
Explanation: Executes part of the module’s workflow.

Line 33: ‘   {‘
Explanation: Executes part of the module’s workflow.

Line 34: ‘   "Encounter/e1": {‘
Explanation: Executes part of the module’s workflow.

Line 35: ‘   "resourceType": "Encounter",‘
Explanation: Executes part of the module’s workflow.

Line 36: ‘   "id": "e1",‘
Explanation: Executes part of the module’s workflow.

Line 37: ‘   "identifier": [{"value": "visit-1"}],‘
Explanation: Executes part of the module’s workflow.

Line 38: ‘   }‘
Explanation: Executes part of the module’s workflow.

Line 39: ‘   },‘
Explanation: Executes part of the module’s workflow.

Line 40: ‘   )‘
Explanation: Executes part of the module’s workflow.

Line 41: ‘   retriever = FHIRRetriever(output_dir=output_directory)‘
Explanation: Assigns or computes a value used by later code.

Line 42: ‘   repaired = retriever.database.connection.execute(‘
Explanation: Runs a SQL statement or schema script against SQLite.

Line 43: ‘   "SELECT encounter_id FROM observations WHERE observation_id =
’o1’"‘
Explanation: Assigns or computes a value used by later code.

Line 44: ‘   ).fetchone()‘
Explanation: Executes part of the module’s workflow.

Line 45: ‘   retriever.database.close()‘
Explanation: Executes part of the module’s workflow.

Line 46: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 47: ‘   self.assertEqual(repaired, ("visit-1",))‘
Explanation: Test assertion verifying the expected behavior.

test_fhir_analyse.py

Purpose: Checks statistics, grouping, and analysis CLI output.

Line 1: ‘import sqlite3‘
Explanation: Imports a library or project dependency used by this module.

Line 2: ‘import tempfile‘
Explanation: Imports a library or project dependency used by this module.

Line 3: ‘import unittest‘
Explanation: Imports a library or project dependency used by this module.

Line 4: ‘from pathlib import Path‘
Explanation: Imports a library or project dependency used by this module.

Line 5: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 6: ‘from fhir_analyse import analyse, main‘
Explanation: Imports a library or project dependency used by this module.

Line 7: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 8: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 9: ‘class FHIRAnalyseTests(unittest.TestCase):‘
Explanation: Declares a class that groups related state and behavior.

Line 10: ‘   def test_calculates_statistics_by_sex(self):‘
Explanation: Defines the test_calculates_statistics_by_sex callable.

Line 11: ‘   with tempfile.TemporaryDirectory() as temporary_directory:‘
Explanation: Controls execution flow for the surrounding operation.

Line 12: ‘   database_path = Path(temporary_directory) / "fhir.sqlite3"‘
Explanation: Assigns or computes a value used by later code.

Line 13: ‘   database = sqlite3.connect(database_path)‘
Explanation: Opens a SQLite connection to the configured database.

Line 14: ‘   database.executescript(‘
Explanation: Runs a SQL statement or schema script against SQLite.

Line 15: ‘   """‘
Explanation: Executes part of the module’s workflow.

Line 16: ‘   CREATE TABLE patients (patient_id TEXT PRIMARY KEY, birth_date TEXT, gender TEXT);‘
Explanation: Executes part of the module’s workflow.

Line 17: ‘   CREATE TABLE encounters (encounter_id TEXT PRIMARY KEY, encounter_type TEXT);‘
Explanation: Executes part of the module’s workflow.

Line 18: ‘   CREATE TABLE observations (patient_id TEXT, encounter_id TEXT, observation_subtype TEXT,‘
Explanation: Executes part of the module’s workflow.

Line 19: ‘   observation_code TEXT, effective_date_time TEXT, value TEXT);‘
Explanation: Executes part of the module’s workflow.

Line 20: ‘   INSERT INTO patients VALUES (’PAT-1’, ’1980-01-01’, ’female’);‘
Explanation: Executes part of the module’s workflow.

Line 21: ‘   INSERT INTO encounters VALUES (’ENC-1’, ’ambulatory’);‘
Explanation: Executes part of the module’s workflow.

Line 22: ‘   INSERT INTO observations VALUES (’PAT-1’, ’ENC-1’, ’Alanine Aminotransferase’, ’1742-6’,‘
Explanation: Executes part of the module’s workflow.

Line 23: ‘   ’2024-01-01’, ’10’);‘
Explanation: Executes part of the module’s workflow.

Line 24: ‘   INSERT INTO observations VALUES (’PAT-1’, ’ENC-1’, ’Alanine Aminotransferase’, ’1742-6’,‘
Explanation: Executes part of the module’s workflow.

Line 25: ‘   ’2024-01-02’, ’20’);‘
Explanation: Executes part of the module’s workflow.

Line 26: ‘   """‘

Explanation: Executes part of the module’s workflow.

Line 27: ‘   )‘
Explanation: Executes part of the module’s workflow.

Line 28: ‘   database.close()‘
Explanation: Executes part of the module’s workflow.

Line 29: ‘   result = analyse(database_path, "Alanine Aminotransferase",
group_by="sex")‘
Explanation: Assigns or computes a value used by later code.

Line 30: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 31: ‘   self.assertEqual(result["female"], {‘
Explanation: Test assertion verifying the expected behavior.

Line 32: ‘   "count": 2,‘
Explanation: Executes part of the module’s workflow.

Line 33: ‘   "mean": 15.0,‘
Explanation: Executes part of the module’s workflow.

Line 34: ‘   "median": 15.0,‘
Explanation: Executes part of the module’s workflow.

Line 35: ‘   "standard_deviation": 7.0710678118654755,‘
Explanation: Executes part of the module’s workflow.

Line 36: ‘   "minimum": 10.0,‘
Explanation: Executes part of the module’s workflow.

Line 37: ‘   "maximum": 20.0,‘
Explanation: Executes part of the module’s workflow.

Line 38: ‘   })‘
Explanation: Executes part of the module’s workflow.

Line 39: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 40: ‘   def test_cli_writes_analysis_text_file(self):‘
Explanation: Defines the test_cli_writes_analysis_text_file callable.

Line 41: ‘   with tempfile.TemporaryDirectory() as temporary_directory:‘
Explanation: Controls execution flow for the surrounding operation.

Line 42: ‘   database_path = Path(temporary_directory) /
"fhir_resources.sqlite3"‘
Explanation: Assigns or computes a value used by later code.

Line 43: ‘   database = sqlite3.connect(database_path)‘
Explanation: Opens a SQLite connection to the configured database.

Line 44: ‘   database.executescript(‘
Explanation: Runs a SQL statement or schema script against SQLite.

Line 45: ‘   """‘
Explanation: Executes part of the module’s workflow.

Line 46: ‘   CREATE TABLE patients (patient_id TEXT PRIMARY KEY, birth_date

TEXT, gender TEXT);‘
Explanation: Executes part of the module’s workflow.

Line 47: ‘   CREATE TABLE encounters (encounter_id TEXT PRIMARY KEY, encounter_type TEXT);‘
Explanation: Executes part of the module’s workflow.

Line 48: ‘   CREATE TABLE observations (patient_id TEXT, encounter_id TEXT, observation_subtype TEXT,‘
Explanation: Executes part of the module’s workflow.

Line 49: ‘   observation_code TEXT, effective_date_time TEXT, value TEXT);‘
Explanation: Executes part of the module’s workflow.

Line 50: ‘   INSERT INTO patients VALUES (’PAT-1’, ’1980-01-01’, ’female’);‘
Explanation: Executes part of the module’s workflow.

Line 51: ‘   INSERT INTO observations VALUES (’PAT-1’, NULL, ’Test’, ’test’, ’2024-01-01’, ’10’);‘
Explanation: Executes part of the module’s workflow.

Line 52: ‘   """‘
Explanation: Executes part of the module’s workflow.

Line 53: ‘   )‘
Explanation: Executes part of the module’s workflow.

Line 54: ‘   database.close()‘
Explanation: Executes part of the module’s workflow.

Line 55: ‘   import sys‘
Explanation: Imports a library or project dependency used by this module.

Line 56: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 57: ‘   previous_arguments = sys.argv‘
Explanation: Assigns or computes a value used by later code.

Line 58: ‘   try:‘
Explanation: Controls execution flow for the surrounding operation.

Line 59: ‘   sys.argv = ["fhir_analyse", "--obs-value", "Test", "--output-dir", temporary_directory]‘
Explanation: Assigns or computes a value used by later code.

Line 60: ‘   self.assertEqual(main(), 0)‘
Explanation: Test assertion verifying the expected behavior.

Line 61: ‘   finally:‘
Explanation: Controls execution flow for the surrounding operation.

Line 62: ‘   sys.argv = previous_arguments‘
Explanation: Assigns or computes a value used by later code.

Line 63: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 64: ‘   self.assertIn(’"count": 1’, (Path(temporary_directory) / "analysis.txt").read_text())‘

Explanation: Test assertion verifying the expected behavior.

Line 65: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 66: ‘   def test_calculates_statistics_by_encounter_type_via_encounter_id(self):‘
Explanation: Defines the test_calculates_statistics_by_encounter_type_via_encounter_id callable.

Line 67: ‘   with tempfile.TemporaryDirectory() as temporary_directory:‘
Explanation: Controls execution flow for the surrounding operation.

Line 68: ‘   database_path = Path(temporary_directory) / "fhir.sqlite3"‘
Explanation: Assigns or computes a value used by later code.

Line 69: ‘   database = sqlite3.connect(database_path)‘
Explanation: Opens a SQLite connection to the configured database.

Line 70: ‘   database.executescript(‘
Explanation: Runs a SQL statement or schema script against SQLite.

Line 71: ‘   """‘
Explanation: Executes part of the module’s workflow.

Line 72: ‘   CREATE TABLE patients (patient_id TEXT PRIMARY KEY, birth_date TEXT, gender TEXT);‘
Explanation: Executes part of the module’s workflow.

Line 73: ‘   CREATE TABLE encounters (encounter_id TEXT PRIMARY KEY, encounter_type TEXT);‘
Explanation: Executes part of the module’s workflow.

Line 74: ‘   CREATE TABLE observations (patient_id TEXT, encounter_id TEXT, observation_subtype TEXT,‘
Explanation: Executes part of the module’s workflow.

Line 75: ‘   observation_code TEXT, effective_date_time TEXT, value TEXT);‘
Explanation: Executes part of the module’s workflow.

Line 76: ‘   INSERT INTO patients VALUES (’PAT-1’, ’1980-01-01’, ’female’);‘
Explanation: Executes part of the module’s workflow.

Line 77: ‘   INSERT INTO encounters VALUES (’ENC-1’, ’ambulatory’);‘
Explanation: Executes part of the module’s workflow.

Line 78: ‘   INSERT INTO observations VALUES (’PAT-1’, ’ENC-1’, ’Alanine Aminotransferase’, ’1742-6’,‘
Explanation: Executes part of the module’s workflow.

Line 79: ‘   ’2024-01-01’, ’10’);‘
Explanation: Executes part of the module’s workflow.

Line 80: ‘   INSERT INTO observations VALUES (’PAT-1’, ’ENC-1’, ’Alanine Aminotransferase’, ’1742-6’,‘
Explanation: Executes part of the module’s workflow.

Line 81: ‘   ’2024-01-02’, ’20’);‘
Explanation: Executes part of the module’s workflow.

Line 82: ‘   """‘

Explanation: Executes part of the module’s workflow.

Line 83: ‘   )‘
Explanation: Executes part of the module’s workflow.

Line 84: ‘   database.close()‘
Explanation: Executes part of the module’s workflow.

Line 85: ‘   result = analyse(database_path, "1742-6", group_by="encounter-
type")‘
Explanation: Assigns or computes a value used by later code.

Line 86: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 87: ‘   self.assertEqual(result["ambulatory"]["count"], 2)‘
Explanation: Test assertion verifying the expected behavior.

Line 88: ‘   self.assertEqual(result["ambulatory"]["mean"], 15.0)‘
Explanation: Test assertion verifying the expected behavior.

test_fhir_api.py

Purpose: Checks API endpoints, validation, filtering, and name removal.

Line 1: ‘import sqlite3‘
Explanation: Imports a library or project dependency used by this module.

Line 2: ‘import tempfile‘
Explanation: Imports a library or project dependency used by this module.

Line 3: ‘import unittest‘
Explanation: Imports a library or project dependency used by this module.

Line 4: ‘from pathlib import Path‘
Explanation: Imports a library or project dependency used by this module.

Line 5: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 6: ‘from fastapi.testclient import TestClient‘
Explanation: Imports a library or project dependency used by this module.

Line 7: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 8: ‘import fhir_api‘
Explanation: Imports a library or project dependency used by this module.

Line 9: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 10: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 11: ‘class FHIRApiTests(unittest.TestCase):‘
Explanation: Declares a class that groups related state and behavior.

Line 12: ‘   def setUp(self):‘
Explanation: Defines the setUp callable.

Line 13: ‘   self.temporary_directory = tempfile.TemporaryDirectory()‘

Explanation: Assigns or computes a value used by later code.

Line 14: ‘   database_path = Path(self.temporary_directory.name) / "fhir.sqlite3"‘
Explanation: Assigns or computes a value used by later code.

Line 15: ‘   database = sqlite3.connect(database_path)‘
Explanation: Opens a SQLite connection to the configured database.

Line 16: ‘   database.executescript(‘
Explanation: Runs a SQL statement or schema script against SQLite.

Line 17: ‘   """‘
Explanation: Executes part of the module’s workflow.

Line 18: ‘   CREATE TABLE patients (patient_id TEXT PRIMARY KEY, family_name TEXT, given_name TEXT,‘
Explanation: Executes part of the module’s workflow.

Line 19: ‘   gender TEXT, birth_date TEXT);‘
Explanation: Executes part of the module’s workflow.

Line 20: ‘   CREATE TABLE encounters (encounter_id TEXT PRIMARY KEY, encounter_type TEXT);‘
Explanation: Executes part of the module’s workflow.

Line 21: ‘   CREATE TABLE observations (observation_id TEXT PRIMARY KEY, patient_id TEXT, encounter_id TEXT,‘
Explanation: Executes part of the module’s workflow.

Line 22: ‘   observation_code TEXT, observation_subtype TEXT, effective_date_time TEXT, value TEXT);‘
Explanation: Executes part of the module’s workflow.

Line 23: ‘   INSERT INTO patients VALUES (’PAT-1’, ’Doe’, ’Jane’, ’female’, ’1980-01-01’);‘
Explanation: Executes part of the module’s workflow.

Line 24: ‘   INSERT INTO encounters VALUES (’ENC-1’, ’ambulatory’);‘
Explanation: Executes part of the module’s workflow.

Line 25: ‘   INSERT INTO observations VALUES (’OBS-1’, ’PAT-1’, ’ENC-1’, ’1742-6’,‘
Explanation: Executes part of the module’s workflow.

Line 26: ‘   ’Alanine Aminotransferase’, ’2024-01-01’, ’10’);‘
Explanation: Executes part of the module’s workflow.

Line 27: ‘   """‘
Explanation: Executes part of the module’s workflow.

Line 28: ‘   )‘
Explanation: Executes part of the module’s workflow.

Line 29: ‘   database.close()‘
Explanation: Executes part of the module’s workflow.

Line 30: ‘   self.previous_database_path = fhir_api.DATABASE_PATH‘
Explanation: Assigns or computes a value used by later code.

Line 31: ‘   fhir_api.DATABASE_PATH = database_path‘
Explanation: Assigns or computes a value used by later code.

Line 32: ‘   self.client = TestClient(fhir_api.app)‘
Explanation: Assigns or computes a value used by later code.

Line 33: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 34: ‘   def tearDown(self):‘
Explanation: Defines the tearDown callable.

Line 35: ‘   fhir_api.DATABASE_PATH = self.previous_database_path‘
Explanation: Assigns or computes a value used by later code.

Line 36: ‘   self.temporary_directory.cleanup()‘
Explanation: Executes part of the module’s workflow.

Line 37: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 38: ‘   def test_api_endpoints_and_validation(self):‘
Explanation: Defines the test_api_endpoints_and_validation callable.

Line 39: ‘   self.assertEqual(self.client.get("/health").status_code, 200)‘
Explanation: Performs or configures an HTTP request to the FHIR service.

Line 40: ‘   self.assertEqual(self.client.get("/patients?limit=1").json()["total"], 1)‘
Explanation: Performs or configures an HTTP request to the FHIR service.

Line 41: ‘   self.assertEqual(self.client.get("/patients/PAT-1").status_code, 200)‘
Explanation: Performs or configures an HTTP request to the FHIR service.

Line 42: ‘   self.assertEqual(self.client.get("/patients/PAT-404").status_code, 404)‘
Explanation: Performs or configures an HTTP request to the FHIR service.

Line 43: ‘   self.assertEqual(self.client.get("/observations?observation_code=1742-6").json()["total"], 1)‘
Explanation: Performs or configures an HTTP request to the FHIR service.

Line 44: ‘   self.assertEqual(self.client.get("/observations?limit=0").status_code, 422)‘
Explanation: Performs or configures an HTTP request to the FHIR service.

Line 45: ‘   self.assertIn("ambulatory", self.client.get("/analysis/observations?observation=1742-6&group_by=encounter-type").json())‘
Explanation: Performs or configures an HTTP request to the FHIR service.

Line 46: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 47: ‘   def test_patient_responses_do_not_include_names(self):‘
Explanation: Defines the test_patient_responses_do_not_include_names callable.

Line 48: ‘   collection_item = self.client.get("/patients?limit=1").json()["items"][0]‘
Explanation: Performs or configures an HTTP request to the FHIR service.

Line 49: ‘   single_patient = self.client.get("/patients/PAT-1").json()‘
Explanation: Performs or configures an HTTP request to the FHIR service.

Line 50: ‘   for response in (collection_item, single_patient):‘

Explanation: Controls execution flow for the surrounding operation.

Line 51: ‘   self.assertNotIn("family_name", response)‘
Explanation: Test assertion verifying the expected behavior.

Line 52: ‘   self.assertNotIn("given_name", response)‘
Explanation: Test assertion verifying the expected behavior.

Line 53: ‘   self.assertEqual(single_patient["patient_id"], "PAT-1")‘
Explanation: Test assertion verifying the expected behavior.

Line 54: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 55: ‘   def test_patients_endpoint_fetches_live_when_cache_is_short(self):‘
Explanation: Defines the test_patients_endpoint_fetches_live_when_cache_is_short
callable.

Line 56: ‘   calls = []‘
Explanation: Assigns or computes a value used by later code.

Line 57: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 58: ‘   def fake_get_all_patients(limit=None, **retriever_options):‘
Explanation: Defines the fake_get_all_patients callable.

Line 59: ‘   calls.append((limit, retriever_options))‘
Explanation: Executes part of the module’s workflow.

Line 60: ‘   from dataclasses import dataclass, field‘
Explanation: Imports a library or project dependency used by this module.

Line 61: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 62: ‘   @dataclass‘
Explanation: Decorator that registers or configures the following definition.

Line 63: ‘   class Report:‘
Explanation: Declares a class that groups related state and behavior.

Line 64: ‘   resources: dict = field(default_factory=dict)‘
Explanation: Assigns or computes a value used by later code.

Line 65: ‘   failures: list = field(default_factory=list)‘
Explanation: Assigns or computes a value used by later code.

Line 66: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 67: ‘   return Report()‘
Explanation: Returns the computed result to the caller.

Line 68: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 69: ‘   original = fhir_api.get_all_patients‘
Explanation: Assigns or computes a value used by later code.

Line 70: ‘   fhir_api.get_all_patients = fake_get_all_patients‘
Explanation: Assigns or computes a value used by later code.

Line 71: ‘   try:‘
Explanation: Controls execution flow for the surrounding operation.

Line 72: ‘   response = self.client.get("/patients?limit=10")‘
Explanation: Performs or configures an HTTP request to the FHIR service.

Line 73: ‘   finally:‘
Explanation: Controls execution flow for the surrounding operation.

Line 74: ‘   fhir_api.get_all_patients = original‘
Explanation: Assigns or computes a value used by later code.

Line 75: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 76: ‘   self.assertEqual(response.status_code, 200)‘
Explanation: Test assertion verifying the expected behavior.

Line 77: ‘   self.assertEqual(len(calls), 1)‘
Explanation: Test assertion verifying the expected behavior.

Line 78: ‘   self.assertEqual(calls[0][0], 10)‘
Explanation: Test assertion verifying the expected behavior.

test_fhir_retriever.py

Purpose: Checks retrieval, pagination, caching, pseudonymization, database projection, and links.

Line 1: ‘import json‘
Explanation: Imports a library or project dependency used by this module.

Line 2: ‘import gzip‘
Explanation: Imports a library or project dependency used by this module.

Line 3: ‘import csv‘
Explanation: Imports a library or project dependency used by this module.

Line 4: ‘import tempfile‘
Explanation: Imports a library or project dependency used by this module.

Line 5: ‘import unittest‘
Explanation: Imports a library or project dependency used by this module.

Line 6: ‘import sqlite3‘
Explanation: Imports a library or project dependency used by this module.

Line 7: ‘import os‘
Explanation: Imports a library or project dependency used by this module.

Line 8: ‘from datetime import date‘
Explanation: Imports a library or project dependency used by this module.

Line 9: ‘from pathlib import Path‘
Explanation: Imports a library or project dependency used by this module.

Line 10: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 11: ‘from fhir_retriever import (‘
Explanation: Imports a library or project dependency used by this module.

Line 12: ‘   FHIRRetriever,‘
Explanation: Executes part of the module’s workflow.

Line 13: ‘   get_observations_and_encounters_for_patient,‘
Explanation: Executes part of the module’s workflow.

Line 14: ‘   get_observations_for_patient,‘
Explanation: Executes part of the module’s workflow.

Line 15: ‘   get_encounters_for_patient,‘
Explanation: Executes part of the module’s workflow.

Line 16: ‘   get_conditions_for_patient,‘
Explanation: Executes part of the module’s workflow.

Line 17: ‘   get_all_patients,‘
Explanation: Executes part of the module’s workflow.

Line 18: ‘   get_related_for_patient,‘
Explanation: Executes part of the module’s workflow.

Line 19: ‘   get_patient,‘
Explanation: Executes part of the module’s workflow.

Line 20: ‘)‘
Explanation: Executes part of the module’s workflow.

Line 21: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 22: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 23: ‘os.environ.setdefault("FHIR_PSEUDONYMIZATION_KEY", "test-only-key-not-for-production")‘
Explanation: Executes part of the module’s workflow.

Line 24: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 25: ‘class FakeResponse:‘
Explanation: Declares a class that groups related state and behavior.

Line 26: ‘   def __init__(self, body, error=None):‘
Explanation: Defines the __init__ callable.

Line 27: ‘   self.body = body‘
Explanation: Assigns or computes a value used by later code.

Line 28: ‘   self.error = error‘
Explanation: Assigns or computes a value used by later code.

Line 29: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 30: ‘   def raise_for_status(self):‘
Explanation: Defines the raise_for_status callable.

Line 31: ‘   if self.error:‘
Explanation: Controls execution flow for the surrounding operation.

Line 32: ‘   raise self.error‘
Explanation: Raises an error so invalid or unavailable work is reported clearly.

Line 33: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 34: ‘   def json(self):‘
Explanation: Defines the json callable.

Line 35: ‘   return self.body‘
Explanation: Returns the computed result to the caller.

Line 36: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 37: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 38: ‘class FakeSession:‘
Explanation: Declares a class that groups related state and behavior.

Line 39: ‘   def __init__(self, responses):‘
Explanation: Defines the __init__ callable.

Line 40: ‘   self.responses = responses‘
Explanation: Assigns or computes a value used by later code.

Line 41: ‘   self.calls = []‘
Explanation: Assigns or computes a value used by later code.

Line 42: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 43: ‘   def get(self, url, params, timeout):‘
Explanation: Defines the get callable.

Line 44: ‘   self.calls.append((url, params, timeout))‘
Explanation: Executes part of the module’s workflow.

Line 45: ‘   response = self.responses.get((url, tuple(sorted((params or
{}).items()))))‘
Explanation: Performs or configures an HTTP request to the FHIR service.

Line 46: ‘   if response is None:‘
Explanation: Controls execution flow for the surrounding operation.

Line 47: ‘   raise AssertionError(f"Unexpected request: {url} {params}")‘
Explanation: Raises an error so invalid or unavailable work is reported clearly.

Line 48: ‘   return response‘
Explanation: Returns the computed result to the caller.

Line 49: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 50: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 51: ‘def bundle(*resources, next_url=None):‘
Explanation: Defines the bundle callable.

Line 52: ‘   result = {"resourceType": "Bundle", "entry": [{"resource": resource} for

resource in resources]}‘
Explanation: Assigns or computes a value used by later code.

Line 53: ‘   if next_url:‘
Explanation: Controls execution flow for the surrounding operation.

Line 54: ‘   result["link"] = [{"relation": "next", "url": next_url}]‘
Explanation: Assigns or computes a value used by later code.

Line 55: ‘   return result‘
Explanation: Returns the computed result to the caller.

Line 56: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 57: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 58: ‘class FHIRRetrieverTests(unittest.TestCase):‘
Explanation: Declares a class that groups related state and behavior.

Line 59: ‘   endpoint = "https://example.test/fhir"‘
Explanation: Assigns or computes a value used by later code.

Line 60: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 61: ‘   def response_map(self, condition_response=None):‘
Explanation: Defines the response_map callable.

Line 62: ‘   patient_url = f"{self.endpoint}/Patient"‘
Explanation: Assigns or computes a value used by later code.

Line 63: ‘   condition_url = f"{self.endpoint}/Condition"‘
Explanation: Assigns or computes a value used by later code.

Line 64: ‘   observation_url = f"{self.endpoint}/Observation"‘
Explanation: Assigns or computes a value used by later code.

Line 65: ‘   return {‘
Explanation: Returns the computed result to the caller.

Line 66: ‘   (patient_url, (("_count", "50"),)): FakeResponse(‘
Explanation: Executes part of the module’s workflow.

Line 67: ‘   bundle({"resourceType": "Patient", "id": "one"})‘
Explanation: Executes part of the module’s workflow.

Line 68: ‘   ),‘
Explanation: Executes part of the module’s workflow.

Line 69: ‘   (condition_url, (("_count", "50"), ("patient", "one"))):
condition_response‘
Explanation: Executes part of the module’s workflow.

Line 70: ‘   or FakeResponse(bundle({"resourceType": "Condition", "id":
"c1"})),‘
Explanation: Executes part of the module’s workflow.

Line 71: ‘   (observation_url, (("_count", "50"), ("patient", "one"))):
FakeResponse(‘
Explanation: Executes part of the module’s workflow.

Line 72: ‘   bundle({"resourceType": "Observation", "id": "o1"})‘
Explanation: Executes part of the module’s workflow.

Line 73: ‘   ),‘
Explanation: Executes part of the module’s workflow.

Line 74: ‘   }‘
Explanation: Executes part of the module’s workflow.

Line 75: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 76: ‘   def test_collects_paginated_patients_and_related_resources(self):‘
Explanation: Defines the test_collects_paginated_patients_and_related_resources callable.

Line 77: ‘   patient_url = f"{self.endpoint}/Patient"‘
Explanation: Assigns or computes a value used by later code.

Line 78: ‘   page_two = f"{patient_url}?page=2"‘
Explanation: Assigns or computes a value used by later code.

Line 79: ‘   responses = self.response_map()‘
Explanation: Assigns or computes a value used by later code.

Line 80: ‘   responses[(patient_url, (("_count", "50"),))] = FakeResponse(‘
Explanation: Assigns or computes a value used by later code.

Line 81: ‘   bundle({"resourceType": "Patient", "id": "one"}, next_url=page_two)‘
Explanation: Assigns or computes a value used by later code.

Line 82: ‘   )‘
Explanation: Executes part of the module’s workflow.

Line 83: ‘   responses[(page_two, ())] = FakeResponse(bundle({"resourceType": "Patient", "id": "two"}))‘
Explanation: Assigns or computes a value used by later code.

Line 84: ‘   responses[(f"{self.endpoint}/Condition", (("_count", "50"), ("patient", "two")))] = FakeResponse(bundle())‘
Explanation: Assigns or computes a value used by later code.

Line 85: ‘   responses[(f"{self.endpoint}/Observation", (("_count", "50"), ("patient", "two")))] = FakeResponse(bundle())‘
Explanation: Assigns or computes a value used by later code.

Line 86: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 87: ‘   with tempfile.TemporaryDirectory() as temporary_directory:‘
Explanation: Controls execution flow for the surrounding operation.

Line 88: ‘   report = FHIRRetriever(self.endpoint, temporary_directory, session=FakeSession(responses)).retrieve()‘
Explanation: Assigns or computes a value used by later code.

Line 89: ‘   with gzip.open(Path(temporary_directory) / "resources.json.gz", "rt") as file:‘
Explanation: Controls execution flow for the surrounding operation.

Line 90: ‘   saved = json.load(file)‘
Explanation: Assigns or computes a value used by later code.

Line 91: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 92: ‘   self.assertEqual(‘
Explanation: Test assertion verifying the expected behavior.

Line 93: ‘   report.resources,‘
Explanation: Executes part of the module’s workflow.

Line 94: ‘   {"Patient": 2, "Condition": 1, "Observation": 1, "Encounter": 0},‘
Explanation: Executes part of the module’s workflow.

Line 95: ‘   )‘
Explanation: Executes part of the module’s workflow.

Line 96: ‘   self.assertFalse(report.failures)‘
Explanation: Test assertion verifying the expected behavior.

Line 97: ‘   self.assertEqual(set(saved), {"Patient/one", "Patient/two",
"Condition/c1", "Observation/o1"})‘
Explanation: Test assertion verifying the expected behavior.

Line 98: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 99: ‘   def
test_partial_failure_is_reported_and_rerun_retries_only_missing_query(self):‘
Explanation: Defines the
test_partial_failure_is_reported_and_rerun_retries_only_missing_query callable.

Line 100: ‘   from requests import ConnectionError‘
Explanation: Imports a library or project dependency used by this module.

Line 101: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 102: ‘   with tempfile.TemporaryDirectory() as temporary_directory:‘
Explanation: Controls execution flow for the surrounding operation.

Line 103: ‘   failed = FHIRRetriever(‘
Explanation: Assigns or computes a value used by later code.

Line 104: ‘   self.endpoint,‘
Explanation: Executes part of the module’s workflow.

Line 105: ‘   temporary_directory,‘
Explanation: Executes part of the module’s workflow.

Line 106: ‘   session=FakeSession(self.response_map(FakeResponse({},
ConnectionError("offline")))),‘
Explanation: Assigns or computes a value used by later code.

Line 107: ‘   ).retrieve()‘
Explanation: Executes part of the module’s workflow.

Line 108: ‘   rerun_session = FakeSession(self.response_map())‘
Explanation: Assigns or computes a value used by later code.

Line 109: ‘   rerun = FHIRRetriever(self.endpoint, temporary_directory,

session=rerun_session).retrieve()‘
Explanation: Assigns or computes a value used by later code.

Line 110: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 111: ‘   self.assertEqual(len(failed.failures), 1)‘
Explanation: Test assertion verifying the expected behavior.

Line 112: ‘   self.assertEqual(failed.failures[0].query, "Condition?patient=one")‘
Explanation: Test assertion verifying the expected behavior.

Line 113: ‘   self.assertFalse(rerun.failures)‘
Explanation: Test assertion verifying the expected behavior.

Line 114: ‘   condition_calls = [call for call in rerun_session.calls if
call[0].endswith("/Condition")]‘
Explanation: Assigns or computes a value used by later code.

Line 115: ‘   observation_calls = [call for call in rerun_session.calls if
call[0].endswith("/Observation")]‘
Explanation: Assigns or computes a value used by later code.

Line 116: ‘   self.assertEqual(len(condition_calls), 1)‘
Explanation: Test assertion verifying the expected behavior.

Line 117: ‘   self.assertEqual(observation_calls, [])‘
Explanation: Test assertion verifying the expected behavior.

Line 118: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 119: ‘   def
test_failed_later_page_keeps_cached_resources_and_rerun_completes_query(self):‘
Explanation: Defines the
test_failed_later_page_keeps_cached_resources_and_rerun_completes_query callable.

Line 120: ‘   from requests import Timeout‘
Explanation: Imports a library or project dependency used by this module.

Line 121: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 122: ‘   observation_url = f"{self.endpoint}/Observation"‘
Explanation: Assigns or computes a value used by later code.

Line 123: ‘   observation_page_two = f"{observation_url}?page=2"‘
Explanation: Assigns or computes a value used by later code.

Line 124: ‘   failed_responses = self.response_map()‘
Explanation: Assigns or computes a value used by later code.

Line 125: ‘   failed_responses[(observation_url, (("_count", "50"), ("patient",
"one")))] = FakeResponse(‘
Explanation: Assigns or computes a value used by later code.

Line 126: ‘   bundle(‘
Explanation: Executes part of the module’s workflow.

Line 127: ‘   {"resourceType": "Observation", "id": "o1", "subject":
{"reference": "Patient/one"}},‘
Explanation: Executes part of the module’s workflow.

Line 128: ‘   next_url=observation_page_two,‘
Explanation: Assigns or computes a value used by later code.

Line 129: ‘   )‘
Explanation: Executes part of the module’s workflow.

Line 130: ‘   )‘
Explanation: Executes part of the module’s workflow.

Line 131: ‘   failed_responses[(observation_page_two, ())] = FakeResponse({},
Timeout("read timed out"))‘
Explanation: Assigns or computes a value used by later code.

Line 132: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 133: ‘   with tempfile.TemporaryDirectory() as temporary_directory:‘
Explanation: Controls execution flow for the surrounding operation.

Line 134: ‘   failed = FHIRRetriever(‘
Explanation: Assigns or computes a value used by later code.

Line 135: ‘   self.endpoint, temporary_directory,
session=FakeSession(failed_responses)‘
Explanation: Assigns or computes a value used by later code.

Line 136: ‘   ).retrieve()‘
Explanation: Executes part of the module’s workflow.

Line 137: ‘   with gzip.open(Path(temporary_directory) / "resources.json.gz",
"rt") as file:‘
Explanation: Controls execution flow for the surrounding operation.

Line 138: ‘   partial_cache = json.load(file)‘
Explanation: Assigns or computes a value used by later code.

Line 139: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 140: ‘   rerun_responses = self.response_map()‘
Explanation: Assigns or computes a value used by later code.

Line 141: ‘   rerun_responses[(observation_url, (("_count", "50"), ("patient",
"one")))] = FakeResponse(‘
Explanation: Assigns or computes a value used by later code.

Line 142: ‘   bundle(‘
Explanation: Executes part of the module’s workflow.

Line 143: ‘   {"resourceType": "Observation", "id": "o1", "subject":
{"reference": "Patient/one"}},‘
Explanation: Executes part of the module’s workflow.

Line 144: ‘   next_url=observation_page_two,‘
Explanation: Assigns or computes a value used by later code.

Line 145: ‘   )‘
Explanation: Executes part of the module’s workflow.

Line 146: ‘   )‘
Explanation: Executes part of the module’s workflow.

Line 147: ‘   rerun_responses[(observation_page_two, ())] = FakeResponse(‘
Explanation: Assigns or computes a value used by later code.

Line 148: ‘   bundle({"resourceType": "Observation", "id": "o2", "subject": {"reference": "Patient/one"}})‘
Explanation: Executes part of the module’s workflow.

Line 149: ‘   )‘
Explanation: Executes part of the module’s workflow.

Line 150: ‘   rerun = FHIRRetriever(‘
Explanation: Assigns or computes a value used by later code.

Line 151: ‘   self.endpoint, temporary_directory, session=FakeSession(rerun_responses)‘
Explanation: Assigns or computes a value used by later code.

Line 152: ‘   ).retrieve()‘
Explanation: Executes part of the module’s workflow.

Line 153: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 154: ‘   self.assertEqual(len(failed.failures), 1)‘
Explanation: Test assertion verifying the expected behavior.

Line 155: ‘   self.assertEqual(failed.failures[0].query, "Observation?patient=one")‘
Explanation: Test assertion verifying the expected behavior.

Line 156: ‘   self.assertIn("Observation/o1", partial_cache)‘
Explanation: Test assertion verifying the expected behavior.

Line 157: ‘   self.assertFalse(rerun.failures)‘
Explanation: Test assertion verifying the expected behavior.

Line 158: ‘   self.assertEqual(rerun.resources["Observation"], 1)‘
Explanation: Test assertion verifying the expected behavior.

Line 159: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 160: ‘   def test_completed_cache_reruns_without_network_requests(self):‘
Explanation: Defines the test_completed_cache_reruns_without_network_requests callable.

Line 161: ‘   with tempfile.TemporaryDirectory() as temporary_directory:‘
Explanation: Controls execution flow for the surrounding operation.

Line 162: ‘   FHIRRetriever(‘
Explanation: Executes part of the module’s workflow.

Line 163: ‘   self.endpoint, temporary_directory, session=FakeSession(self.response_map())‘
Explanation: Assigns or computes a value used by later code.

Line 164: ‘   ).retrieve()‘
Explanation: Executes part of the module’s workflow.

Line 165: ‘   cached_session = FakeSession({})‘
Explanation: Assigns or computes a value used by later code.

Line 166: ‘   report = FHIRRetriever(‘
Explanation: Assigns or computes a value used by later code.

Line 167: ‘   self.endpoint, temporary_directory, session=cached_session‘
Explanation: Assigns or computes a value used by later code.

Line 168: ‘   ).retrieve()‘
Explanation: Executes part of the module’s workflow.

Line 169: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 170: ‘   self.assertFalse(report.failures)‘
Explanation: Test assertion verifying the expected behavior.

Line 171: ‘   self.assertEqual(‘
Explanation: Test assertion verifying the expected behavior.

Line 172: ‘   report.resources,‘
Explanation: Executes part of the module’s workflow.

Line 173: ‘   {"Patient": 0, "Condition": 0, "Observation": 0, "Encounter": 0},‘
Explanation: Executes part of the module’s workflow.

Line 174: ‘   )‘
Explanation: Executes part of the module’s workflow.

Line 175: ‘   self.assertEqual(cached_session.calls, [])‘
Explanation: Test assertion verifying the expected behavior.

Line 176: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 177: ‘   def test_refresh_updates_cached_resources(self):‘
Explanation: Defines the test_refresh_updates_cached_resources callable.

Line 178: ‘   with tempfile.TemporaryDirectory() as temporary_directory:‘
Explanation: Controls execution flow for the surrounding operation.

Line 179: ‘   FHIRRetriever(‘
Explanation: Executes part of the module’s workflow.

Line 180: ‘   self.endpoint, temporary_directory, session=FakeSession(self.response_map())‘
Explanation: Assigns or computes a value used by later code.

Line 181: ‘   ).retrieve()‘
Explanation: Executes part of the module’s workflow.

Line 182: ‘   refreshed_responses = self.response_map()‘
Explanation: Assigns or computes a value used by later code.

Line 183: ‘   refreshed_responses[(f"{self.endpoint}/Patient", (("_count", "50"),))] = FakeResponse(‘
Explanation: Assigns or computes a value used by later code.

Line 184: ‘   bundle({"resourceType": "Patient", "id": "one", "active": False})‘
Explanation: Executes part of the module’s workflow.

Line 185: ‘   )‘

Explanation: Executes part of the module’s workflow.

Line 186: ‘   refreshed_session = FakeSession(refreshed_responses)‘
Explanation: Assigns or computes a value used by later code.

Line 187: ‘   FHIRRetriever(‘
Explanation: Executes part of the module’s workflow.

Line 188: ‘   self.endpoint,‘
Explanation: Executes part of the module’s workflow.

Line 189: ‘   temporary_directory,‘
Explanation: Executes part of the module’s workflow.

Line 190: ‘   refresh=True,‘
Explanation: Assigns or computes a value used by later code.

Line 191: ‘   session=refreshed_session,‘
Explanation: Assigns or computes a value used by later code.

Line 192: ‘   ).retrieve()‘
Explanation: Executes part of the module’s workflow.

Line 193: ‘   with gzip.open(Path(temporary_directory) / "resources.json.gz",
"rt") as file:‘
Explanation: Controls execution flow for the surrounding operation.

Line 194: ‘   saved = json.load(file)‘
Explanation: Assigns or computes a value used by later code.

Line 195: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 196: ‘   self.assertEqual(len(refreshed_session.calls), 3)‘
Explanation: Test assertion verifying the expected behavior.

Line 197: ‘   self.assertFalse(saved["Patient/one"]["active"])‘
Explanation: Test assertion verifying the expected behavior.

Line 198: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 199: ‘   def
test_database_links_conditions_and_observations_to_patient_and_encounter(self):‘
Explanation: Defines the
test_database_links_conditions_and_observations_to_patient_and_encounter callable.

Line 200: ‘   responses = self.response_map()‘
Explanation: Assigns or computes a value used by later code.

Line 201: ‘   responses[(f"{self.endpoint}/Condition", (("_count", "50"),
("patient", "one")))] = FakeResponse(‘
Explanation: Assigns or computes a value used by later code.

Line 202: ‘   bundle(‘
Explanation: Executes part of the module’s workflow.

Line 203: ‘   {‘
Explanation: Executes part of the module’s workflow.

Line 204: ‘   "resourceType": "Condition",‘
Explanation: Executes part of the module’s workflow.

Line 205: ‘   "id": "c1",‘
Explanation: Executes part of the module’s workflow.

Line 206: ‘   "subject": {"reference": "Patient/one"},‘
Explanation: Executes part of the module’s workflow.

Line 207: ‘   "encounter": {"reference": "Encounter/e1"},‘
Explanation: Executes part of the module’s workflow.

Line 208: ‘   }‘
Explanation: Executes part of the module’s workflow.

Line 209: ‘   )‘
Explanation: Executes part of the module’s workflow.

Line 210: ‘   )‘
Explanation: Executes part of the module’s workflow.

Line 211: ‘   responses[(f"{self.endpoint}/Observation", (("_count", "50"),
("patient", "one")))] = FakeResponse(‘
Explanation: Assigns or computes a value used by later code.

Line 212: ‘   bundle(‘
Explanation: Executes part of the module’s workflow.

Line 213: ‘   {‘
Explanation: Executes part of the module’s workflow.

Line 214: ‘   "resourceType": "Observation",‘
Explanation: Executes part of the module’s workflow.

Line 215: ‘   "id": "o1",‘
Explanation: Executes part of the module’s workflow.

Line 216: ‘   "subject": {"reference":
"https://example.test/fhir/Patient/one"},‘
Explanation: Executes part of the module’s workflow.

Line 217: ‘   "encounter": {"reference": "Encounter/e1"},‘
Explanation: Executes part of the module’s workflow.

Line 218: ‘   }‘
Explanation: Executes part of the module’s workflow.

Line 219: ‘   )‘
Explanation: Executes part of the module’s workflow.

Line 220: ‘   )‘
Explanation: Executes part of the module’s workflow.

Line 221: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 222: ‘   with tempfile.TemporaryDirectory() as temporary_directory:‘
Explanation: Controls execution flow for the surrounding operation.

Line 223: ‘   FHIRRetriever(‘
Explanation: Executes part of the module’s workflow.

Line 224: ‘   self.endpoint, temporary_directory,
session=FakeSession(responses)‘

Explanation: Assigns or computes a value used by later code.

Line 225: ‘   ).retrieve()‘
Explanation: Executes part of the module’s workflow.

Line 226: ‘   database = sqlite3.connect(Path(temporary_directory) / "fhir_resources.sqlite3")‘
Explanation: Opens a SQLite connection to the configured database.

Line 227: ‘   condition = database.execute(‘
Explanation: Runs a SQL statement or schema script against SQLite.

Line 228: ‘   "SELECT patient_id, encounter_id FROM conditions WHERE condition_id = ’c1’"‘
Explanation: Assigns or computes a value used by later code.

Line 229: ‘   ).fetchone()‘
Explanation: Executes part of the module’s workflow.

Line 230: ‘   observation = database.execute(‘
Explanation: Runs a SQL statement or schema script against SQLite.

Line 231: ‘   "SELECT patient_id, encounter_id FROM observations WHERE observation_id = ’o1’"‘
Explanation: Assigns or computes a value used by later code.

Line 232: ‘   ).fetchone()‘
Explanation: Executes part of the module’s workflow.

Line 233: ‘   database.close()‘
Explanation: Executes part of the module’s workflow.

Line 234: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 235: ‘   self.assertEqual(condition, ("one", "e1"))‘
Explanation: Test assertion verifying the expected behavior.

Line 236: ‘   self.assertEqual(observation, ("one", "e1"))‘
Explanation: Test assertion verifying the expected behavior.

Line 237: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 238: ‘   def test_patient_id_limits_related_searches_to_the_selected_patient(self):‘
Explanation: Defines the test_patient_id_limits_related_searches_to_the_selected_patient callable.

Line 239: ‘   patient_url = f"{self.endpoint}/Patient?_id=one"‘
Explanation: Assigns or computes a value used by later code.

Line 240: ‘   responses = {‘
Explanation: Assigns or computes a value used by later code.

Line 241: ‘   (patient_url, (("_count", "50"), ("_id", "one"))): FakeResponse(‘
Explanation: Executes part of the module’s workflow.

Line 242: ‘   bundle({"resourceType": "Patient", "id": "one"})‘
Explanation: Executes part of the module’s workflow.

Line 243: ‘   ),‘

Explanation: Executes part of the module’s workflow.

Line 244: ‘   (f"{self.endpoint}/Condition", (("_count", "50"), ("patient", "one"))): FakeResponse(bundle()),‘
Explanation: Executes part of the module’s workflow.

Line 245: ‘   (f"{self.endpoint}/Observation", (("_count", "50"), ("patient", "one"))): FakeResponse(bundle()),‘
Explanation: Executes part of the module’s workflow.

Line 246: ‘   }‘
Explanation: Executes part of the module’s workflow.

Line 247: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 248: ‘   with tempfile.TemporaryDirectory() as temporary_directory:‘
Explanation: Controls execution flow for the surrounding operation.

Line 249: ‘   session = FakeSession(responses)‘
Explanation: Assigns or computes a value used by later code.

Line 250: ‘   report = FHIRRetriever(‘
Explanation: Assigns or computes a value used by later code.

Line 251: ‘   self.endpoint,‘
Explanation: Executes part of the module’s workflow.

Line 252: ‘   temporary_directory,‘
Explanation: Executes part of the module’s workflow.

Line 253: ‘   patient_id="one",‘
Explanation: Assigns or computes a value used by later code.

Line 254: ‘   session=session,‘
Explanation: Assigns or computes a value used by later code.

Line 255: ‘   ).retrieve()‘
Explanation: Executes part of the module’s workflow.

Line 256: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 257: ‘   self.assertFalse(report.failures)‘
Explanation: Test assertion verifying the expected behavior.

Line 258: ‘   self.assertEqual(len(session.calls), 3)‘
Explanation: Test assertion verifying the expected behavior.

Line 259: ‘   self.assertTrue(all(call[1].get("patient", "one") == "one" for call in session.calls))‘
Explanation: Performs or configures an HTTP request to the FHIR service.

Line 260: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 261: ‘   def test_all_patients_limit_stops_after_requested_total(self):‘
Explanation: Defines the test_all_patients_limit_stops_after_requested_total callable.

Line 262: ‘   patient_url = f"{self.endpoint}/Patient"‘
Explanation: Assigns or computes a value used by later code.

Line 263: ‘   page_two = f"{patient_url}?page=2"‘
Explanation: Assigns or computes a value used by later code.

Line 264: ‘   responses = {‘
Explanation: Assigns or computes a value used by later code.

Line 265: ‘   (patient_url, (("_count", "2"),)): FakeResponse(‘
Explanation: Executes part of the module’s workflow.

Line 266: ‘   bundle(‘
Explanation: Executes part of the module’s workflow.

Line 267: ‘   {"resourceType": "Patient", "id": "one"},‘
Explanation: Executes part of the module’s workflow.

Line 268: ‘   {"resourceType": "Patient", "id": "two"},‘
Explanation: Executes part of the module’s workflow.

Line 269: ‘   next_url=page_two,‘
Explanation: Assigns or computes a value used by later code.

Line 270: ‘   )‘
Explanation: Executes part of the module’s workflow.

Line 271: ‘   ),‘
Explanation: Executes part of the module’s workflow.

Line 272: ‘   (page_two, ()): FakeResponse(bundle({"resourceType": "Patient", "id": "three"})),‘
Explanation: Executes part of the module’s workflow.

Line 273: ‘   }‘
Explanation: Executes part of the module’s workflow.

Line 274: ‘   with tempfile.TemporaryDirectory() as temporary_directory:‘
Explanation: Controls execution flow for the surrounding operation.

Line 275: ‘   session = FakeSession(responses)‘
Explanation: Assigns or computes a value used by later code.

Line 276: ‘   report = get_all_patients(‘
Explanation: Assigns or computes a value used by later code.

Line 277: ‘   2, endpoint=self.endpoint, output_dir=temporary_directory, session=session‘
Explanation: Assigns or computes a value used by later code.

Line 278: ‘   )‘
Explanation: Executes part of the module’s workflow.

Line 279: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 280: ‘   self.assertFalse(report.failures)‘
Explanation: Test assertion verifying the expected behavior.

Line 281: ‘   self.assertEqual(report.resources["Patient"], 2)‘
Explanation: Test assertion verifying the expected behavior.

Line 282: ‘   self.assertEqual(len(session.calls), 1)‘
Explanation: Test assertion verifying the expected behavior.

Line 283: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 284: ‘   def
test_patient_checkpoint_without_cached_resource_refetches_patient(self):‘
Explanation: Defines the
test_patient_checkpoint_without_cached_resource_refetches_patient callable.

Line 285: ‘   patient_url = f"{self.endpoint}/Patient?_id=one"‘
Explanation: Assigns or computes a value used by later code.

Line 286: ‘   responses = {‘
Explanation: Assigns or computes a value used by later code.

Line 287: ‘   (patient_url, (("_count", "50"), ("_id", "one"))): FakeResponse(‘
Explanation: Executes part of the module’s workflow.

Line 288: ‘   bundle({"resourceType": "Patient", "id": "one"})‘
Explanation: Executes part of the module’s workflow.

Line 289: ‘   )‘
Explanation: Executes part of the module’s workflow.

Line 290: ‘   }‘
Explanation: Executes part of the module’s workflow.

Line 291: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 292: ‘   with tempfile.TemporaryDirectory() as temporary_directory:‘
Explanation: Controls execution flow for the surrounding operation.

Line 293: ‘   checkpoint = Path(temporary_directory) / "checkpoint.json"‘
Explanation: Assigns or computes a value used by later code.

Line 294: ‘   checkpoint.write_text(‘
Explanation: Executes part of the module’s workflow.

Line 295: ‘   json.dumps({"endpoint": f"{self.endpoint}/", "completed":
["Patient?_id=one"]})‘
Explanation: Assigns or computes a value used by later code.

Line 296: ‘   )‘
Explanation: Executes part of the module’s workflow.

Line 297: ‘   session = FakeSession(responses)‘
Explanation: Assigns or computes a value used by later code.

Line 298: ‘   retriever = FHIRRetriever(‘
Explanation: Assigns or computes a value used by later code.

Line 299: ‘   self.endpoint, temporary_directory, patient_id="one",
session=session‘
Explanation: Assigns or computes a value used by later code.

Line 300: ‘   )‘
Explanation: Executes part of the module’s workflow.

Line 301: ‘   report = retriever.get_patient("one")‘
Explanation: Assigns or computes a value used by later code.

Line 302: ‘   database = sqlite3.connect(Path(temporary_directory) /

"fhir_resources.sqlite3")‘
Explanation: Opens a SQLite connection to the configured database.

Line 303: ‘   saved_patient = database.execute(‘
Explanation: Runs a SQL statement or schema script against SQLite.

Line 304: ‘   "SELECT patient_id FROM patients WHERE patient_id = ’one’"‘
Explanation: Assigns or computes a value used by later code.

Line 305: ‘   ).fetchone()‘
Explanation: Executes part of the module’s workflow.

Line 306: ‘   database.close()‘
Explanation: Executes part of the module’s workflow.

Line 307: ‘   retriever.database.close()‘
Explanation: Executes part of the module’s workflow.

Line 308: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 309: ‘   self.assertFalse(report.failures)‘
Explanation: Test assertion verifying the expected behavior.

Line 310: ‘   self.assertEqual(len(session.calls), 1)‘
Explanation: Test assertion verifying the expected behavior.

Line 311: ‘   self.assertEqual(saved_patient, ("one",))‘
Explanation: Test assertion verifying the expected behavior.

Line 312: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 313: ‘   def test_patient_database_uses_structured_demographic_columns(self):‘
Explanation: Defines the test_patient_database_uses_structured_demographic_columns
callable.

Line 314: ‘   responses = self.response_map()‘
Explanation: Assigns or computes a value used by later code.

Line 315: ‘   responses[(f"{self.endpoint}/Patient", (("_count", "50"),))] =
FakeResponse(‘
Explanation: Assigns or computes a value used by later code.

Line 316: ‘   bundle(‘
Explanation: Executes part of the module’s workflow.

Line 317: ‘   {‘
Explanation: Executes part of the module’s workflow.

Line 318: ‘   "resourceType": "Patient",‘
Explanation: Executes part of the module’s workflow.

Line 319: ‘   "id": "one",‘
Explanation: Executes part of the module’s workflow.

Line 320: ‘   "name": [{"family": "Doe", "given": ["Jane", "Marie"]}],‘
Explanation: Executes part of the module’s workflow.

Line 321: ‘   "gender": "female",‘
Explanation: Executes part of the module’s workflow.

Line 322: ‘   "birthDate": "1980-01-02",‘
Explanation: Executes part of the module’s workflow.

Line 323: ‘   }‘
Explanation: Executes part of the module’s workflow.

Line 324: ‘   )‘
Explanation: Executes part of the module’s workflow.

Line 325: ‘   )‘
Explanation: Executes part of the module’s workflow.

Line 326: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 327: ‘   with tempfile.TemporaryDirectory() as temporary_directory:‘
Explanation: Controls execution flow for the surrounding operation.

Line 328: ‘   retriever = FHIRRetriever(‘
Explanation: Assigns or computes a value used by later code.

Line 329: ‘   self.endpoint, temporary_directory,
session=FakeSession(responses)‘
Explanation: Assigns or computes a value used by later code.

Line 330: ‘   )‘
Explanation: Executes part of the module’s workflow.

Line 331: ‘   retriever.get_all_patients()‘
Explanation: Executes part of the module’s workflow.

Line 332: ‘   columns = [column[1] for column in
retriever.database.connection.execute("PRAGMA table_info(patients)")]‘
Explanation: Runs a SQL statement or schema script against SQLite.

Line 333: ‘   patient = retriever.database.connection.execute(‘
Explanation: Runs a SQL statement or schema script against SQLite.

Line 334: ‘   "SELECT patient_id, family_name, given_name, gender,
birth_date FROM patients"‘
Explanation: Executes part of the module’s workflow.

Line 335: ‘   ).fetchone()‘
Explanation: Executes part of the module’s workflow.

Line 336: ‘   retriever.database.close()‘
Explanation: Executes part of the module’s workflow.

Line 337: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 338: ‘   self.assertEqual(‘
Explanation: Test assertion verifying the expected behavior.

Line 339: ‘   columns, ["patient_id", "family_name", "given_name", "gender",
"birth_date", "date_shift_days"]‘
Explanation: Executes part of the module’s workflow.

Line 340: ‘   )‘
Explanation: Executes part of the module’s workflow.

Line 341: ‘   self.assertTrue(patient[0].startswith("PAT-"))‘

Explanation: Test assertion verifying the expected behavior.

Line 342: ‘   self.assertEqual(patient[1:4], ("Doe", "Jane Marie", "female"))‘
Explanation: Test assertion verifying the expected behavior.

Line 343: ‘   date.fromisoformat(patient[4]) # birth_date is shifted but must remain a valid ISO date‘
Explanation: Executes part of the module’s workflow.

Line 344: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 345: ‘   def test_observation_database_uses_requested_fhir_columns(self):‘
Explanation: Defines the test_observation_database_uses_requested_fhir_columns callable.

Line 346: ‘   responses = self.response_map()‘
Explanation: Assigns or computes a value used by later code.

Line 347: ‘   observation_url = f"{self.endpoint}/Observation"‘
Explanation: Assigns or computes a value used by later code.

Line 348: ‘   responses[(observation_url, (("_count", "50"), ("patient", "one")))] = FakeResponse(‘
Explanation: Assigns or computes a value used by later code.

Line 349: ‘   bundle(‘
Explanation: Executes part of the module’s workflow.

Line 350: ‘   {‘
Explanation: Executes part of the module’s workflow.

Line 351: ‘   "resourceType": "Observation",‘
Explanation: Executes part of the module’s workflow.

Line 352: ‘   "id": "o1",‘
Explanation: Executes part of the module’s workflow.

Line 353: ‘   "subject": {"reference": "Patient/one"},‘
Explanation: Executes part of the module’s workflow.

Line 354: ‘   "encounter": {"reference": "Encounter/e1"},‘
Explanation: Executes part of the module’s workflow.

Line 355: ‘   "category": [{"coding": [{"code": "vital-signs"}]}],‘
Explanation: Executes part of the module’s workflow.

Line 356: ‘   "code": {"coding": [{"code": "8480-6", "display": "Systolic blood pressure"}]},‘
Explanation: Executes part of the module’s workflow.

Line 357: ‘   "effectiveDateTime": "2024-01-15T09:30:00Z",‘
Explanation: Executes part of the module’s workflow.

Line 358: ‘   "issued": "2024-01-15T09:35:00Z",‘
Explanation: Executes part of the module’s workflow.

Line 359: ‘   "valueQuantity": {"value": 120, "unit": "mmHg", "code": "mm[Hg]"},‘
Explanation: Executes part of the module’s workflow.

Line 360: ‘   }‘
Explanation: Executes part of the module’s workflow.

Line 361: ‘   )‘
Explanation: Executes part of the module’s workflow.

Line 362: ‘   )‘
Explanation: Executes part of the module’s workflow.

Line 363: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 364: ‘   with tempfile.TemporaryDirectory() as temporary_directory:‘
Explanation: Controls execution flow for the surrounding operation.

Line 365: ‘   FHIRRetriever(self.endpoint, temporary_directory, session=FakeSession(responses)).retrieve()‘
Explanation: Assigns or computes a value used by later code.

Line 366: ‘   database = sqlite3.connect(Path(temporary_directory) / "fhir_resources.sqlite3")‘
Explanation: Opens a SQLite connection to the configured database.

Line 367: ‘   columns = [column[1] for column in database.execute("PRAGMA table_info(observations)")]‘
Explanation: Runs a SQL statement or schema script against SQLite.

Line 368: ‘   observation = database.execute(‘
Explanation: Runs a SQL statement or schema script against SQLite.

Line 369: ‘   "SELECT observation_type, observation_code, observation_subtype, encounter_id, "‘
Explanation: Executes part of the module’s workflow.

Line 370: ‘   "effective_date_time, issued, value, unit, value_code FROM observations"‘
Explanation: Executes part of the module’s workflow.

Line 371: ‘   ).fetchone()‘
Explanation: Executes part of the module’s workflow.

Line 372: ‘   database.close()‘
Explanation: Executes part of the module’s workflow.

Line 373: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 374: ‘   self.assertNotIn("resource_json", columns)‘
Explanation: Test assertion verifying the expected behavior.

Line 375: ‘   self.assertEqual(‘
Explanation: Test assertion verifying the expected behavior.

Line 376: ‘   observation,‘
Explanation: Executes part of the module’s workflow.

Line 377: ‘   ("vital-signs", "8480-6", "Systolic blood pressure", "e1", "2024-01-15T09:30:00Z", "2024-01-15T09:35:00Z", "120", "mmHg", "mm[Hg]"),‘
Explanation: Executes part of the module’s workflow.

Line 378: ‘   )‘
Explanation: Executes part of the module’s workflow.

Line 379: ‘<blank>‘

Explanation: Blank line used to separate logical sections.

Line 380: ‘   def
test_patient_observation_and_encounter_function_populates_linked_tables(self):‘
Explanation: Defines the
test_patient_observation_and_encounter_function_populates_linked_tables callable.

Line 381: ‘   patient_url = f"{self.endpoint}/Patient?_id=one"‘
Explanation: Assigns or computes a value used by later code.

Line 382: ‘   responses = {‘
Explanation: Assigns or computes a value used by later code.

Line 383: ‘   (patient_url, (("_count", "50"), ("_id", "one"))): FakeResponse(‘
Explanation: Executes part of the module’s workflow.

Line 384: ‘   bundle({"resourceType": "Patient", "id": "one"})‘
Explanation: Executes part of the module’s workflow.

Line 385: ‘   ),‘
Explanation: Executes part of the module’s workflow.

Line 386: ‘   (f"{self.endpoint}/Observation", (("_count", "50"), ("patient",
"one"))): FakeResponse(‘
Explanation: Executes part of the module’s workflow.

Line 387: ‘   bundle(‘
Explanation: Executes part of the module’s workflow.

Line 388: ‘   {‘
Explanation: Executes part of the module’s workflow.

Line 389: ‘   "resourceType": "Observation",‘
Explanation: Executes part of the module’s workflow.

Line 390: ‘   "id": "o1",‘
Explanation: Executes part of the module’s workflow.

Line 391: ‘   "subject": {"reference": "Patient/one"},‘
Explanation: Executes part of the module’s workflow.

Line 392: ‘   "encounter": {"reference": "Encounter/e1"},‘
Explanation: Executes part of the module’s workflow.

Line 393: ‘   }‘
Explanation: Executes part of the module’s workflow.

Line 394: ‘   )‘
Explanation: Executes part of the module’s workflow.

Line 395: ‘   ),‘
Explanation: Executes part of the module’s workflow.

Line 396: ‘   (f"{self.endpoint}/Encounter", (("_count", "50"), ("patient",
"one"))): FakeResponse(‘
Explanation: Executes part of the module’s workflow.

Line 397: ‘   bundle(‘
Explanation: Executes part of the module’s workflow.

Line 398: ‘   {‘
Explanation: Executes part of the module’s workflow.

Line 399: ‘   "resourceType": "Encounter",‘
Explanation: Executes part of the module’s workflow.

Line 400: ‘   "id": "e1",‘
Explanation: Executes part of the module’s workflow.

Line 401: ‘   "identifier": [{"value": "visit-1"}],‘
Explanation: Executes part of the module’s workflow.

Line 402: ‘   "class": {"display": "ambulatory"},‘
Explanation: Executes part of the module’s workflow.

Line 403: ‘   "period": {"start": "2024-01-01", "end":
"2024-01-02"},‘
Explanation: Executes part of the module’s workflow.

Line 404: ‘   "subject": {"reference": "Patient/one"},‘
Explanation: Executes part of the module’s workflow.

Line 405: ‘   }‘
Explanation: Executes part of the module’s workflow.

Line 406: ‘   )‘
Explanation: Executes part of the module’s workflow.

Line 407: ‘   ),‘
Explanation: Executes part of the module’s workflow.

Line 408: ‘   }‘
Explanation: Executes part of the module’s workflow.

Line 409: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 410: ‘   with tempfile.TemporaryDirectory() as temporary_directory:‘
Explanation: Controls execution flow for the surrounding operation.

Line 411: ‘   report = get_observations_and_encounters_for_patient(‘
Explanation: Assigns or computes a value used by later code.

Line 412: ‘   "one",‘
Explanation: Executes part of the module’s workflow.

Line 413: ‘   endpoint=self.endpoint,‘
Explanation: Assigns or computes a value used by later code.

Line 414: ‘   output_dir=temporary_directory,‘
Explanation: Assigns or computes a value used by later code.

Line 415: ‘   session=FakeSession(responses),‘
Explanation: Assigns or computes a value used by later code.

Line 416: ‘   )‘
Explanation: Executes part of the module’s workflow.

Line 417: ‘   database = sqlite3.connect(Path(temporary_directory) /
"fhir_resources.sqlite3")‘
Explanation: Opens a SQLite connection to the configured database.

Line 418: ‘   result = database.execute(‘
Explanation: Runs a SQL statement or schema script against SQLite.

Line 419: ‘   "SELECT o.patient_id, o.encounter_id, e.patient_id "‘
Explanation: Executes part of the module’s workflow.

Line 420: ‘   "FROM observations o JOIN encounters e ON e.encounter_id = o.encounter_id"‘
Explanation: Assigns or computes a value used by later code.

Line 421: ‘   ).fetchone()‘
Explanation: Executes part of the module’s workflow.

Line 422: ‘   encounter = database.execute(‘
Explanation: Runs a SQL statement or schema script against SQLite.

Line 423: ‘   "SELECT encounter_type, encounter_id, start, end, patient_id FROM encounters"‘
Explanation: Executes part of the module’s workflow.

Line 424: ‘   ).fetchone()‘
Explanation: Executes part of the module’s workflow.

Line 425: ‘   database.close()‘
Explanation: Executes part of the module’s workflow.

Line 426: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 427: ‘   self.assertFalse(report.failures)‘
Explanation: Test assertion verifying the expected behavior.

Line 428: ‘   self.assertEqual(result, ("one", "visit-1", "one"))‘
Explanation: Test assertion verifying the expected behavior.

Line 429: ‘   self.assertEqual(encounter, ("ambulatory", "visit-1", "2024-01-01", "2024-01-02", "one"))‘
Explanation: Test assertion verifying the expected behavior.

Line 430: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 431: ‘   def test_patient_observation_function_does_not_request_or_store_encounters(self):‘
Explanation: Defines the test_patient_observation_function_does_not_request_or_store_encounters callable.

Line 432: ‘   patient_url = f"{self.endpoint}/Patient?_id=one"‘
Explanation: Assigns or computes a value used by later code.

Line 433: ‘   responses = {‘
Explanation: Assigns or computes a value used by later code.

Line 434: ‘   (patient_url, (("_count", "50"), ("_id", "one"))): FakeResponse(‘
Explanation: Executes part of the module’s workflow.

Line 435: ‘   bundle({"resourceType": "Patient", "id": "one"})‘
Explanation: Executes part of the module’s workflow.

Line 436: ‘   ),‘
Explanation: Executes part of the module’s workflow.

Line 437: ‘   (f"{self.endpoint}/Observation", (("_count", "50"), ("patient", "one"))): FakeResponse(‘

Explanation: Executes part of the module’s workflow.

Line 438: ‘   bundle(‘
Explanation: Executes part of the module’s workflow.

Line 439: ‘   {‘
Explanation: Executes part of the module’s workflow.

Line 440: ‘   "resourceType": "Observation",‘
Explanation: Executes part of the module’s workflow.

Line 441: ‘   "id": "o1",‘
Explanation: Executes part of the module’s workflow.

Line 442: ‘   "subject": {"reference": "Patient/one"},‘
Explanation: Executes part of the module’s workflow.

Line 443: ‘   "encounter": {"reference": "Encounter/e1"},‘
Explanation: Executes part of the module’s workflow.

Line 444: ‘   }‘
Explanation: Executes part of the module’s workflow.

Line 445: ‘   )‘
Explanation: Executes part of the module’s workflow.

Line 446: ‘   ),‘
Explanation: Executes part of the module’s workflow.

Line 447: ‘   }‘
Explanation: Executes part of the module’s workflow.

Line 448: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 449: ‘   with tempfile.TemporaryDirectory() as temporary_directory:‘
Explanation: Controls execution flow for the surrounding operation.

Line 450: ‘   session = FakeSession(responses)‘
Explanation: Assigns or computes a value used by later code.

Line 451: ‘   report = get_observations_for_patient(‘
Explanation: Assigns or computes a value used by later code.

Line 452: ‘   "one",‘
Explanation: Executes part of the module’s workflow.

Line 453: ‘   endpoint=self.endpoint,‘
Explanation: Assigns or computes a value used by later code.

Line 454: ‘   output_dir=temporary_directory,‘
Explanation: Assigns or computes a value used by later code.

Line 455: ‘   session=session,‘
Explanation: Assigns or computes a value used by later code.

Line 456: ‘   )‘
Explanation: Executes part of the module’s workflow.

Line 457: ‘   database = sqlite3.connect(Path(temporary_directory) /
"fhir_resources.sqlite3")‘
Explanation: Opens a SQLite connection to the configured database.

Line 458: ‘   observation_count = database.execute("SELECT COUNT(*) FROM observations").fetchone()[0]‘
Explanation: Runs a SQL statement or schema script against SQLite.

Line 459: ‘   encounter_count = database.execute("SELECT COUNT(*) FROM encounters").fetchone()[0]‘
Explanation: Runs a SQL statement or schema script against SQLite.

Line 460: ‘   database.close()‘
Explanation: Executes part of the module’s workflow.

Line 461: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 462: ‘   self.assertFalse(report.failures)‘
Explanation: Test assertion verifying the expected behavior.

Line 463: ‘   self.assertEqual(observation_count, 1)‘
Explanation: Test assertion verifying the expected behavior.

Line 464: ‘   self.assertEqual(encounter_count, 0)‘
Explanation: Test assertion verifying the expected behavior.

Line 465: ‘   self.assertFalse(any(call[0].endswith("/Encounter") for call in session.calls))‘
Explanation: Test assertion verifying the expected behavior.

Line 466: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 467: ‘   def test_patient_encounter_function_does_not_request_or_store_observations(self):‘
Explanation: Defines the test_patient_encounter_function_does_not_request_or_store_observations callable.

Line 468: ‘   patient_url = f"{self.endpoint}/Patient?_id=one"‘
Explanation: Assigns or computes a value used by later code.

Line 469: ‘   responses = {‘
Explanation: Assigns or computes a value used by later code.

Line 470: ‘   (patient_url, (("_count", "50"), ("_id", "one"))): FakeResponse(‘
Explanation: Executes part of the module’s workflow.

Line 471: ‘   bundle({"resourceType": "Patient", "id": "one"})‘
Explanation: Executes part of the module’s workflow.

Line 472: ‘   ),‘
Explanation: Executes part of the module’s workflow.

Line 473: ‘   (f"{self.endpoint}/Encounter", (("_count", "50"), ("patient", "one"))): FakeResponse(‘
Explanation: Executes part of the module’s workflow.

Line 474: ‘   bundle(‘
Explanation: Executes part of the module’s workflow.

Line 475: ‘   {‘
Explanation: Executes part of the module’s workflow.

Line 476: ‘   "resourceType": "Encounter",‘

Explanation: Executes part of the module’s workflow.

Line 477: ‘   "id": "e1",‘
Explanation: Executes part of the module’s workflow.

Line 478: ‘   "identifier": [{"value": "visit-1"}],‘
Explanation: Executes part of the module’s workflow.

Line 479: ‘   "subject": {"reference": "Patient/one"},‘
Explanation: Executes part of the module’s workflow.

Line 480: ‘   }‘
Explanation: Executes part of the module’s workflow.

Line 481: ‘   )‘
Explanation: Executes part of the module’s workflow.

Line 482: ‘   ),‘
Explanation: Executes part of the module’s workflow.

Line 483: ‘   }‘
Explanation: Executes part of the module’s workflow.

Line 484: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 485: ‘   with tempfile.TemporaryDirectory() as temporary_directory:‘
Explanation: Controls execution flow for the surrounding operation.

Line 486: ‘   session = FakeSession(responses)‘
Explanation: Assigns or computes a value used by later code.

Line 487: ‘   report = get_encounters_for_patient(‘
Explanation: Assigns or computes a value used by later code.

Line 488: ‘   "one",‘
Explanation: Executes part of the module’s workflow.

Line 489: ‘   endpoint=self.endpoint,‘
Explanation: Assigns or computes a value used by later code.

Line 490: ‘   output_dir=temporary_directory,‘
Explanation: Assigns or computes a value used by later code.

Line 491: ‘   session=session,‘
Explanation: Assigns or computes a value used by later code.

Line 492: ‘   )‘
Explanation: Executes part of the module’s workflow.

Line 493: ‘   database = sqlite3.connect(Path(temporary_directory) /
"fhir_resources.sqlite3")‘
Explanation: Opens a SQLite connection to the configured database.

Line 494: ‘   observation_count = database.execute("SELECT COUNT(*) FROM
observations").fetchone()[0]‘
Explanation: Runs a SQL statement or schema script against SQLite.

Line 495: ‘   encounter_count = database.execute("SELECT COUNT(*) FROM
encounters").fetchone()[0]‘
Explanation: Runs a SQL statement or schema script against SQLite.

Line 496: ‘   database.close()‘
Explanation: Executes part of the module’s workflow.

Line 497: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 498: ‘   self.assertFalse(report.failures)‘
Explanation: Test assertion verifying the expected behavior.

Line 499: ‘   self.assertEqual(observation_count, 0)‘
Explanation: Test assertion verifying the expected behavior.

Line 500: ‘   self.assertEqual(encounter_count, 1)‘
Explanation: Test assertion verifying the expected behavior.

Line 501: ‘   self.assertFalse(any(call[0].endswith("/Observation") for call in session.calls))‘
Explanation: Test assertion verifying the expected behavior.

Line 502: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 503: ‘   def test_patient_condition_function_does_not_request_observations_or_encounters(self):‘
Explanation: Defines the test_patient_condition_function_does_not_request_observations_or_encounters callable.

Line 504: ‘   patient_url = f"{self.endpoint}/Patient"‘
Explanation: Assigns or computes a value used by later code.

Line 505: ‘   responses = {‘
Explanation: Assigns or computes a value used by later code.

Line 506: ‘   (patient_url, (("_count", "50"), ("_id", "one"))): FakeResponse(‘
Explanation: Executes part of the module’s workflow.

Line 507: ‘   bundle({"resourceType": "Patient", "id": "one"})‘
Explanation: Executes part of the module’s workflow.

Line 508: ‘   ),‘
Explanation: Executes part of the module’s workflow.

Line 509: ‘   (f"{self.endpoint}/Condition", (("_count", "50"), ("patient", "one"))): FakeResponse(‘
Explanation: Executes part of the module’s workflow.

Line 510: ‘   bundle({"resourceType": "Condition", "id": "c1", "subject": {"reference": "Patient/one"}})‘
Explanation: Executes part of the module’s workflow.

Line 511: ‘   ),‘
Explanation: Executes part of the module’s workflow.

Line 512: ‘   }‘
Explanation: Executes part of the module’s workflow.

Line 513: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 514: ‘   with tempfile.TemporaryDirectory() as temporary_directory:‘
Explanation: Controls execution flow for the surrounding operation.

Line 515: ‘   session = FakeSession(responses)‘
Explanation: Assigns or computes a value used by later code.

Line 516: ‘   report = get_conditions_for_patient(‘
Explanation: Assigns or computes a value used by later code.

Line 517: ‘   "one", endpoint=self.endpoint,
output_dir=temporary_directory, session=session‘
Explanation: Assigns or computes a value used by later code.

Line 518: ‘   )‘
Explanation: Executes part of the module’s workflow.

Line 519: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 520: ‘   self.assertFalse(report.failures)‘
Explanation: Test assertion verifying the expected behavior.

Line 521: ‘   self.assertEqual(report.resources["Condition"], 1)‘
Explanation: Test assertion verifying the expected behavior.

Line 522: ‘   self.assertFalse(any(call[0].endswith("/Observation") for call in
session.calls))‘
Explanation: Test assertion verifying the expected behavior.

Line 523: ‘   self.assertFalse(any(call[0].endswith("/Encounter") for call in
session.calls))‘
Explanation: Test assertion verifying the expected behavior.

Line 524: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 525: ‘   def
test_patient_related_function_combines_condition_observation_and_encounter(self):‘
Explanation: Defines the
test_patient_related_function_combines_condition_observation_and_encounter callable.

Line 526: ‘   patient_url = f"{self.endpoint}/Patient"‘
Explanation: Assigns or computes a value used by later code.

Line 527: ‘   responses = {‘
Explanation: Assigns or computes a value used by later code.

Line 528: ‘   (patient_url, (("_count", "50"), ("_id", "one"))): FakeResponse(‘
Explanation: Executes part of the module’s workflow.

Line 529: ‘   bundle({"resourceType": "Patient", "id": "one"})‘
Explanation: Executes part of the module’s workflow.

Line 530: ‘   ),‘
Explanation: Executes part of the module’s workflow.

Line 531: ‘   (f"{self.endpoint}/Condition", (("_count", "50"), ("patient",
"one"))): FakeResponse(bundle()),‘
Explanation: Executes part of the module’s workflow.

Line 532: ‘   (f"{self.endpoint}/Observation", (("_count", "50"), ("patient",
"one"))): FakeResponse(bundle()),‘
Explanation: Executes part of the module’s workflow.

Line 533: ‘   (f"{self.endpoint}/Encounter", (("_count", "50"), ("patient",

"one"))): FakeResponse(bundle()),‘
Explanation: Executes part of the module’s workflow.

Line 534: ‘   }‘
Explanation: Executes part of the module’s workflow.

Line 535: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 536: ‘   with tempfile.TemporaryDirectory() as temporary_directory:‘
Explanation: Controls execution flow for the surrounding operation.

Line 537: ‘   session = FakeSession(responses)‘
Explanation: Assigns or computes a value used by later code.

Line 538: ‘   report = get_related_for_patient(‘
Explanation: Assigns or computes a value used by later code.

Line 539: ‘   "one",‘
Explanation: Executes part of the module’s workflow.

Line 540: ‘   ("Condition", "Observation", "Encounter"),‘
Explanation: Executes part of the module’s workflow.

Line 541: ‘   endpoint=self.endpoint,‘
Explanation: Assigns or computes a value used by later code.

Line 542: ‘   output_dir=temporary_directory,‘
Explanation: Assigns or computes a value used by later code.

Line 543: ‘   session=session,‘
Explanation: Assigns or computes a value used by later code.

Line 544: ‘   )‘
Explanation: Executes part of the module’s workflow.

Line 545: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 546: ‘   self.assertFalse(report.failures)‘
Explanation: Test assertion verifying the expected behavior.

Line 547: ‘   self.assertEqual(‘
Explanation: Test assertion verifying the expected behavior.

Line 548: ‘   {call[0].rsplit("/", 1)[-1] for call in session.calls},‘
Explanation: Executes part of the module’s workflow.

Line 549: ‘   {"Patient", "Condition", "Observation", "Encounter"},‘
Explanation: Executes part of the module’s workflow.

Line 550: ‘   )‘
Explanation: Executes part of the module’s workflow.

Line 551: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 552: ‘   def test_condition_database_uses_requested_structured_columns(self):‘
Explanation: Defines the test_condition_database_uses_requested_structured_columns
callable.

Line 553: ‘   responses = self.response_map()‘

Explanation: Assigns or computes a value used by later code.

Line 554: ‘   responses[(f"{self.endpoint}/Condition", (("_count", "50"), ("patient", "one")))] = FakeResponse(‘
Explanation: Assigns or computes a value used by later code.

Line 555: ‘   bundle(‘
Explanation: Executes part of the module’s workflow.

Line 556: ‘   {‘
Explanation: Executes part of the module’s workflow.

Line 557: ‘   "resourceType": "Condition",‘
Explanation: Executes part of the module’s workflow.

Line 558: ‘   "id": "c1",‘
Explanation: Executes part of the module’s workflow.

Line 559: ‘   "subject": {"reference": "Patient/one"},‘
Explanation: Executes part of the module’s workflow.

Line 560: ‘   "encounter": {"reference": "Encounter/e1"},‘
Explanation: Executes part of the module’s workflow.

Line 561: ‘   "clinicalStatus": {"coding": [{"code": "active"}]},‘
Explanation: Executes part of the module’s workflow.

Line 562: ‘   "verificationStatus": {"coding": [{"code": "confirmed"}]},‘
Explanation: Executes part of the module’s workflow.

Line 563: ‘   "category": [{"coding": [{"code": "problem-list-item"}]}],‘
Explanation: Executes part of the module’s workflow.

Line 564: ‘   "code": {"coding": [{"code": "44054006", "display": "Diabetes mellitus type 2"}]},‘
Explanation: Executes part of the module’s workflow.

Line 565: ‘   "onsetDateTime": "2024-01-01",‘
Explanation: Executes part of the module’s workflow.

Line 566: ‘   "recordedDate": "2024-01-02",‘
Explanation: Executes part of the module’s workflow.

Line 567: ‘   }‘
Explanation: Executes part of the module’s workflow.

Line 568: ‘   )‘
Explanation: Executes part of the module’s workflow.

Line 569: ‘   )‘
Explanation: Executes part of the module’s workflow.

Line 570: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 571: ‘   with tempfile.TemporaryDirectory() as temporary_directory:‘
Explanation: Controls execution flow for the surrounding operation.

Line 572: ‘   FHIRRetriever(self.endpoint, temporary_directory, session=FakeSession(responses)).retrieve()‘

Explanation: Assigns or computes a value used by later code.

Line 573: ‘   database = sqlite3.connect(Path(temporary_directory) / "fhir_resources.sqlite3")‘
Explanation: Opens a SQLite connection to the configured database.

Line 574: ‘   condition = database.execute(‘
Explanation: Runs a SQL statement or schema script against SQLite.

Line 575: ‘   "SELECT clinicalStatus, verificationStatus, category, condition, condition_code, "‘
Explanation: Executes part of the module’s workflow.

Line 576: ‘   "onsetDateTime, recorded_Date FROM conditions"‘
Explanation: Executes part of the module’s workflow.

Line 577: ‘   ).fetchone()‘
Explanation: Executes part of the module’s workflow.

Line 578: ‘   database.close()‘
Explanation: Executes part of the module’s workflow.

Line 579: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 580: ‘   self.assertEqual(‘
Explanation: Test assertion verifying the expected behavior.

Line 581: ‘   condition[:5],‘
Explanation: Executes part of the module’s workflow.

Line 582: ‘   ("active", "confirmed", "problem-list-item", "Diabetes mellitus type 2", "44054006"),‘
Explanation: Executes part of the module’s workflow.

Line 583: ‘   )‘
Explanation: Executes part of the module’s workflow.

Line 584: ‘   self.assertNotEqual(condition[5:], ("2024-01-01", "2024-01-02"))‘
Explanation: Test assertion verifying the expected behavior.

Line 585: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 586: ‘   def test_database_tables_are_exported_as_schema_aligned_csv_files(self):‘
Explanation: Defines the test_database_tables_are_exported_as_schema_aligned_csv_files callable.

Line 587: ‘   patient_url = f"{self.endpoint}/Patient?_id=one"‘
Explanation: Assigns or computes a value used by later code.

Line 588: ‘   responses = {‘
Explanation: Assigns or computes a value used by later code.

Line 589: ‘   (patient_url, (("_count", "50"), ("_id", "one"))): FakeResponse(‘
Explanation: Executes part of the module’s workflow.

Line 590: ‘   bundle(‘
Explanation: Executes part of the module’s workflow.

Line 591: ‘   {‘
Explanation: Executes part of the module’s workflow.

Line 592: ‘   "resourceType": "Patient",‘
Explanation: Executes part of the module’s workflow.

Line 593: ‘   "id": "one",‘
Explanation: Executes part of the module’s workflow.

Line 594: ‘   "name": [{"family": "Doe", "given": ["Jane"]}],‘
Explanation: Executes part of the module’s workflow.

Line 595: ‘   "gender": "female",‘
Explanation: Executes part of the module’s workflow.

Line 596: ‘   "birthDate": "1980-01-02",‘
Explanation: Executes part of the module’s workflow.

Line 597: ‘   }‘
Explanation: Executes part of the module’s workflow.

Line 598: ‘   )‘
Explanation: Executes part of the module’s workflow.

Line 599: ‘   )‘
Explanation: Executes part of the module’s workflow.

Line 600: ‘   }‘
Explanation: Executes part of the module’s workflow.

Line 601: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 602: ‘   with tempfile.TemporaryDirectory() as temporary_directory:‘
Explanation: Controls execution flow for the surrounding operation.

Line 603: ‘   get_patient(‘
Explanation: Executes part of the module’s workflow.

Line 604: ‘   "one",‘
Explanation: Executes part of the module’s workflow.

Line 605: ‘   endpoint=self.endpoint,‘
Explanation: Assigns or computes a value used by later code.

Line 606: ‘   output_dir=temporary_directory,‘
Explanation: Assigns or computes a value used by later code.

Line 607: ‘   session=FakeSession(responses),‘
Explanation: Assigns or computes a value used by later code.

Line 608: ‘   )‘
Explanation: Executes part of the module’s workflow.

Line 609: ‘   with (Path(temporary_directory) /
"patients.csv").open(newline="") as file:‘
Explanation: Controls execution flow for the surrounding operation.

Line 610: ‘   patient_rows = list(csv.reader(file))‘
Explanation: Assigns or computes a value used by later code.

Line 611: ‘   with (Path(temporary_directory) /
"observations.csv").open(newline="") as file:‘
Explanation: Controls execution flow for the surrounding operation.

Line 612: ‘   observation_rows = list(csv.reader(file))‘
Explanation: Assigns or computes a value used by later code.

Line 613: ‘   with (Path(temporary_directory) /
"conditions.csv").open(newline="") as file:‘
Explanation: Controls execution flow for the surrounding operation.

Line 614: ‘   condition_rows = list(csv.reader(file))‘
Explanation: Assigns or computes a value used by later code.

Line 615: ‘   with (Path(temporary_directory) /
"encounters.csv").open(newline="") as file:‘
Explanation: Controls execution flow for the surrounding operation.

Line 616: ‘   encounter_rows = list(csv.reader(file))‘
Explanation: Assigns or computes a value used by later code.

Line 617: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 618: ‘   self.assertEqual(‘
Explanation: Test assertion verifying the expected behavior.

Line 619: ‘   patient_rows[0],‘
Explanation: Executes part of the module’s workflow.

Line 620: ‘   ["patient_id", "family_name", "given_name", "gender",
"birth_date", "date_shift_days"],‘
Explanation: Executes part of the module’s workflow.

Line 621: ‘   )‘
Explanation: Executes part of the module’s workflow.

Line 622: ‘   self.assertEqual(patient_rows[1][:5], ["one", "Doe", "Jane",
"female", "1980-01-02"])‘
Explanation: Test assertion verifying the expected behavior.

Line 623: ‘   self.assertEqual(‘
Explanation: Test assertion verifying the expected behavior.

Line 624: ‘   condition_rows,‘
Explanation: Executes part of the module’s workflow.

Line 625: ‘   [["clinicalStatus", "verificationStatus", "category",
"condition", "condition_code", "patient_id", "encounter_id", "onsetDateTime",
"recorded_Date"]],‘
Explanation: Executes part of the module’s workflow.

Line 626: ‘   )‘
Explanation: Executes part of the module’s workflow.

Line 627: ‘   self.assertEqual(observation_rows, [["observation_id", "patient_id",
"encounter_id", "observation_type", "observation_code", "observation_subtype",
"effective_date_time", "issued", "value", "unit", "value_code"]])‘
Explanation: Test assertion verifying the expected behavior.

Line 628: ‘   self.assertEqual(encounter_rows, [["encounter_type", "encounter_id",
"start", "end", "patient_id"]])‘
Explanation: Test assertion verifying the expected behavior.

Line 629: ‘<blank>‘

Explanation: Blank line used to separate logical sections.

Line 630: ‘<blank>‘
Explanation: Blank line used to separate logical sections.

Line 631: ‘if __name__ == "__main__":‘
Explanation: Controls execution flow for the surrounding operation.

Line 632: ‘   unittest.main()‘
Explanation: Executes part of the module’s workflow.